

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

A Mechanically Verified Garbage Collector for OCaml

A Thesis

Submitted by

SHEERA SHAMSU

For the award of the degree

Of

DOCTOR OF PHILOSOPHY

June 2026



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

A Mechanically Verified Garbage Collector for OCaml

A Thesis

Submitted by

SHEERA SHAMSU

For the award of the degree

Of

DOCTOR OF PHILOSOPHY

June 2026

© 2026 Indian Institute of Technology Madras

Even though you're not equipped,

Keep searching:

Equipment isn't necessary on the way to the

Sustainer.

Whoever you see engaged in search,

Become her friend and cast your head in

front of her,

For choosing to be a neighbour of seekers,

You become one yourself;

Protected by conquerors,

You will, yourself, learn to conquer.

If an ant seeks the rank of Solomon,

Don't smile contemptuously upon its quest.

Everything you possess of skill, and wealth,

and handicraft,

Wasn't it first merely a thought and a quest?

– Rumi, *"The Quest"*

To my father, who held everything together so I could reach this moment, and to my mother, whose quiet strength and unwavering love sustained me.

To my children, Diyaa and Eshaal, whose love carried me through.

THESIS CERTIFICATE

This is to undertake that the thesis titled **A MECHANICALLY VERIFIED GARBAGE COLLECTOR FOR OCAML**, submitted by me to the Indian Institute of Technology Madras, for the award of **Doctor of Philosophy**, is a bona fide record of the research work done by me under the supervision of **Dr. KC Sivaramakrishnan, Dr. Kartik Nagar**. The contents of this thesis, in full or in parts, have not been submitted to any other institute or university for the award of any degree or diploma.

Chennai 600036 **Sheera Shamsu**

Date: April 2026

Dr. KC Sivaramakrishnan
Research advisor
Assistant Professor
Department of Computer Science and Engineering
IIT Madras

Dr. Kartik Nagar
Research co-advisor
Assistant Professor
Department of Computer Science and Engineering
IIT Madras

© 2026 Indian Institute of Technology Madras

LIST OF PUBLICATIONS

I. REFEREED JOURNALS BASED ON THESIS

Shamsu, S., Kafle, D., Maroo, D., Nagar, K., Bhargavan, K., and Sivaramakrishnan, K. C. 2025. *A Mechanically Verified Garbage Collector for OCaml*. Journal of Automated Reasoning 69, 11 (May 2025), 43 pages. https://doi.org/10.1007/s10817-025-09721-0

II. PUBLISHED SOFTWARE

Verified OCaml GC: A Mechanically Verified Garbage Collector for OCaml. URL: https://github.com/fplaunchpad/verified_ocaml_gc/tree/main

ACKNOWLEDGEMENTS

Everything you see has its roots in the unseen world. The forms may change, yet the essence remains the same. Every wondrous sight will vanish, every sweet word will fade. But do not be disheartened, the source they come from is eternal.

– Rumi, *"A Garden beyond Paradise"*

This has been a long and deeply transformational journey. What began as an academic pursuit gradually became something far more personal, shaping the person I am today. I am grateful to everyone who walked alongside me, whether for a brief moment or throughout.

The most profound gratitude of this entire journey belongs to my advisors, Dr. KC Sivaramakrishnan and Dr. Kartik Nagar, whose support has meant more to me than words can fully express.

KC, you believed in my capabilities and entrusted me with a problem that is both challenging and deeply meaningful. The development of an efficient, mechanically verified garbage collector for OCaml is the kind of work that extends far beyond the pages of a thesis, and that vision of consequence came from you. The countless nights you spent reviewing my work ahead of deadlines, and your tireless involvement in seeing this project through to its end, moved me deeply. Your conviction that a PhD should produce something reusable, something that does not merely gather dust on a shelf, has fundamentally shaped how I think about research and its purpose. I also carry deep admiration for the way you communicate. Your ability to take the most complex concepts and present them with grace, clarity, and elegance is something I will spend years aspiring toward. Your tutorial on the OCaml language and its runtime internals is a gift I will carry long after this PhD.

I feel truly and humbly fortunate to have worked with Kartik sir. He is a gentle and grounded human being, a patient teacher, and an exceptional advisor in the truest sense. The foundations of formal verification that I now hold with confidence were built steadily and carefully under his guidance. He taught me not merely the techniques, but the deeper importance of rigour, of being precise in formalisms and always careful in reasoning. Some of my most treasured moments during this PhD were the long whiteboard sessions with him, where hours would pass in deep and unhurried discussion, slowly untangling the knots in my thinking and bringing light to problems I had wrestled with in the dark. His intricate and probing way of thinking, always asking the right question at the right moment, offered guidance that no textbook could have provided. I am sincerely grateful for his patience, and for the genuine care he brought to every conversation we shared.

I extend my sincere gratitude to the DC committee members Dr. Sukhendu Das, Dr. Chester Rebeiro, Dr. Rupesh Nasre, and Dr. Nitin Chandrachoodan for their prompt review and guidance.

My gratitude also goes to the CS Department at IIT Madras, Head of the Department (CS), the CS office, the IIT Madras QIP Centre, the Government of Kerala, AICTE, and the members of the interview board for selecting me for this extraordinary institution. IIT Madras has been more than a place of learning. It has been a formative period that shaped me into a more thoughtful researcher, a stronger human being, and someone capable of facing life's most difficult challenges with resilience and clarity.

I am deeply indebted to Dr. Jayalal Sharma and Dr. Narayanaswamy for their insightful talks and generous support. And during the weakest and most vulnerable point of my PhD journey, one professor extended his hand to me not as an academic but as a compassionate human being. Dr. Rupesh Nasre, you are an exceptional researcher, but what touched me most deeply was your willingness to listen, without judgement, without conditions. Your very presence is a quiet comfort to every student who walks through your door.

Dr. Krishna Nandivada's Analysis of Parallel Programs course was both intellectually stimulating and transformational in shaping my thinking. His exams and assignments taught me not just the subject, but the discipline of approaching problems with rigour and care. Among the courses that brought me genuine joy amidst the pressures of PhD life were the Linear Algebra course by Dr. Mitesh Khapra and Advanced Algorithms by Dr. Raghavendra Rao. Both were, in every sense, a blessing.

Dr. Karthikeyan Bhargavan, a senior research scientist at Inria Paris, appeared in my life at a moment when I was struggling greatly with the Low* tool. His patient guidance helped me not only master Low* but also develop a deeper, proof-oriented approach to design. I will forever be grateful for his time and his belief in my work.

I am sincerely indebted to Dipesh Kafle and Dhruv Maroo for their kind help in setting up the environments required to test my work.

Dr. Nikhil Swamy, a Partner Researcher at Microsoft, was kind enough to review my work and offer me an internship to develop applications using Pulse, a DSL within F* that supports concurrent separation logic. Nik, I have long admired you, whether it is your way of communicating, your way of building tools, or the expressive richness you bring to everything you create. It is precisely that expressiveness that drew me so deeply to F*. I am equally grateful to Dr. Aseem Rastogi, Principal Researcher at Microsoft, for mentoring me so thoughtfully on programming, presentations, and the finer aspects of academic communication. Thank you both for your support and belief in me.

I am grateful to the editors and reviewers of the *Journal of Automated Reasoning* for their thorough and constructive engagement with my work. Their depth of review truly strengthened my contribution. I also owe a debt of gratitude to the OCaml Software Foundation and Dr. Gabriel Scherer, Permanent Researcher at Inria and Université Paris Cité, for funding my participation in ICFP/OOPSLA 2025 and my talk at the OCaml Workshop, an opportunity I will always treasure.

My dear friend Alex Rozanov helped me with mathematical proof styles, enabling me to organise my proofs with clarity and elegance. I also extend my gratitude to Dr. Aymeric Fromherz, Researcher at Inria, and the many members of the F* community who guided me with patience and generosity during my early days of learning.

My lifelong guru, Dr. Murali Krishnan, Professor at NIT Calicut, showed me the path toward a meaningful and fulfilling PhD journey. His constant support helped me discover my strengths and honestly confront the areas where I needed to grow. He is like a candle in the darkness for many students seeking direction. I am also deeply grateful to Dr. Vineeth Paleri, my first compiler course teacher, from whom I learned the structured, precise thinking essential not only in programming but in approaching any real-world problem with clarity.

My father was my greatest foundation. Even as he lay bedridden in his final days, his love and strength never left me. When life delivered its heaviest blows, it was he who shielded me and lovingly cared for my daughter Diyaa in my absence. Though he is no longer with us, his presence lives on in every page of this thesis. My mother, too, has been an unshakeable pillar, fulfilling our financial needs with quiet dignity when we needed her most. My daughter Diyaa, who stayed away from me through much of this journey, bore that distance with a maturity and grace far beyond her years, never once making me feel the weight of my absence. And little Eshaal, who has grown alongside this PhD journey from the age of four to eight, understood and supported me in her own innocent way through every step, her laughter and love a quiet strength I carried everywhere. I am also grateful to my brother Shiya Shamsu, who supported me financially during a difficult four-month period of half-pay leave, without hesitation and with nothing but love. I acknowledge Eshaal's father for contributing, in his own way, to my realization of the strength and independence required to carry out this journey.

I express my sincere gratitude to TKM College of Engineering and the TKM Trust

for their unwavering support and continuous encouragement throughout this journey. I am grateful to Janab T.K. Shahal Hassan Musaliar, Chairman of the TKM College Trust, for his visionary leadership and for fostering an institution that made this journey possible. I am also grateful to our current Principal Dr. Sadiq A Mehrab for his support and encouragement. I am especially grateful to former Principal Dr. Ayoob S for his guidance and encouragement to pursue my PhD without delay. He has been a constant source of support and a well-wisher throughout. I owe a deep debt of gratitude to Dr. Shahul Hameed, also a former Principal, for his empathetic and deeply human response to my genuine concerns at a time when it mattered most. My sincere thanks also go to Dr. Muhammed Siddik A of the civil engineering department for his guidance and compassionate words when I needed it most, and to Prof. Shaleena of the electrical engineering department for her reliable support during a time when I was severely weakened, both physically and mentally. Prof. Natheera Beevi M., Head of the MCA Department, has been a constant well-wisher who stood firmly by my side whenever I faced difficulties, and I am truly grateful for her sincerity and care. I am equally grateful to Shailaja ma'am, a retired computer programmer who was like a mother to me, always ready to listen and offer warm comfort whenever I needed it most. I am also indebted to my colleagues Prof. Thowsif, Dr. Sofia, Dr. Mohammed Siyad, and Prof. Sharos for kindly assisting me with the formalities for my bond papers. I thank all the faculty and staff members of the MCA Department, TKM College of Engineering, for their support. I also extend my sincere thanks to the office staff of TKM College of Engineering for their assistance and cooperation.

And how can I forget my dear students, especially the 2022–24 and 2023–25 batches, who celebrated every small achievement of mine with such genuine pride and joy? Your enthusiasm was a source of energy and warmth that kept me going.

My friends Sandeep and Shaheem stood by me during my financial struggles, without hesitation or condition. They are the kind of friends who make me believe in the goodness

of people. I am also grateful to Dr. Ezhudheen of Government Engineering College, Thrissur, for the many insightful discussions that have enriched my thinking. My friend Sudha Parimala's calm and gentle nature, and her willingness to spend hours listening to my fears and struggles with patience, have meant a great deal to me. I would also like to acknowledge Shashank Shekar Dubey, who, despite the short time we had together, offered comfort through his words when I needed it most.

I am grateful, from the depths of my heart, to Laila, our caretaker, but in truth a quiet pillar of strength in my home. She cares for my children, cooks, manages the household, and seems to know, without being told, when I need a glass of lemon juice or what food will bring comfort on a difficult day. Her presence has been one of the most grounding blessings of this journey.

Finally, my PACER friends, Aman, Ramya, Vimala, Nihaal, Omkaar, Anju, and all fellow PACERs, made me feel embraced and supported throughout.

If I have missed any name, please know that the omission does not diminish my gratitude. Every person who touched this journey, whether directly or indirectly, knowingly or unknowingly, has my deepest and most sincere thanks.

Above all, I am grateful to my Creator for weaving support into the very fabric of my existence, for placing the right souls at the right moments along my path, and for reminding me, time and again, that every struggle was never without purpose. This journey was not merely the pursuit of a degree. It was the unfolding of a destiny, a quiet and sacred movement toward the very purpose for which I was brought into being.

- Sheera Shamsu

ABSTRACT

KEYWORDS formal verification, verified garbage collectors, verified memory management systems, mark-and-sweep garbage collection, F*, Low*, mechanized formal proofs, graph traversal proofs, abstract specifications, practical garbage collector, end-to-end garbage collector verification, verification of OCaml garbage collectors, OCaml runtime, graph reachability, depth-first search verification, layered verification architecture, memory safety, closure representation, infix objects, C code extraction

The OCaml programming language finds application across diverse domains, including systems programming, web development, scientific computing, formal verification, and symbolic mathematics. OCaml is a memory-safe programming language that uses a garbage collector (GC) to free unreachable memory automatically. It features a low-latency, high-performance GC tuned for functional programming, with two generations: a minor heap collected using a copying collector and a major heap collected using an incremental mark-and-sweep collector. Alongside the intricacies of an efficient GC design, the OCaml compiler uses efficient object representations for some object classes, such as interior pointers for supporting mutually recursive functions, which further complicates the GC design. The GC is a critical component of the OCaml runtime system, and its correctness is essential for the safety of every OCaml program that runs on top of it.

In this thesis, we propose a strategy for crafting a correct, proof-oriented GC from scratch, designed to evolve with additional language features. Our approach neatly separates abstract GC correctness from OCaml-specific GC correctness, offering the ability to integrate further GC optimisations while preserving the core abstract correctness guarantees. The correctness specifications are based on abstract graph reachability, allowing us to describe GC correctness without referring to implementation details, and making them reusable across different collector designs.

As an initial step to demonstrate the viability of our approach, we developed a verified stop-the-world mark-and-sweep GC for OCaml 4.14.1. We adopt a layered verification architecture that descends from abstract graph reachability, through an OCaml-specific functional GC layer, down to memory-safe executable C code. The abstract layer establishes the correctness of a depth-first search algorithm over graphs using inductively defined reachability. The functional layer proves that the GC correctly identifies and preserves all reachable objects while reclaiming unreachable ones, operating on OCaml-style objects layouts including closures and infix objects. The low-level layer verifies memory safety of the implementation, ensuring freedom from buffer overflows, use-after-free, and unsafe pointer dereferences. We further demonstrate the general applicability of the reachability-based specification framework by extending it to two additional collector designs used in the OCaml runtime: an incremental mark-and-sweep collector, which ensures correctness despite mutator-induced changes to the heap during the garbage collection (GC) cycle, and a copying collector, whose correctness is expressed through a graph isomorphism that arises from the object relocation mapping. Although these extensions are not implemented and verified, they illustrate that the same foundational specification can be applied to different collector designs without necessitating huge changes in abstract correctness specifications.

The approach is fully mechanised in F* and its low-level subset Low*. We use the KaRaMeL compiler to extract the verified Low* implementation to portable C, and integrate the extracted code with the OCaml 4.14.1 runtime. The verification comprises approximately 39,000 lines of F*/Low* code and proofs. Our GC is evaluated against the standard OCaml GC and the Boehm–Demers–Weiser conservative GC on a range of benchmarks from the Computer Language Benchmarks Game as well as larger real-world OCaml programs. The experimental results show that the verified GC is competitive with the standard OCaml GC on many workloads and consistently outperforms the conservative collector, demonstrating that strong formal correctness guarantees need not come at an unacceptable performance cost.

CONTENTS

Page

ACKNOWLEDGEMENTS i

ABSTRACT vii

LIST OF TABLES xiii

LIST OF FIGURES xv

CHAPTER 1 INTRODUCTION 1

1.1 Introduction . . . 2
1.2 Memory Management Bugs . . . 3
1.3 Automatic Memory Management through Garbage Collectors . . . 5
1.3.1 Some real GC bugs . . . 5
1.4 OCaml GC As A Verification Target . . . 7
1.5 The Essential GC Correctness Properties . . . 9
1.6 Formal Verification . . . 11
1.7 Our tool of choice: F* and Low* . . . 13
1.8 Thesis Contributions and Chapter Organisation . . . 15

CHAPTER 2 BACKGROUND 19

2.1 Introduction . . . 20
2.2 Garbage Collection: A Vast Design Space . . . 20
2.2.1 Mark-and-Sweep Garbage Collectors . . . 22
2.2.2 Copying Garbage Collectors . . . 24
2.2.3 Alternative Collector Designs . . . 25
2.2.4 Incremental and Concurrent Collectors . . . 26
2.2.5 Generational Collectors . . . 27
2.3 Garbage Collection in OCaml . . . 28
2.3.1 Object Representation . . . 28
2.3.2 OCaml 4.14.1 Runtime System . . . 36
2.3.3 OCaml 5.0 Runtime System . . . 38
2.4 Foundations of Programming and Verification in F* and Low* . . . 42
2.4.1 Refinement Types and Key Language Constructs . . . 43
2.4.2 Specifications in Hoare Logic Style . . . 44
2.4.3 A Primer on Low* . . . 45
2.5 Chapter Summary . . . 47

CHAPTER 3 RELATED WORK 51

3.1 Introduction . . . 52

3.2 Garbage Collector Verification . . . 53
3.2.1 Model-Based and Abstract-Level Verification . . . 53
3.2.2 Verification of Intermediate Representations . . . 54
3.2.3 End-to-End and Code-Level Verification . . . 55
3.2.4 Concurrent Collector Verification . . . 58
3.3 Graph-Based Verification . . . 59
3.3.1 Graph Libraries in Proof Assistants . . . 59
3.3.2 Verified Graph Algorithms . . . 60
3.4 Verification of Heap-Manipulating Programs . . . 61
3.4.1 Foundations and Tooling . . . 61
3.4.2 Automated Reasoning Tools . . . 62
3.4.3 Choice of Verification Framework . . . 64
3.5 Verification of Other Memory Management Systems . . . 65
3.5.1 Verified Allocators . . . 65
3.5.2 OS Kernel Memory Management . . . 66
3.6 Chapter Summary . . . 66

CHAPTER 4 A MECHANICALLY VERIFIED GC FOR OCAML 69

4.1 Introduction . . . 70
4.2 Abstract GC Correctness . . . 72
4.3 Towards a Verified OCaml GC . . . 74
4.4 OCaml GC Specification . . . 81
4.4.1 Basic Definitions . . . 81
4.4.2 Specification for GC functions . . . 87
4.4.3 Object graph construction . . . 89
4.5 A Layered Verification Framework and Correctness Proofs for a Bare Minimal OCaml GC . . . 92
4.5.1 Layer 1 – A verified depth-first search implementation in F⋆ . . . 93
4.5.2 Layer 2 – Functional mark-and-sweep in F⋆ . . . 97
4.5.3 Layer 3 – Imperative mark-and-sweep in Low⋆ . . . 106
4.5.4 Tuning F⋆ and Z3 . . . 113
4.6 End-to-End GC Correctness . . . 115
4.7 From a Minimal to an Integration-Ready Garbage Collector . . . 117
4.7.1 The Types of Heap Manipulations induced by our GC . . . 119
4.7.2 Extending the Heap `well-formedness` . . . 120
4.7.3 Adapting the Graph Construction . . . 121
4.7.4 Extending `mark` and Its Correctness Proof . . . 122
4.7.5 Correctness of `sweep` with Coalescing . . . 123
4.8 Benchmarking the Garbage Collector . . . 126
4.8.1 Verification effort . . . 126
4.8.2 Integration with OCaml . . . 129
4.8.3 GC evaluation . . . 130
4.9 Chapter Summary . . . 133

CHAPTER 5 FUTURE WORK AND CONCLUSIONS 135

5.1 Summary of Contributions . . . 136

5.2 Challenges and Limitations . . . 136
5.3 Key Insight: The Importance of a Verification-Friendly Design . . . 137
5.4 Abstract GC Correctness: A Reusable Foundation . . . 138
5.4.1 Incremental mark-and-sweep GC . . . 139
5.4.2 Copying collector . . . 143
5.5 Future Directions . . . 145
5.6 Final Reflection . . . 147

REFERENCES **149**

CURRICULUM VITAE **159**

DOCTORAL COMMITTEE **161**

LIST OF TABLES

Table	Caption	Page
3.1	Comparison with related work on verified garbage collectors	59
4.1	Approximate Verification effort. Development effort (Dev effort) is in person-months.	127
4.2	Benchmark characteristics. Alloc, Promote and RSS are in MB.	131

LIST OF FIGURES

Figure	Caption	Page
2.1	Tricolor marking. Initially, the objects that are directly pointed to the root pointers are grey in the heap. The stack is populated with such objects first.	23
2.2	Layout of an OCaml object. The header and each field of an OCaml object occupy one word.	28
2.3	Memory layout of the `list [1; 2]` in the OCaml major heap. Each cons cell is a heap block with a header word followed by two fields. The header encodes wosize = 2 and tag = 0 (below `No_scan_tag`, so both fields are scanned by the GC). Field 0 holds the head value as a tagged immediate integer. Its LSB is 1, indicating to the GC that it is not a pointer. Value of field 1 of cons cell 1 is a word-aligned pointer to cons cell 2 (its LSB is 0). Field 1 of cons cell 2 contains the empty list `[]`, which is represented as the immediate integer 0 (LSB = 1) and it indicates the end of the list.	30
2.4	The layout of a closure object with arity 2 and environment size 2.	32
2.5	The layout of a mutually-recursive closure object layout with arities 2 and 1 and environment size 2.	33
2.6	A Low* code to swap the contents of two memory locations r_0 and r_1.	48
2.7	Extracted C code from the Low* code in Figure 2.6	49
4.1	Extract C code for the top-level stop-the-world mark-and-sweep GC function.	75
4.2	Base version of `mark` and `darken` function	77
4.3	Version of `mark` function that deals with `no_scan` objects	78
4.4	Version of `mark` and `darken` function that deals with `closure` and `infix` objects	79
4.5	Extracted C code of `sweep` function without coalescing	82
4.6	Heap state after the sweep phase, illustrating the effect of coalescing versus non-coalescing of free memory blocks	83
4.7	Extracted C code of `sweep` function with coalescing	84
4.8	The `graph` type	85
4.9	Graph reachability as an inductive predicate `reach`	85
4.10	Basic Definitions in F*	86
4.11	Algebraic properties of the constituent functions of our verified GC	88
4.12	Constructing graph from the heap	90
4.13	Mark reachability and reachable subgraph preserving lemma	91
4.14	The layering approach for GC verification	92
4.15	Functional `dfs`	94
4.16	`mark` implementation	95
4.17	Correctness specification of `dfs`	95
4.18	Specifications of `colorHeader`	98

4.19 Behavioural equivalence between `mark` and `dfs` 100
4.20 A comparison of `push_unvisited_body` and `darken_body` functions . 102
4.21 A comparison of `push_unvisited_body` and `darken_body` functions . 103
4.22 The proof flow of `mark_reachability_lemma` 104
4.23 A Low⋆ implementation of mark 107
4.24 Pre- and post-conditions of the Low⋆ `mark` implementation in Figure 4.23 108
4.25 Pre- and post-conditions of the Low⋆ `sweep` implementation 111
4.26 Pre- and post-conditions of the Low⋆ `mark_sweep_gc` implementation . 112
4.27 Overall correctness theorem for the mark-and-sweep GC 116
4.28 New well_formedness conditions 120
4.29 New liveness Specification for `sweep` 124
4.30 Normalized running time of different OCaml GCs. The numbers in the parenthesis next to the benchmark names are the running time in seconds for the baseline OCaml 4.14.1 GC. 132

CHAPTER 1

INTRODUCTION

Every program has two purposes: The one for which it was written and another for which it wasn't.

– Alan Perlis, *"Epigrams on Programming"*

1.1 INTRODUCTION

Modern society relies heavily on software that powers critical infrastructure, from financial systems and medical devices to transportation networks, communication systems, and even national security. These systems evolve over many years, growing into codebases with millions or even billions of lines of code, shaped by the work of many engineers. As these systems grow, they also become harder to maintain and extend. Engineers who take over often have to work with code that isn't fully documented, and they rarely have the chance to talk to the original developers. In these situations, they end up relying on their own assumptions and interpretation of how things are supposed to work. With tight deadlines adding extra pressure, it's easy for misunderstandings about the original intent to creep in, and these often turn into serious software bugs.

These kinds of issues are not just hypothetical; they have played out in real-world systems, sometimes with far-reaching consequences. While some bugs cause minor inconveniences such as visual glitches, incorrect calculations or unexpected behaviour, others lead to data corruption, system crashes, security breaches, or loss of life. For example, the 2024 CrowdStrike incident showed how a faulty software update could have widespread consequences, crashing around 8.5 million Windows machines and disrupting airlines, hospitals, and emergency services worldwide (Martin *et al.*, 2025). The Heartbleed vulnerability in 2014 similarly exposed private keys and sensitive user data due to a buffer over-read in OpenSSL (Durumeric *et al.*, 2014). Going further back, the Ariane 5 rocket explosion in 1996 was caused by an integer overflow, resulting in the loss of 370 million in hardware (Lions, 1996). Even earlier, the Therac-25 radiation therapy incidents in the 1980s were caused by software errors and race conditions, resulting in patients receiving massive overdoses of radiation, with fatal consequences (Leveson and Turner, 1993).

A natural response to such failures is to rely on software testing to ensure correctness. Testing remains the most widely used technique in practice, helping to uncover bugs

before deployment. However, as Dijkstra observed,

> "Program testing can be used to show the presence of bugs, but never to show their absence" (Dijkstra, 1970).

No amount of testing can exhaustively explore all possible execution paths, input combinations, and system states. As a result, critical bugs often remain hidden in rarely exercised edge cases, only surfacing in production when the consequences are most severe.

1.2 MEMORY MANAGEMENT BUGS

The most notorious classes of software bugs are the **memory management errors**. They arise from incorrect handling of program memory, such as improper allocation, access, or deallocation. These issues are often difficult to debug because their effects may appear far from where they originate, and they can have serious implications for both system reliability and security.

In languages like C and C++, where programmers have full control over memory, memory errors occur for many reasons. One common class is *use-after-free*, in which memory is accessed after it has been deallocated. In such cases, the memory may have been reassigned for other purposes, causing the program to read or write unintended data. A well-known example is the 2014 Internet Explorer vulnerability (CVE-2014-1776), which arose from a use-after-free error in the VGX.DLL component and was exploited in targeted attacks (National Vulnerability Database (NVD), 2014).

Another closely related issue is *double-free*, where the same memory region is deallocated more than once. As a result, the memory allocator's internal state may become corrupted, leading to arbitrary code execution. The 2019 WhatsApp vulnerability (CVE-2019-3568), for instance, involved memory corruption, including double-free conditions, triggered by

malformed RTP packets (Meta Platforms, 2019).

When allocated memory is no longer needed, the memory should be freed for future allocations. When a programmer forgets to free memory, it leads to a gradual exhaustion of available memory, resulting in *memory leaks* that can degrade system performance or cause crashes. For instance, a memory leak vulnerability in Cisco IOS and IOS XE Software caused affected routers to continuously consume memory until an eventual reload, resulting in a denial-of-service condition across network infrastructure (Cisco Systems, 2015).

A different class of errors involves violating memory boundaries. *Buffer overflows* occur when a program writes beyond the limits of allocated memory, corrupting adjacent data. These vulnerabilities have historically been the basis of many security exploits, including the Morris Worm (1988), which exploited a buffer overflow in the Unix finger daemon (Eichin and Rochlis, 1989).

When a program reads memory before it has been properly initialized, *uninitialized memory access* occurs. This could expose residual or sensitive data. An example is a vulnerability in Mozilla Firefox (CVE-2014-1564) caused by the use of uninitialized memory when rendering certain truncated images, which could allow an attacker to leak sensitive data across web origins or bypass security measures such as ASLR (Mozilla Foundation, 2014).

Dangling pointers arise when references to deallocated memory are retained and later used, causing undefined behaviours. In many cases, use-after-free vulnerabilities are caused by dangling-pointer issues. The program may appear to function normally until the invalid pointer is dereferenced. Recent analysis of Chromium-based browsers has identified multiple critical use-after-free vulnerabilities stemming from dangling pointers in the Blink rendering engine and V8 JavaScript engine, including CVE-2021-21206 and CVE-2021-37975, both of which were actively exploited in the wild (Google Chrome

Team, 2021).

The discussion so far shows that many of the most serious and hard-to-find software errors do not arise from complex algorithms, but from the challenges of managing memory correctly. Such issues persist even with advanced testing and analysis because programmers must manually manage memory. This involves handling memory allocation, tracking object lifetimes, and ensuring safe memory access across large, continuously evolving codebases. All these are complex tasks and need to be exercised carefully. Here, the problem is not about detecting errors; it is about the careful design of memory management so that such errors will not occur.

1.3 AUTOMATIC MEMORY MANAGEMENT THROUGH GARBAGE COLLECTORS

One natural way to address this issue is to shift the burden of manual memory management from the programmer to the runtime system. Garbage Collectors (GC) perform this job through automatic memory management. It frees the programmers from explicit intervention, thereby avoiding an entire class of errors, such as use-after-free and double-free. Languages like Java, Python, Go, OCaml, and JavaScript employ garbage collectors to eliminate entire classes of memory bugs.

But what if the GC is not correct? For example, if the collector reclaims memory that programs are still using, the GC essentially introduces use-after-free errors. At the same time, if it fails to reclaim memory no longer needed by programs, the memory will soon be exhausted, leading to a memory leak.

Beyond these extremes, other potential issues arising from incorrect garbage collection include heap-traversal faults and incorrect compaction and allocation, which could corrupt memory structures. At the same time, concurrent collectors introduce additional complexity, potentially leading to race conditions that violate core safety invariants.

1.3.1 Some real GC bugs

GCs evolved through decades of careful engineering. Still, they show subtle but critical bugs. The Z Garbage Collector (ZGC) relies on load barriers to ensure that object references remain valid even as objects are moved during execution. However, in JDK 17 and 18, a subtle but critical issue arose due to an aggressive optimisation performed by the C2 Just-In-Time (JIT) compiler. While optimising object array cloning, the compiler unintentionally skipped the load barriers that ZGC relies on. As a result, the program could read or copy object references while ZGC was relocating them, causing threads to observe outdated or invalid references (JDK-8284632, 2022).

In G1GC (JDK 11), object headers contain a mark word that encodes important metadata. The metadata stores synchronisation state and identity hash codes. During object evacuation, these mark words must be preserved and restored correctly. Due to a buggy implementation in the specific code that preserves this, objects may lose their synchronisation state or experience changes in their identity hash codes after garbage collection (JDK-8210708, 2019). This led to subtle, hard-to-reproduce application-level errors, such as synchronisation failures and inconsistencies in hash-based data structures.

Go's garbage collector examines the stacks of goroutines to find references to live objects. To do this correctly, it must reliably distinguish actual pointers from ordinary data values. However, in some situations, particularly when compiler optimisations or changes in stack layout are involved, the GC failed to recognise real pointers or treated non-pointer values as if they were valid references (GOLANG-22350, 2017).

During the sweep phase, the Go GC reclaims memory from unused objects and returns it to allocation pools. A bug in this phase caused incorrect handling of certain memory spans under concurrent conditions. As a result, memory regions could be reused while still being accessed by the program, leading to corruption and unpredictable crashes. This issue emphasises the importance of maintaining strict invariants during concurrent

memory reuse, particularly in lock-free or low-lock designs (GOLANG-3636, 2019).

In OCaml 5.2 and 5.3, where parallel heap compaction was reintroduced, a serious issue was found in how the garbage collector calculated memory positions during compaction. This problem was more likely to occur on 32-bit systems or when handling very large heaps. During the stop-the-world phase, the GC computes the memory locations where live objects should be moved. An integer overflow in these calculations caused the objects to be placed in the wrong locations, leading to unstable behaviour and potential crashes (Barnes and OCaml Development Team, 2024).

In OCaml 4.01.0, a bug caused issues in how free memory blocks were linked together during the sweep phase. As a result, the free list could become corrupted, with some blocks being lost or reused incorrectly or even crashes (Doligez and OCaml Development Team, 2014). Such bugs are particularly hidden because they may only manifest under specific memory pressure conditions, making them difficult to reproduce and diagnose in production systems.

1.4 OCAML GC AS A VERIFICATION TARGET

Given the importance of GC correctness and the limits of testing, this thesis explores formal verification as a way to obtain strong, mathematical guarantees about GC behaviour. We focus on OCaml because it combines practical relevance with interesting verification challenges.

OCaml is widely used in industry and academia. Its application ranges from financial platforms like Jane Street to formal verification tools such as Coq and Why3. Systems like MirageOS (a unikernel-based OS) and large-scale developer tools such as Facebook's Flow and Hack are also built using OCaml. OCaml has a mature, sophisticated runtime that has been developed over decades, supports a rich object model, and provides tight integration with C. This makes it a realistic and meaningful target for verification, while

also demonstrating that formal methods can scale beyond simplified systems.

From a research perspective, OCaml's GC poses many challenges. It must correctly handle different kinds of heap objects, including closures and infix objects, each with distinct traversal behaviour. The generational GC design of OCaml introduces concepts such as write barriers and cross-heap references, while recent versions (OCaml 5.0) add concurrency, which requires reasoning about thread interleavings and memory consistency. These features make OCaml an ideal setting to conduct realistic GC verification. Its strong type system ensures well-structured heaps, predictable object layouts, and a GC with clear responsibilities and interfaces.

As a starting point for our larger verification goals, we start with OCaml 4.14.1 GC. One might wonder why we have not started with the more recent OCaml 5.0, which introduces multicore execution and concurrent garbage collection. The reason is quite straightforward. Our goal is to untangle the complex invariants that are interwoven in a real-world GC. For that, we need to start with a simpler, sequential setting, where we can reason about the fundamental invariants of memory management in a manageable way. And then add more features to extend the system until we can create a sophisticated, performant GC. The benefits of this experiment are manyfold. First, we can get an overall insight into the specifications needed to ensure the correct behaviour of the GC. Second, it can help us to develop a correct-by-construction GC. And as a byproduct, we can actually get a production-grade GC that is guaranteed to be correct.

Within this sequential setting, we further narrow our focus to the major heap of OCaml's garbage collector. The major collector is responsible for long-lived objects and is an important part of the OCaml runtime and a natural target for verification. OCaml 4.14.1 follows an incremental mark-and-sweep design, where marking and sweeping happen in small steps alongside program execution. While incremental collection improves responsiveness, it also makes the system harder to reason about, as the collector constantly

operates on intermediate states.

To simplify the reasoning and isolate the core invariants, we begin by abstracting away this incremental behaviour and consider a stop-the-world variant of the algorithm. The mark-and-sweep algorithm proceeds in two distinct phases: a marking phase that traverses all reachable objects from the program roots, followed by a sweeping phase that reclaims the remaining memory. In this setting, the entire GC finishes execution before the program resumes. One has to note that this is not a purely abstract or idealised version of mark-and-sweep. The collector operates on real OCaml objects and respects the runtime's actual object layout that include headers, tags, and field representations. As a result, the model remains faithful to the structure and semantics of OCaml's heap, allowing us to reason about correctness in a setting that closely reflects the behaviour of the real system. We have developed a verified garbage collector for OCaml 4.14.1, though it is important to note that this is an alternative collector rather than a direct verification of the existing OCaml GC. The original OCaml GC is a generational collector, and fully replicating its design in a verified setting would require several additional pieces of work: making the major collector incremental so that it can handle heap changes introduced by the mutator mid-cycle, adding a verified copying minor collector, and verifying the allocator. Support for concurrency, which was introduced in OCaml 5.0, is also beyond the scope of this work. These boundaries are a deliberate starting point rather than fundamental limitations, and potential extensions are discussed in Chapter 5 (Section 5.4).

1.5 THE ESSENTIAL GC CORRECTNESS PROPERTIES

Having identified the specific component of the OCaml runtime that we aim to verify, we now turn to a more fundamental question: what does it mean for a garbage collector to be correct? At a high-level, the GC operates on a heap of OCaml objects where each object has a well-defined layout that includes headers and fields. The fields may contain pointers to other objects. A set of roots, such as stack variables, global references, and

registers, acts as the entry point into this heap. An object is considered *reachable* if it can be accessed by following pointers starting from these roots. The most fundamental property, called the *safety* property or the *soundness*, says that no reachable object is ever reclaimed. In other words, every object that can be reached from the roots must be marked during the marking phase. if such an object is not marked during the mark phase, then during the sweep phase, it may be reclaimed, leading to invalid memory accesses when the program later attempts to use it. On the other hand, any unreachable object (garbage) can be safely reclaimed.

A complementary requirement is that unreachable objects should be reclaimed as early as possible. This property is called the *liveness* or *completeness* of the GC. While some delay in reclamation is acceptable in practice, persistent failure to collect unreachable objects leads to memory leaks. It is also equally important that the GC should preserve the structure of the objects (as well as the data values in its fields) and that it should not introduce any *dangling pointers*, if the GC starts on a heap that is free of any dangling pointers.

Now, even in a stop-the-world setting, where the program is paused during collection, subtle issues can arise. Things can go wrong in how the collector walks through the heap, reads object fields, or keeps track of marking. These are not problems with the algorithm in theory, but with how it is put into practice on a real heap. While the correctness properties are conceptually clear, ensuring that they hold in all cases is far from straightforward. The number of possible heap configurations and object interactions is too large to explore through testing alone. This is why we turn to a more rigorous approach with the help of formal verification to reason about GC correctness.

This thesis is guided by three concrete research questions:

1. Can we formally verify a complete, end-to-end mark-and-sweep GC that extracts to portable C and integrates with a real language runtime?

2. What is the verification cost in terms of proof effort, and what specification strategies make this tractable?
3. Does the verified collector perform competitively with the existing OCaml garbage collector on real programs?

The remainder of this thesis addresses each of these questions in turn, beginning with the choice of verification tooling and the design of our layered specification framework.

1.6 FORMAL VERIFICATION

Formal verification establishes program correctness by constructing mathematical proofs that a system satisfies its specification. Formal verification aims to reason about *all possible executions* of a program, in contrast to testing.

More intuitively, formal verification treats a program not just as code to be executed, but as a mathematical object whose behaviour can be precisely described and analysed. There are three key elements in formal verification: (1) a precise specification of the desired behaviour, (2) a formal model of the program behaviour, and (3) a proof that the model satisfies the specification. In proof-oriented development, as practised in F* and Low*, the implementation itself serves as the formal model: specifications are attached directly to the code as type annotations and pre- and post-conditions, and the proof checker verifies that the annotated code satisfies them. For imperative Low* code, a further proof establishes behavioural equivalence with a pure functional counterpart, bridging the specification and execution levels.

In the context of garbage collection, formal verification helps us establish properties such as memory safety, correctness in identifying reachable objects, and the safe reclamation of garbage objects. Instead of checking a few heap configurations through testing, we can show that these properties hold for all valid heaps and in every possible execution. Another important advantage of formal verification is that it forces us to make assumptions

and invariants explicit. Things that are often taken for granted, like how objects are traversed, how memory is organised, or how invariants are maintained, must be clearly stated and carefully preserved. This not only improves confidence in correctness but also leads to a deeper understanding of the system being verified.

Formal verification has been successfully used in real, high-assurance systems. A well-known example is the seL4 microkernel, which has been fully verified for functional correctness, providing strong guarantees about the system's behaviour (Klein *et al.*, 2009). Projects like EverCrypt (from Project Everest) use F* to build high-performance cryptographic libraries that are both verified and used in practice (Protzenko *et al.*, 2020). Similarly, the CompCert C compiler demonstrates that it is possible to build a compiler formally proven to generate correct machine code, reducing the risk of subtle bugs (Leroy, 2009). Together, these examples show that formal verification can go well beyond small or simplified models. It can be applied to practical, performance-critical systems, making it a promising approach for verifying components such as garbage collectors.

While powerful, formal verification is not a one-size-fits-all solution. Different tools offer different trade-offs in terms of expressiveness, automation, scalability, and suitability for low-level systems. Some frameworks are very good at high-level reasoning, but they are not always a natural fit for low-level systems code. For example, proof assistants like Rocq (formerly Coq) (Bertot and Castéran, 2013) and Isabelle/HOL (Nipkow *et al.*, 2002) offer strong support for expressive specifications and machine-checked proofs, but working with pointer-heavy, C-like memory models takes considerable extra effort. On the other hand, tools like VCC (Cohen *et al.*, 2009*a*) and Frama-C (Cuoq *et al.*, 2012) are designed for verifying C programs and provide fine-grained control over memory. As deductive verification tools, they typically offer greater automation and require less manual proof effort, though this often comes at the cost of less expressive specifications. In particular, they do not readily support inductively defined reachability predicates of the kind central to GC correctness: expressing that an object is reachable requires an

inductive predicate that quantifies over all paths through the heap graph, and reasoning about it requires structural induction over those paths, neither of which fits naturally into the first-order annotation languages that deductive tools are built around. Nor do these tools natively bridge high-level functional specifications with low-level C extraction in a single framework. These differences highlight that no single tool is ideal for all settings. Choosing the right framework is therefore essential to make verification both practical and meaningful.

1.7 OUR TOOL OF CHOICE: F* AND LOW*

With these considerations in mind, we turn to F* (Swamy *et al.*, 2016; Martínez *et al.*, 2019) and its low-level subset Low* (Protzenko *et al.*, 2017). F* has been used to verify low-level systems code at scale, including cryptographic libraries such as EverCrypt (Protzenko *et al.*, 2020), parsers, and firmware, demonstrating that high assurance and practical deployability can go together. It provides SMT-based automation via Z3, the full power of dependent types for both specifications and proofs, higher-order expressive logics, and the Low* DSL for C-compatible memory models with support for machine integers and heap and stack allocated arrays. Crucially, F* compiles verified code to readable, idiomatic C via the KaRaMeL compiler (Protzenko *et al.*, 2017), closing the gap between abstract correctness properties and concrete memory manipulations. This combination makes F* and Low* uniquely well-suited for a system like a garbage collector, which must be both formally correct and practically deployable in a production runtime.

Our verification target is a mark-and-sweep GC for OCaml, implemented in C and closely tied to the runtime. At a higher level, we can think of the heap as a graph of OCaml objects, where pointers between fields form the edges. From this point of view, the marking phase is simply a depth-first traversal of this graph, starting from the program roots. This interpretation helps bridge the gap between abstract correctness (in terms of

reachability) and concrete implementation (in terms of pointer manipulation), which is exactly the kind of reasoning supported by F* and Low*.

In light of these observations, our GC correctness specifications are founded on abstract graph reachability, enabling us to specify the GC correctness without including the specifics of the GC implementation. This ensures that the GC can evolve to provide additional optimisations and incorporate more features without necessitating alterations to the core correctness specifications. There is a clear distinction between abstract GC correctness and OCaml-specific GC correctness, where the requirements can be managed in separate layers. Setting aside the functional aspects of the GC, it is crucial to ensure that the C implementation of the GC itself does not introduce any memory safety bugs. This mandates a third layer of separation focusing exclusively on the memory safety of the GC implementation, all the while maintaining the functional properties of the GC.

In summary, we have three distinct layers, each addressing a specific aspect essential for ensuring the overall correctness of the GC implementation as follows:

1. An abstract graph interface and a formally verified depth-first search layer (DFS) in F*, wherein the correctness of DFS is specified through inductively defined graph reachability.
2. A system-specific layer in F* that addresses the intricacies of the OCaml GC algorithm, such as the tricolor invariant (Jones *et al.*, 2016), utilized for reasoning about the correctness of mark-and-sweep GCs. This *functional GC layer* serves to bridge the gap between the abstract graph-based specification and its practical implementation in C. In this layer, the GC is implemented to operate on OCaml-style object layout, which is crucial to integrate the GC with the rest of the OCaml runtime. Within this layer, we have illustrated the progression of a practical GC by commencing with a basic GC implementation and systematically integrating diverse memory layouts supporting different OCaml features.
3. A low-level layer in Low* responsible for verifying memory safety of the GC

implementation. The GC code within this layer is extracted to C using a compiler known as KaRaMeL (Protzenko *et al.*, 2017).

At the time of this work, F* and Low* were the main tools available for low-level verification within the F* ecosystem. While Low* is well-suited for reasoning about memory safety in sequential programs, it provides no support for reasoning about concurrency. More recently, the F* ecosystem has introduced Pulse (Ebner *et al.*, 2025), a domain-specific language designed specifically for reasoning about concurrent programs using separation logic. A recent work by the F* team has shown that our verified garbage collector can be translated into Pulse (Ioannidis and Swamy, 2026) with the help of large language models (LLMs). This opens up a promising direction for future work through *agentic proof-oriented programming*. Since our broader goal is to extend this work to concurrent garbage collectors, Pulse provides a promising pathway to achieve this by enabling more precise reasoning about concurrency. In this context, the use of agents further supports this transition by helping bridge the gap between the existing Low* verification and its concurrent counterpart in Pulse.

It is worth noting that our verification relies on a trusted computing base (TCB) comprising F*, Low*, and the KaRaMeL compiler. The correctness of our proofs is contingent on the correctness of these tools. In particular, if KaRaMeL were to introduce errors during C extraction, the extracted code could deviate from the verified model. KaRaMeL's correctness for a substantial subset of Low* has been established through a pen-and-paper proof presented in (Protzenko *et al.*, 2017), rather than a machine-checked verification, which provides confidence, though not an absolute guarantee. We regard this as an acceptable and standard trade-off in verified systems development, consistent with projects such as CompCert and seL4.

1.8 THESIS CONTRIBUTIONS AND CHAPTER ORGANISATION

The full proof development, verified C code, and benchmark infrastructure are publicly available at the verified_ocaml_gc repository. The accompanying paper is described in Shamsu *et al.* (2025). This thesis makes the following contributions:

1. **First verified end-to-end mark-and-sweep GC in C.** To the best of our knowledge, this is the first work to formally verify a complete mark-and-sweep GC that extracts to C and integrates with an industrial-strength language runtime. We demonstrate this by integrating the verified collector with OCaml 4.14.1 and showing that it runs non-trivial OCaml programs competitively with the existing GC. The verification comprises around 39,000 lines of proof.
2. **Layered verification architecture grounded in graph reachability.** We adopt a three-layer approach descending from abstract graph reachability, through an OCaml-specific functional GC layer, down to memory-safe executable C via KaRaMeL extraction. While prior works also employ layered architectures (Myreen, 2010; Ericsson *et al.*, 2019; Hawblitzel and Petrank, 2009; McCreight *et al.*, 2007), none ground their top-level specification in an inductively defined graph reachability predicate and extract to portable C. To the best of our knowledge, ours is the first architecture to connect reachability-based GC correctness to a portable C implementation through an intermediate functional layer.
3. **Design-agnostic reachability specifications.** Our correctness specifications are primarily based on graph reachability, making them reusable across different GC and graph algorithms.
4. **Verification experience and dataset for proof engineering.** We document concrete challenges encountered in SMT-based verification of low-level systems code, including quantifier instantiation, trigger engineering, and proof decomposition strategies in F* and Low*. We also release a substantial dataset of verification conditions and proofs to support future work in machine-learning-assisted proof synthesis and proof repair.

The remainder of this thesis is organised as follows. Chapter 2 provides a discussion on various types of garbage collectors, followed by an overview of OCaml memory management in Section 2.3. Section 2.4 lays the foundation for F* and Low*. Chapter 3 examines related work and Chapter 4, which is the main contribution of this work, presents abstract correctness specifications for garbage collectors in Section 4.2 and describes the path toward a verified OCaml garbage collector in Section 4.3. Section 4.4 is dedicated to OCaml-specific garbage collector correctness specifications, while Section 4.5 elaborates on the layered design of our specification framework and the proof strategies employed at each layer. Section 4.7 discusses how the collector can be enriched to support more complex object layouts and types, while reusing the same high-level specifications established for a minimal collector. Section 4.8 presents benchmarks and experimental evaluations of the verified GC integrated with the OCaml 4.14.1 runtime. Finally, Chapter 5 summarizes the conclusions drawn from this research, presents abstract correctness specifications for incremental and copying collectors as a foundation for future verification efforts, and outlines directions for future work.

CHAPTER 2

BACKGROUND

Another example of a deep module is the garbage collector in a language such as Go or Java. This module has no interface at all; it works invisibly behind the scenes to reclaim unused memory. Adding garbage collection to a system actually shrinks its overall interface, since it eliminates the interface for freeing objects. The implementation of a garbage collector is quite complex, but that complexity is hidden from programmers using the language.

– John Ousterhout, *"A Philosophy of Software Design"*

2.1 INTRODUCTION

This chapter is partially adapted from the article entitled "A Mechanically Verified Garbage Collector for OCaml," published in the *Journal of Automated Reasoning* (Shamsu *et al.*, 2025).

This chapter provides the necessary background needed for the formal development of a verified GC for OCaml 4.14.1. We begin with an overview of garbage collection techniques and the design trade-offs that shape modern collectors (Jones *et al.*, 2011). We then describe the OCaml object representation and heap organisation, including the minor and major heaps and the language features that directly affect garbage collection behaviour (Madhavapeddy and Minsky, 2022*a*). Finally, we introduce the foundations of the tools of our choice, F* (Swamy *et al.*, 2016; Martínez *et al.*, 2019) and Low* (Protzenko *et al.*, 2017), and highlight how these tools implement the frameworks needed for reasoning about heap-manipulating programs, such as a GC. This foundational knowledge is necessary to understand the formal heap models, specifications, and invariants developed in later chapters, which form the core technical contributions of this thesis.

2.2 GARBAGE COLLECTION: A VAST DESIGN SPACE

Modern garbage collectors are drawn from a large design space of techniques for automatic heap management. At its core, a garbage collector does two things, (1) identify heap-allocated objects that are still in use by programs and (2) then reclaim those objects that are no longer used by any program. Conceptually, this is a simple task, but the way it is achieved across different language runtimes and system designs adds complexity. This is because the GC design has to deal with several competing system demands such as throughput (overall execution efficiency), latency and pause time (responsiveness), memory locality (cache performance), and, often, support for concurrency and parallelism (scalability on multi-core systems). We discuss these performance indicators in some detail in Section 4.8, where we evaluate the practical impact of our verified collector.

A GC must also respect language-specific constraints such as object representation, pointer discipline, and mutability. These constraints strongly influence how memory can be traversed and manipulated at runtime. From a verification standpoint, the design space is also influenced by the need to formally analyse reachability, heap structure, termination, and correctness.

Two fundamental correctness properties guide the design of any garbage collector, namely, *safety* and *liveness*. Safety requires that every object reachable from the program's roots before collection remain allocated after collection. In other words, the collector must never free a live (in use) object. Liveness, in contrast, requires that every object that is unreachable before collection is eventually reclaimed. We can model garbage collection over a heap as a graph traversal problem, where nodes are objects and edges represent interconnections via object field references. A detailed discussion about object representation in OCaml will be discussed in Section 2.3.1. While safety can be defined in terms of graph reachability, liveness is fundamentally harder to define. This is because liveness is a global property, whereas the decision to decide whether an object is live is made locally (Jones *et al.*, 2011). As a result, practical garbage collectors approximate liveness by computing reachability: any object not reachable from the roots via pointer traversal is considered garbage. This approximation is sound with respect to safety (all reachable objects are preserved) but only conservative with respect to liveness (some objects that will never be accessed again may still be retained if they remain reachable at the beginning of the garbage collection cycle).

Most modern garbage collectors belong to the family of *tracing collectors*. A tracing collector begins from a finite set of roots, such as stack variables, registers, and global variables and traverses the object graph through the pointer fields of objects. Any object that can be reached in this traversal is considered live. The collector's task is to ensure that this traversal is both complete and correct with respect to the language's object model. Two important classes of tracing collection are mark-and-sweep and copying collection.

In the remainder of this Section, we first examine the mark-and-sweep collector in detail, including the tri-color abstraction commonly used to reason about correctness. We then compare it with copying collectors, where objects are relocated during the process. We also discuss mark-compact and reference counting as two alternate collection approaches. Advanced collector variants, such as incremental, concurrent, and parallel collectors, are built on top of foundational techniques that will be discussed towards the end of the section.

2.2.1 Mark-and-Sweep Garbage Collectors

The mark-and-sweep GC runs in two phases: mark and sweep. The mark phase performs a depth-first traversal of the object graph reachable from the *root set* of pointers. In the mark phase, a mark stack (worklist) is used to store objects discovered during the traversal whose field pointers have not yet been scanned. After the mark stack is filled with the roots, the mark proceeds by popping an object from the mark stack. The fields of the popped object are then scanned to find any unvisited field pointers. All such unvisited field pointers are pushed onto the stack to continue the traversal. The mark terminates when the mark stack is empty, which means that all reachable objects have been examined. During the sweep phase, all unmarked objects are found by scanning the entire heap, and the unmarked memory is returned to the free list.

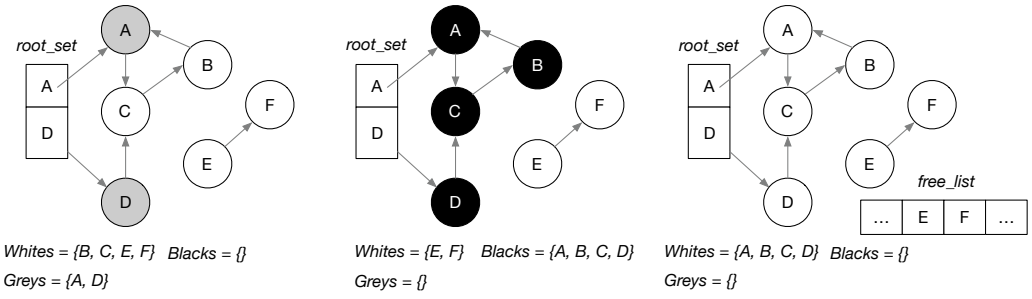
The correctness of mark-and-sweep depends crucially on the mark phase because it must mark exactly those objects that are reachable from the roots. This requires that the root set be precisely identified before collection begins, because any missing root could lead to the collection of live objects (unsafe). In contrast, any spurious root merely retains additional garbage (conservative but safe).

The main disadvantage of mark-and-sweep GC is heap fragmentation caused by object freeing. Copying collectors, another type of tracing collector, address this disadvantage, which we will discuss in the Section 2.2.2. The following Section focuses on *tri-color*

marking, which offers a formal framework for structuring and reasoning about the traversal of the object graph. This method is widely used in both incremental and concurrent mark-and-sweep collectors, which we will describe in Section 2.2.4

2.2.1.1 Tri-color Mark-and-Sweep

We now describe the details of a tricolor *stop-the-world* mark-and-sweep GC algorithm. As mentioned previously, our verified GC is based on this algorithm. Figure 2.1 shows a run of the mark-and-sweep GC. At the start of the mark phase, all objects directly pointed from the root set are colored grey and are added to the *mark stack* (Figure 2.1a).The mark phase uses a mark stack to store the objects that are discovered, but not yet fully explored. Objects which are free in the heap are colored blue and are maintained in a *free list*, a linked list of free objects. Every other object is white. We have the invariant that all objects in the mark stack are grey and every live object is reachable from a grey object transitively through a sequence of white objects.



(a) Start of mark (b) After mark (c) After sweep

Figure 2.1: Tricolor marking. Initially, the objects that are directly pointed to the root pointers are grey in the heap. The stack is populated with such objects first.

The mark phase proceeds by popping a grey object from the mark stack. All of the white successors of the popped object are marked grey and pushed onto the mark stack. Finally, the popped object is marked black, meaning that it has been fully processed and all of its successors have been examined. Thus, a black object never points to a white object. When the mark stack is empty, the mark phase ends (Figure 2.1b). We are guaranteed

that all the reachable objects are marked black, and all unreachable objects remain white, and there are no grey objects.

The sweep phase performs a linear traversal of the heap from the low address to the high address and examines each object. If the object is black, it is alive. Sweep changes its color to white. If the object is white, it is dead. Sweep changes its color to blue and adds it to the free list. During sweep, we have the invariant that an object whose address is less than the traversal pointer is either white (live) or blue (free) and is on the free list. After sweep (Figure 2.1c), we are guaranteed that all live objects are white and all unreachable objects are in the free list with color blue, due to this invariant.

2.2.2 Copying Garbage Collectors

Copying collectors form a major family of tracing garbage collectors in which live objects are relocated during collection. A copying collector identifies all objects reachable from the roots and moves them into a new region of memory. Any objects that are not copied are implicitly discarded. This relocation has the immediate benefit of producing a compact heap with no fragmentation, allowing allocation to proceed very efficiently.

The most well-known and widely used copying collectors are semispace collectors, which divide the heap into two equal regions. At any time, allocation occurs only in the active half. When that region fills, the collector copies all reachable objects into the other half and then swaps the roles of the two spaces.

A well-known semispace copying algorithm is Cheney's algorithm, in which the collector begins by copying objects reachable from the roots into the target space (often called ToSpace). Then the outgoing pointers of the copied objects are scanned in a breadth-first fashion so that any unmoved objects reachable through the field pointers can be copied to the ToSpace. Forwarding pointers ensure that each object is copied at most once and that all references are updated consistently. The result is a compact, non-fragmented, contiguous region containing exactly the live objects.

At the same time, copying collectors complicate other aspects of the system. Object relocation interacts poorly with conservative roots, foreign code and components that rely on stable object addresses. Maintaining correctness in the presence of pointer updates and write barriers, especially in incremental or concurrent settings, introduces its own challenges.

2.2.3 Alternative Collector Designs

Two important classes of alternative collector designs are reference counting and mark–compact collection, which we will discuss next.

In **reference counting**, each object is associated with a count indicating the number of references that are currently pointing to it. The count is updated when references are created and destroyed, and the object is reclaimed when its reference count drops to zero. This approach has an appealing property that it does not require global GC pauses. An immediate disadvantage of reference counting is that it cannot reclaim cyclic garbage, where unreachable objects taking part in cyclic reference cannot be reclaimed through reference counting alone.

Also, keeping track of counts adds extra work to every pointer assignment. It makes it harder to think about mutation, sharing, and aliasing, especially in a language like OCaml that has complex heap structures and higher-order values. These traits refer to a less-than-ideal foundation for a formally verified collector in a practical functional runtime.

Mark–compact collection is a refinement of mark–and–sweep. Like mark–and–sweep, it begins with a tracing phase that marks all objects reachable from the roots. After sweeping the unmarked objects, the collector compacts the heap by relocating all the surviving objects together into a contiguous region. Similar to a copying collection, compaction essentially eliminates fragmentation, while maintaining a single logical heap.

In terms of computational complexity, compared to mark-and-sweep GC, an additional phase is required to compute new addresses, relocate the objects and rewrite their pointers. Therefore, mark compact collection inherits the drawbacks of copying collection with respect to object relocation, along with an extra pass to compact the heap.

In contrast, mark–and–sweep occupies a useful middle ground. It is a full-trace collector capable of reclaiming cyclic structures while preserving object addresses. The algorithmic structure of mark-and-sweep also provides a clean semantic model based on reachability in a heap graph, which makes it particularly attractive to formal reasoning about correctness, invariants, and safety properties for a language such as OCaml that combines functional and mutable features.

2.2.4 Incremental and Concurrent Collectors

The basic mark–and–sweep algorithm works as a stop-the-world process, pausing the program entirely while garbage collection is performed. Such long pauses can be problematic, especially for interactive and real-time systems. To reduce these pauses, modern collectors use **incremental** and **concurrent** techniques, which allow parts of the collection to run alongside program execution.

Both incremental and concurrent collectors refine the marking phase of mark–and–sweep. Instead of performing marking in a single monolithic pass, they break it into smaller steps interleaved with program execution (incremental collection) or run in parallel with the mutator on separate threads or cores (concurrent collection). The sweep phase may likewise be performed. However, these techniques require write barriers and synchronisation protocols to prevent the mutator from violating the assumptions on mark, which could affect reachability. A write barrier is a small piece of code executed on pointer updates that records or constrains mutations to the heap. A detailed discussion about the need for write barriers to preserve reachability is discussed in the Chapter 5, Section 5.4 .

2.2.5 Generational Collectors

In a different direction, **generational** garbage collection focuses on improving efficiency by taking advantage of the observation that most objects die young (generational hypothesis). Such collectors group objects into different generations that are collected at different rates. The heap is divided into generations, with new objects allocated in a young region, usually called the minor heap, which is collected frequently. The objects that survive during a minor collection are promoted to the old region, which is often called as the major heap. Typically, the young generation is managed by a copying collector, and the old generation by mark–and–sweep or mark–compact. Write barriers are again essential here as they record references from older to younger objects so that minor collections can be performed without tracing the entire heap.

Each of these advanced collectors introduces additional invariants and runtime mechanisms that significantly increase the complexity of specification, reasoning, and implementation. Copying collectors require relocation proofs: one must show that all pointer fields are updated consistently after objects move, and that forwarding pointers are handled correctly. Incremental and concurrent collectors introduce write barrier invariants: the proof must account for mutator interference during collection, requiring rely-guarantee or separation logic reasoning about concurrent heap access. Generational collectors add cross-generation reference proofs: the remembered set must be shown to be complete, and the interaction between minor and major collection must be verified jointly. Mark-and-sweep avoids all of these: objects are never moved, no write barrier is needed in the stop-the-world setting, and the correctness argument reduces cleanly to reachability in a static heap graph. By first establishing a verified mark-and-sweep collector, we obtain a strong foundation on which more advanced, low-pause and high-throughput designs can later be understood and extended.

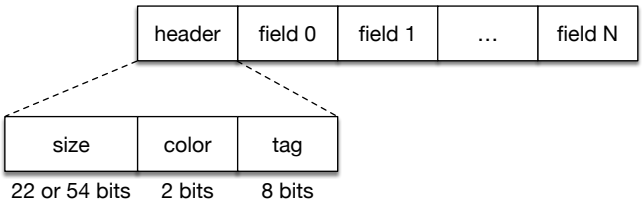


Figure 2.2: Layout of an OCaml object. The header and each field of an OCaml object occupy one word.

2.3 GARBAGE COLLECTION IN OCAML

OCaml manages memory automatically through a garbage collector that is deeply integrated into the runtime system. The design of the OCaml GC has been carefully tuned over decades to deliver low-latency, predictable performance for the kinds of allocation-heavy functional programs for which OCaml is typically used. This section describes the GC as it exists in two versions of OCaml that are relevant to this thesis. We implemented a verified major collector for OCaml 4.14.1 runtime, which is the central part of this thesis. Whereas OCaml 5.0 is the most recent OCaml version, which introduced native shared-memory parallelism through a fundamental redesign of the runtime. Understanding its GC in relation to OCaml 4.14.1 situates the contribution of this thesis within the broader trajectory of OCaml's development. We begin with the object representation, which serves as the common foundation for both versions, before describing the GC algorithms for each.

2.3.1 Object Representation

Every object in OCaml has a word-sized header in which meta-data about the object is stored (Madhavapeddy and Minsky, 2022*b*). A typical OCaml object is represented as a *block* in the OCaml heap, which has a header followed by variable number of fields. Figure 2.2 shows the layout of an OCaml object. The header is a single word (64 bits on a 64-bit platform). It consists of an 8-bit tag, 2 bits for the object color (encoding the four colors blue, white, grey, and black), and 54 bits encoding the object size in words (called *wosize*, short for word size). On a 32-bit platform the layout is analogous with 22 bits for wosize. Every field of the object is also word-sized, which ensures that

the pointers to objects are always word-aligned. Immediate values such as integers and booleans are also word-sized. Immediate values are encoded with their least significant bit (LSB) to be 1, with the rest of the bits encoding the value of the data type. Thus, OCaml integers are 31-bits and 63-bits long on 32-bit and 64-bit platforms. Pointers are always guaranteed to be word-aligned and have 0 as their LSB. While the representation is not compact, it simplifies the GC; by examining the LSB, the GC can decide whether the value is a pointer or an immediate.

Many OCaml language constructs are represented as objects in the heap. For example, variants with parameters, records, arrays, polymorphic variants, closures, floating-point numbers, etc. are all represented as objects in the heap. The tag bits in the header of an OCaml object is used, among other things, to determine whether the fields of the objects may contain pointers. In particular, for objects with tag greater or equal to `No_scan_tag` (251), the fields are all opaque bytes, and are not scanned by the GC. For example, OCaml strings have a tag of `String_tag` (252) and contain opaque bytes and never contain pointers.

If an object's tag is less than `No_scan_tag` (251), then the fields of the objects may be pointers. Among these, apart from `Closure_tag` (247) and `Infix_tag` (249) objects, the GC scans each field of the object to determine if it is a pointer or an immediate value and takes appropriate action.

Well-formed heaps. A heap is said to be *well-formed* if (1) every pointer field of an allocated object points to another allocated object, (2) every object's color is one of the four valid colors, and (3) no allocated object overlaps with a allocated or free block. This definition is central to the formal specifications in Chapter 4, where it appears as a precondition and postcondition of every GC operation.

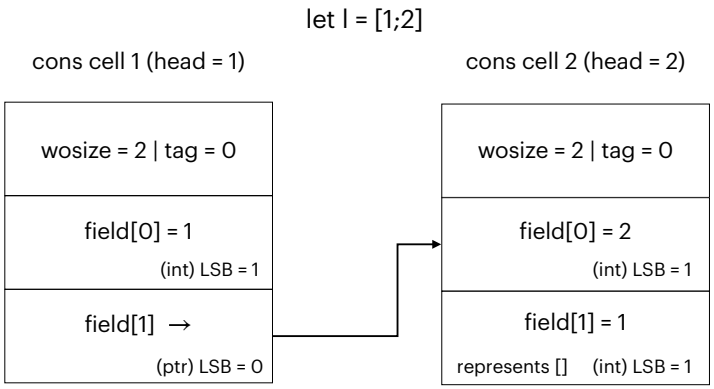


Figure 2.3: Memory layout of the `list [1; 2]` in the OCaml major heap. Each cons cell is a heap block with a header word followed by two fields. The header encodes wosize = 2 and tag = 0 (below `No_scan_tag`, so both fields are scanned by the GC). Field 0 holds the head value as a tagged immediate integer. Its LSB is 1, indicating to the GC that it is not a pointer. Value of field 1 of cons cell 1 is a word-aligned pointer to cons cell 2 (its LSB is 0). Field 1 of cons cell 2 contains the empty list `[]`, which is represented as the immediate integer 0 (LSB = 1) and it indicates the end of the list.

2.3.1.1 A concrete block example

Let us take a non-empty list in OCaml to illustrate these rules. In OCaml, the type `` `a list `` represents a polymorphic list of type `` `a `` which is either `[]` (the empty list, represented by the integer `0`) or `h :: t` (a cons cell represented as a heap block). A polymorphic list is a standard built-in data structure that can hold elements of any single type. The type is denoted by the type variable `` `a `` (pronounced "alpha"). A cons cell `h :: t` is allocated as a block with the tag `0`, and it contains two fields. The first field contains the head value `h` and the second holds the tail pointer `t`.

The GC scans both fields since the tag is less than `No_scan_tag`. If `h` is an integer, then its LSB is 1, and the GC treats it as an immediate. If `h` is again a heap-allocated value, its LSB is `0` and the GC follows the pointer. The tail `t` represents either an empty list or a pointer to the next cons cell. Figure 2.3 shows the memory layout of the list `[1; 2]`, which consists of two cons cells in the major heap. This example illustrates the general pattern for all ordinary OCaml blocks: a header word followed by `wosize` fields, each of which is either a tagged immediate or a word-aligned pointer distinguished by its LSB.

2.3.1.2 Closure objects

In OCaml, functions are first-class values that may reference variables from their enclosing scope. OCaml represents such functions as *closure objects* on the heap, bundling together the function's code pointer and the values of its free variables, called the *environment*. Since closures are heap-allocated, the GC must trace them. However, unlike ordinary OCaml blocks, the environment fields do not necessarily start at the first field; the GC must use the start-of-environment offset encoded in the closure info word to locate them. Mutually recursive functions add a further complication by embedding infix objects within the closure block. A closure for a function or a set of mutually-recursive functions is represented as a heap block with the following structure:

$$
\begin{array}{rcll}
\textit{closure} & ::= & \textit{entrypoint}\ \ (\textit{infix-header}\ \ \textit{entrypoint})^*\ \ \textit{value}^* & \\
\textit{entrypoint} & ::= & \textit{code-pointer}\ \ \textit{closure-info} & \textit{// arity} = 1 \\
\mid & ::= & \textit{code-pointer}\ \ \textit{closure-info}\ \ \textit{code-pointer} & \textit{// arity} > 1 \\
\textit{closure-info} & ::= & \textit{arity}\ [8\text{ bits}] \cdot \textit{start-of-environment}\ [\text{wordsize} - 9\text{ bits}] \cdot 1 &
\end{array}
$$

The BNF grammar above describes the layout of a closure block in memory from top to bottom. A closure block consists of one or more entrypoints followed by the environment, which holds the values of the free variables shared by all functions in the block.

Each entrypoint takes one of two forms depending on the arity of the function it represents. For a function with arity = 1, the currified and total function pointers are identical, so the entrypoint is compactly represented as two words: a single code pointer followed by a closure info word. For a function with arity > 1, the entrypoint requires three words: a *currified pointer* used when the function is partially applied with fewer arguments than its arity, a closure info word, and a *total pointer* used when all arguments are supplied at once.

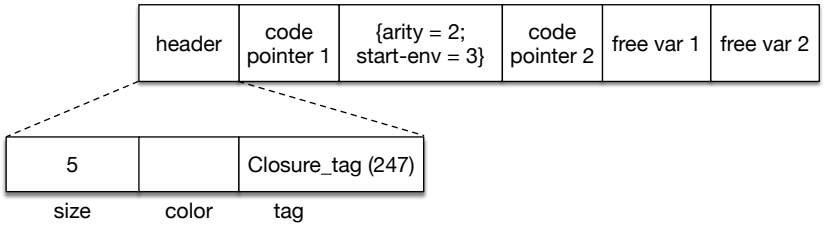


Figure 2.4: The layout of a closure object with arity 2 and environment size 2.

The closure info word encodes two pieces of information: the arity of the function and the start-of-environment offset, which gives the position of the environment relative to the start of that entrypoint. Since the closure info word has its LSB set to 1, the GC treats it as an OCaml immediate value and does not follow it as a pointer. The GC uses the start-of-environment offset to skip the entrypoint words and scan only the environment fields, which are the only fields that may contain heap pointers. Figure 2.4 shows a concrete example of a closure with arity 2 and an environment of size 2, where the environment begins at field index 3.

Mutually recursive functions are represented as a single closure block containing multiple entrypoints, one per function, each preceded by an infix header except the first. All functions share the same environment, which appears once at the end of the block. The infix header stores the byte offset back to the start of the parent closure block, which the GC uses to locate the parent when it encounters a pointer to an inner entrypoint. Importantly, the GC never marks an infix object directly; instead, it uses the offset in the infix header to find and mark the parent closure. Figure 2.5 shows a closure block with two mutually recursive functions of arities 2 and 1 and an environment of size 2. The total wosize of the block is 8, which includes the infix header, and the infix header stores an offset of 4 words back to the start of the parent closure.

Example 1: A simple non-recursive closure (arity = 1). The below function `make_adder` takes an integer `n` and returns a new function that adds `n` to any integer `x` passed to it. For instance, `make_adder 5` returns a function that adds `5` to its argument, so `make_adder 5 3`

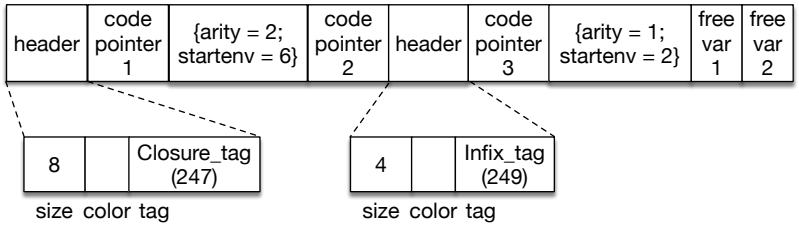


Figure 2.5: The layout of a mutually-recursive closure object layout with arities 2 and 1 and environment size 2.

evaluates to `8`.

```
let make_adder n =
  fun x -> n + x
```

A call such as `make_adder 5` allocates a closure block on the heap for the inner function (arity 1) `fun x -> n + x`, with `n` bound to `5` as its environment variable. On a 64-bit host, the closure block has the following layout:

Word	Content	Notes
—	`header` (closure_tag, size = 3)	Block header
0	code pointer	OCaml value for the closure
1	closure-info {arity=1, startenv=2}	
2	value of `n`	Environment

The block contains a single arity-1 entrypoint (words 0–1) followed by the environment (word 2). The `startenv` field records that the environment begins 2 words ahead of word 0. The block header records `size = 3`, the total number of words after the header.

Example 2: A recursive closure (arity > 1). The function `make_multiplier factor` returns a recursive function `repeat x n` that computes `factor * x` added to itself `n` times, effectively computing `factor * x * n` through repeated addition, capturing `factor` from

the enclosing scope. A call such as `make_multiplier 3` allocates a closure block on the heap for `repeat`, with `factor` bound to `3` as its environment variable, so `repeat 2 4` evaluates to `24`.

```
let make_multiplier factor =
  let rec repeat x n =
    if n = 0 then 0 else factor * x + repeat x (n - 1)
  in
  repeat
```

The function `repeat` has arity 2 (arguments `x` and `n`) and captures one environment variable `factor`. Since arity > 1, the entrypoint requires three words: a currified pointer, a closure-info word, and a total pointer. The closure has the following layout on a 64-bit host:

Word	Content	Notes
—	header (`closure_tag`, size = 4)	Block header
0	currified code pointer	For partial application of `repeat`
1	closure-info {arity=2, startenv=3}	arity and environment offset
2	total code pointer	For full application of `repeat`
3	value of `factor`	Environment

The currified pointer (word 0) is used when `repeat` is partially applied with one argument, the closure-info word (word 1) encodes the arity and the start-of-environment offset, and the total pointer (word 2) is used when both arguments `x` and `n` are supplied at once. The `startenv` field records that the environment begins 3 words ahead of word 0, landing at word 3, where the captured value of `factor` is stored.

Example 3: Mutually recursive closures. The function `make_checker` generates a

random integer `limit` and returns the function `even`, which, together with the mutually recursive function `odd`, checks whether a non-negative integer `n` is even while staying within `limit`. Both `even` and `odd` capture the same `limit` from the enclosing scope. For instance, if `limit` is `10`, then `even 4` returns `true` and `even 11` returns `false` because `11` exceeds `limit`. Calling `make_checker ()` allocates a closure block on the heap containing two mutually recursive functions, `even` and `odd`. Both functions share the same environment variable `limit`, which is captured once in the closure block and accessed by both entrypoints via the shared environment.

```
let make_checker () =
  let limit = Random.int 100 in
  let rec even n = n = 0 || (n > 0 && n <= limit && odd (n - 1))
  and     odd  n = n > 0 && n <= limit && even (n - 1)
  in
  even
```

Both `even` and `odd` have arity 1 and share one environment variable `limit`.

Word	Content	Notes
—	`header` (closure_tag, size = 6)	Block header
0	code pointer of `even`	OCaml value for `even`
1	closure-info {arity=1, startenv=5}	
2	`infix header` (infix_tag, size = 24)	3×8 bytes to word 0
3	code pointer of `odd`	OCaml value for `odd`
4	closure-info {arity=1, startenv=2}	
5	value of `limit`	Shared environment

The header records `size = 6` since words 0–5, including the infix header at word 2, all belong to the same block. The infix header at word 2 stores a byte offset of $3 \times 8 = 24$, the

distance from word 0 to word 3, which the GC subtracts from any pointer to `odd` to recover the parent block. For `even`, `startenv = 5 - 0 = 5` and for `odd`, `startenv = 5 - 3 = 2`. Both closures locate the single copy of `limit` at word 5 via their respective offsets.

2.3.2 OCaml 4.14.1 Runtime System

OCaml 4.14.1 uses a generational garbage collector with two generations. The young generation is managed by a minor heap collected by a stop-the-world copying collector, and the old generation is managed by a major heap collected by an incremental non-moving mark-and-sweep collector. There are pointers from the major heap to the minor heap, which are recorded in a *remembered* set. Writes to an object are managed by a write barrier, which also helps ensure the snapshot-at-the-beginning invariant for the major collector across mutator activity. We will defer a detailed discussion of the necessity of preserving this property until Chapter 5, Section 5.4.1, where we discuss the possibility of reusing our verified GC specifications to extend it to an incremental mark-and-sweep collector.

2.3.2.1 The Major Collector

The major collector is an incremental, non-moving, mark-and-sweep collector that uses the tricolor abstraction described in Section 2.2.1.1. There are two phases in the GC: the mark phase, followed by the sweep phase. The mark phase uses an explicit mark stack to perform marking, with the entire process reduced to a depth-first traversal of the object graph. When the marking phase encounters a white object, it is changed to grey and pushed onto the stack. Marking begins by marking the roots (register, stack, or global variables), which are split into two parts. The register and stack references are marked and pushed onto the mark stack in a stop-the-world fashion. At the same time, the global roots (more numbers in real-world programs) are marked and pushed incrementally to avoid pauses.

After the roots are marked, the mark proceeds in slices (incremental marking) by popping

an object from the stack, changing its color to black, darkening the white successors of that object, and pushing them onto the stack. When the mark stack is empty, sweeping traverses the heap incrementally, reclaiming white objects and resetting black objects to white for the next cycle. Note that new allocations during marking are colored black to prevent premature collection.

Our verified GC implements the major collection algorithm of OCaml 4.14.1 by following the same tricolor scheme, the same mark stack discipline, the same phase structure of marking roots, draining the mark stack, sweeping and operating on OCaml-style heap objects with the same header layout and color encoding, but in a stop-the-world fashion. The correctness properties we establish, such as the set of allocated objects after collection is exactly the reachable subgraph, that no field value of any live object is affected by the GC, and that the heap remains well-formed in the sense that no allocated object points to an unallocated location, therefore apply directly to the OCaml 4.14.1 major collector. The details of these properties and their formal proofs are the subject of the next chapter.

Extending the verified major GC to the full OCaml 4.14.1 runtime would require additionally verifying the incremental scheduling mechanisms, such as the slice budget computation that determines how much GC work is performed per allocation, and the interaction between the major collector and the allocator during the sweep phase. But these are engineering refinements to the core algorithm rather than fundamental changes to the correctness argument, and the proof infrastructure developed in this thesis provides the natural starting point for that extension.

2.3.2.2 The Minor Collector

The minor heap is a small memory area where allocations are performed using a bump-pointer allocator. Most allocations in OCaml initially go into the minor heap. The minor collector, a stop-the-world copying collector, performs garbage collection in the minor

heap. The stop-the-world choice makes sense as the size of the heap is small, and hence the pause time is less. The collector scans the roots, such as the stack, registers, global variables, and the remembered set for pointers into the minor heap. Any minor heap object reachable from the roots is a survivor and is copied to the major heap. When an object is copied, a forwarding pointer is added at its old minor heap location. Any subsequent pointer to the old location is updated to point to the new primary heap address via the forwarding pointer. After all survivors have been promoted, the entire minor heap is reclaimed at once by resetting the bump pointer; that is, no per-object free operation is needed because the heap is collected in its entirety.

The remembered set is a critical component of the minor collector. It records all pointers from the major heap into the minor heap that were created by the mutator via the write barrier. These pointers represent additional roots for the minor collection because otherwise an object in the minor heap pointed to only by a major heap object would be missed if only the stack and registers were scanned. After a minor collection, the remembered set is cleared and rebuilt from scratch in the next mutator activity.

The collection time is proportional to the number of live objects rather than the heap size, as most objects in the minor heap will be dead when collection is triggered (generational hypothesis), and the collector has to deal with only the live objects.

2.3.3 OCaml 5.0 Runtime System

Although this thesis verifies the OCaml 4.14.1 GC, understanding OCaml 5.0 is important for two reasons: it shows where the sequential correctness properties we establish remain valid, and it identifies precisely what a future concurrent verification would additionally require. OCaml 5.0, released in 2022, introduced native support for shared-memory parallelism (Sivaramakrishnan *et al.*, 2020). This is achieved through a fundamental redesign of both the runtime system and the GC.

2.3.3.1 Domains: The Source of Parallelism

The feature that gives OCaml 5.0 its shared-memory parallelism is the domain. Domains are units of parallelism. Each domain is heavyweight and maps onto an OS thread, with the recommendation of one domain per core. They support domain-local storage and atomic memory operations, and there are no restrictions on sharing objects between domains. Each domain can execute OCaml code and allocate memory as needed while running simultaneously in the same address space. This is basically different from OCaml 4.x, where a global interpreter lock prevents true parallelism. There is no global runtime lock in OCaml 5.0, which allows domains to run in parallel, necessitating a ground-up redesign of the runtime and GC to handle concurrent access to shared state.

2.3.3.2 New requirements for the GC design

The GC correctness properties in OCaml 4.14.1, including color invariants, root completeness, sweep soundness, etc., are established entirely within a sequential execution model where GC state transitions are atomic with respect to the mutator. Whereas OCaml 5.0 introduces a fundamentally new layer of complexity due to shared memory access across domains that execute in parallel. Multiple domains simultaneously allocate, mutate heap fields, and perform GC work, meaning that every piece of shared GC state, such as object colors, the mark stack, the free list, or the remembered set, is now subject to concurrent reads and writes. This is not merely a matter of thread safety in the engineering sense. Rather, it changes the correctness argument at its foundation.

For example, in OCaml 4.14.1, a color transition is an atomic event observed by exactly one thread, and the invariant that no marked object points to an unmarked object can be stated and proved as a sequential property. In OCaml 5.0, the same color transition may be observed simultaneously by multiple domains, and the same invariant must now be established in the presence of races, reorderings, and interleaved GC work across domains. The correctness proof must therefore account for this additional layer of shared-memory

concurrency, and it is not sufficient to verify that the algorithm is correct sequentially and assume that concurrency follows. This is precisely the gap that separates our verified OCaml 4.14.1 GC from a complete verification of OCaml 5.0, and the authors think identifying this gap is itself a contribution of the sequential verification presented in this thesis.

2.3.3.3 The Major Collector

In OCaml 4.14.1, the colors black and white change meaning between the mark-and-sweep phases, as we already discussed. A black object indicates a live object discovered during the mark phase that has to be reset to white during sweeping. And a white object means an object that is unreached during marking and therefore should be freed by the sweeping phase. In a sequential GC, this is harmless because the phase is a single global variable visible to the GC thread. In a concurrent GC with multiple domains, however, a domain may read an object's color bits precisely as another domain triggers a phase transition, causing it to misinterpret the color under the wrong phase semantics. OCaml 5.0 eliminates this race by adopting the VCGC (Huelsbergen and Winterbottom, 1998) color scheme in which colors carry phase-independent meanings (Sivaramakrishnan *et al.*, 2020). There are three colors in this setting: *Marked*, *Unmarked* and *Garbage*. A fourth color *Free* is used to indicate free objects. Marking proceeds by changing Unmarked objects into Marked ones, while sweeping proceeds by collecting Garbage objects into Free blocks. Note that in this scheme, the sets of objects affected by marking and sweeping are disjoint, so no synchronisation between them is necessary. And hence no domain ever needs to consult the current phase to correctly interpret a color, removing the entire class of phase-transition bugs.

Furthermore, each domain maintains its own local mark stack. When the write barrier needs to grey an object, it marks it and adds it to the local domain's mark stack. Multiple domains may attempt to mark the same object at the same time, and this is resolved by making marking idempotent: marking an already-marked object is a no-op that produces

the same result as a single marking. Allowing for this possibility is much cheaper than synchronising each access to an object by the collector. On the other hand, sweeping is not idempotent. This is resolved by making each domain sweep only the memory allocated to it, so that the interaction between sweeping and allocation is entirely local to each domain. However, there is one synchronisation point per major collection cycle, at the end, when all domains stop simultaneously to relabel the GC color bit-patterns to begin the next cycle.

2.3.3.4 The Minor Collector

The minor heap is much smaller than the major heap but must sustain a much higher rate of allocation and collection. And most importantly, the minor collector is a non-incremental (stop-the-world) copying collector, which **moves** objects from minor heap to major heap. In OCaml 4.14.1, there is a single minor heap shared by the one mutator thread. When the minor heap fills, the mutator stops, all live objects are copied to the major heap, and the minor heap is reused. In OCaml 5.0, each domain has its own private minor heap arena and allocates independently without synchronisation. When a domain exhausts its minor heap, it signals all other domains to stop simultaneously, and all domains perform minor collection in parallel, each promoting their own live objects to the shared major heap. Objects that two domains attempt to promote simultaneously are serialised using compare-and-swap operations on the object header to ensure each object is promoted exactly once.

2.3.3.5 Relation to This Thesis

To verify a garbage collector, one must establish precise correctness properties about the heap that the GC guarantees to programs running on it, which should extend beyond internal GC mechanisms such as color invariants. Our work establishes these properties for the OCaml 4.14.1 GC, and the details of what they are, how they are formally stated in F*/Low*, and how they are proved are the subject of the next chapter. What the discussion in this section makes clear is that OCaml 5.0 introduces a fundamentally new

layer of complexity due to concurrent shared-memory access across domains, which any future verification effort would have to account for. The sequential correctness properties we establish for OCaml 4.14.1 remain meaningful in the OCaml 5.0 setting as well. If there is only one domain, the two GCs behave identically, and the invariants we prove carry over directly. Whereas in the multi-domain case, the main correctness properties should be re-examined under concurrency as new proof obligations arise from idempotent concurrent marking, per-domain mark stacks, and the disjointness of marking and sweeping across domains. Understanding the OCaml 4.14.1 GC and the changes introduced in OCaml 5.0 is therefore not incidental background; rather, it is the context within which the contribution of this thesis is situated and from which its future extensions naturally follow. This section highlights only the key design decisions of OCaml 5.0 most relevant to GC verification and does not cover all techniques in detail. A comprehensive treatment of OCaml 5.0's GC is given by Sivaramakrishnan *et al.* (2020).

2.4 FOUNDATIONS OF PROGRAMMING AND VERIFICATION IN F* AND LOW*

Our verified GC is developed in F* (Swamy *et al.*, 2016; Martínez *et al.*, 2019) and its low-level subset Low* (Protzenko *et al.*, 2017). F* supports SMT-assisted verification via Z3, a C-compatible memory model with low-level reasoning, and extraction to C code via the KaRaMeL compiler. The TCB of our verification comprises F*'s type checker, the Z3 SMT solver, and KaRaMeL. F*'s core metatheory has been formally studied (Swamy *et al.*, 2016). KaRaMeL's correctness for a substantial subset of Low* has been established through a pen-and-paper proof presented in (Protzenko *et al.*, 2017), rather than a machine-checked verification. This is consistent with the TCBs of projects such as CompCert (Leroy, 2009) and seL4 (Klein *et al.*, 2009).

This section introduces the F*/Low* features needed to understand the proofs in Chapter 4. We begin with refinement types and key language constructs (Section 2.4.1), then cover

how F⋆ encodes pre/postconditions in Hoare logic style (Section 2.4.2). We next explain the Low⋆ memory model and modifies clauses, which are central to heap reasoning in the GC (Section 2.4.3). The SMT tuning techniques used to discharge the more demanding proof obligations in this development are discussed in Section 4.5.4 of Chapter 4.

2.4.1 Refinement Types and Key Language Constructs

F⋆ is a proof-oriented language combining dependent types with SMT automation. We highlight the constructs used directly in Chapter 4.

Refinement types. A refinement type `x:t{P}` constrains values of type `t` by a proposition `P`, where `P` is a logical formula that may express arbitrary pure properties, not merely decidable boolean conditions. For example, `nat` is defined as

```
type nat = n:int{n >= 0}
```

F⋆ uses Z3 to verify that refinements are preserved across all operations, eliminating the need for runtime checks. Refinement types are the primary mechanism by which we encode GC invariants, such as valid heap indices and non-negative word counts, directly in function signatures.

Dependent types. F⋆ is a fully dependently typed language, meaning that types may depend on arbitrary pure expressions, not just values. This enables precise value-dependent specifications. For example, the signature of the function `replicate` below,

```
val replicate: n:nat -> a:Type -> l:list a{length l = n}
```

guarantees that the returned list `l` contains exactly `n` elements, with the length constraint enforced statically by the type checker rather than at runtime.

Logical assertions, assumptions and admits. `assert` obligations are discharged by Z3 whereas `assume` introduces an axiom without proof and should be used sparingly. A

related construct is `admit()`, which unconditionally accepts any goal as proved and is used as a place holder during incremental proof development. Unlike `assume`, which asserts a specific proposition, `admit()` silently closes any open proof obligation and should never appear in a final verified development.

```
assert (x >= 0);    // proven by SMT
assume (x < 100);   // taken as axiom; use with care
admit()             // proof obligation deferred; unsound
```

2.4.2 Specifications in Hoare Logic Style

F* encodes Hoare triples (Hoare, 1969) $\{P\}\ c\ \{Q\}$ through `requires` and `ensures` clauses in function types, discharged automatically by Z3. F* supports two specification styles. **Intrinsic specifications** embed the contract directly in the function's type, so the property is verified once at definition time and every caller benefits automatically. Consider the absolute value function `abs`, which takes an integer `x` and returns its absolute value. The postcondition captures exactly what we expect: the result is non-negative and equals either `x` or `-x`:

```
val abs : x:int -> Pure int
  (requires True)
  (ensures  (fun r -> r ≥ 0 ∧ (r = x ∨ r = -x)))
let abs x = if x >= 0 then x else -x
```

Extrinsic specifications, by contrast, separate the proof from the implementation. The function is defined without a contract, and its correctness properties are established afterwards as standalone lemmas. Here, `abs_correct` states and proves the same postcondition as above, but independently of the function definition:

```
val abs_correct : x:int -> Lemma
  (ensures (abs x ≥ 0 ∧ (abs x = x ∨ abs x = -x)))
let abs_correct x = ()
```

The empty body suffices because Z3 can verify the postcondition by unfolding the definition of `abs` and checking both branches of the conditional. The lemma is then invoked explicitly at any proof point where the property is needed, introducing it into the local context for Z3 to use.

The effect `Pure` marks a total, side-effect-free computation: it is guaranteed to terminate and may not access or modify the heap. The `Lemma` effect is a specialisation of `Pure` used to state and prove logical properties. A `Lemma` has return type `unit` and its `ensures` clause captures the property being established. Its body is a proof term and is erased during extraction, so it carries no runtime cost. In our GC development, `Lemma` is used throughout Layers 1 and 2 for reachability properties such as `dfs_reachability_lemma` (Figure 4.17) and `mark_reachability_lemma` (Figure 4.13).

In our GC verification, intrinsic specifications are used wherever a function must preserve a global heap invariant. For example, the structural properties of the mark stack must hold at every call site. Whereas complex algorithmic properties such as reachability and invariant preservation across GC phases are proved extrinsically as lemmas and invoked where Z3 requires them.

2.4.3 A Primer on Low*

Low* is a subset of F* designed for verified low-level programming. It provides a C-compatible memory model built on F*'s HyperStack, which organises memory into a hierarchy of regions corresponding to stack frames and heap allocations. Low* programs extract to idiomatic C via the KaRaMeL compiler. The primary data structure is the `buffer`, defined in the `LowStar.Buffer` module, which models a contiguous, fixed-length

region of heap memory — the Low* analogue of a C array. Verifying a GC requires reasoning about which heap regions are modified and which are left untouched. This kind of reasoning is essential to ensure memory safety. Low* encodes memory safety through three key predicates in `LowStar.Buffer` that allow precise reasoning about which memory locations are read and written. Specifically, these predicates are:

1. `B.live m b` - buffer `b` is allocated and valid in heap state `m`, ensuring that any access to `b` is within bounds and to live memory.
2. `B.loc_disjoint (B.loc_buffer b1) (B.loc_buffer b2)` - buffers b_1 and b_2 occupy disjoint memory regions, ruling out aliasing between the two locations.
3. `B.modifies l m0 m1` - only the memory locations in footprint `l` were modified between states m_0 and m_1, and everything outside `l` is preserved. This allows callers to conclude that buffers outside the footprint are unaffected without enumerating them explicitly in the postcondition.

To illustrate some of the features of Low* that we will use later in the thesis, a sample Low* code to swap the contents of two memory locations r_0 and r_1 is shown in Figure 2.6, along with its specification in the form of pre and postconditions. We explain much of the Low* syntax through comments in the code. Low* operates on a C-like memory model with explicit heap and stack memory management, which is captured in the module `FStar.HyperStack`. `FStar.HyperStack.ST.get()` is used to obtain the contents of the heap memory at any program point. In Low*, C arrays are modelled using buffers, whose interface is defined in `LowStar.Buffer` module. Additionally, Low* provides support for machine integers of type `8`, `32` or `64` bits. `UInt8.t` is the type of `8` bit machine integers. The effect `HA.Stack` denotes a stateful computation in F*'s HyperStack memory model, restricting the function to the current stack frame.

The `swap` program takes as input, the buffers r_0 and r_1 of length 1, with the pre-conditions asserting that these buffers have been allocated space in memory and they are not aliases. Notice that the pre-condition takes as input the initial heap state as the argument `m`.

The postcondition is specified over both the initial heap state m_0 and the final heap state m_1. For specifying functional correctness in the postcondition, we use the function `LowStar.Buffer.as_seq` which converts the buffer to its sequence counterpart (a sequence is just a functional list). We use the function `Seq.index` to obtain the element of a sequence at a given index. The postcondition asserts that the value stored at index `0` in the location r_1 in the final heap is the value stored at index 0 in location r_0 in the initial heap, and vice versa. The extracted C code from the Low* code is shown in Figure 2.7. Note that the extracted code is readable, idiomatic C that preserves the verified properties from the Low* source and the `modifies` clause ensure that `swap` only modifies the two buffer locations and leaves the rest of memory unchanged.

2.5 CHAPTER SUMMARY

This chapter provided the background necessary to understand the formal development of a verified garbage collector for OCaml presented in subsequent chapters. We began with an overview of the garbage collection design space (Section 2.2), surveying the principal collection strategies, such as mark-and-sweep, copying, mark-compact, and reference counting, along with incremental, concurrent, and generational variants. Among these, mark-and-sweep occupies a particularly suitable position for formal verification as objects are never moved, no write barrier is required in the stop-the-world setting, and correctness reduces cleanly to reachability in a static heap graph. These properties make it the natural starting point for a layered, proof-oriented development.

We then described the OCaml object representation and heap organisation (Section 2.3). Every OCaml object carries a word-sized header encoding its size, color, and tag, with the least significant bit of each field distinguishing pointers from immediate values. The tag determines how the GC treats an object. The objects with tags below `No_scan_tag` have fields that may contain pointers and must be traced, while objects with tags at or above `No_scan_tag` contain only opaque bytes and are skipped. Closure objects, which are the

```
module HA = FStar.HyperStack.All
module ST = FStar.HyperStack.ST
module B = LowStar.Buffer

let swap (r0 r1 : B.buffer UInt8.t)
  : HA.Stack (unit)
  (* PRE-CONDITIONS *)
  (* B.live predicate ensures that buffers must be allocated before
     their use *)
  (λ m -> B.live m r0 ∧ B.live m r1 ∧
     (* Unit length buffers *)
     B.length r0 == 1 ∧ B.length r1 == 1 ∧
     (* Buffer memory locations are not aliased *)
     B.loc_disjoint (B.loc_buffer r0) (B.loc_buffer r1))

  (* POST-CONDITIONS *)
  (λ m0 _ m1 ->  B.live m0 r0 ∧ B.live m1 r1 ∧
     (* Explicitly specify which memory locations is modified *)
     (B.modifies (B.loc_union (B.loc_buffer r0)
                              (B.loc_buffer r1)) m0 m1) ∧
     (* Encode functional correctness *)
     Seq.index (B.as_seq m1 r0) 0 == Seq.index (B.as_seq m0 r1) 0 ∧
     Seq.index (B.as_seq m1 r1) 0 == Seq.index (B.as_seq m0 r0) 0) =
      (* Initial memory m0 *)
       let m0 = ST.get() in
      (* Initial values in the single length buffers r0 and r1 *)
       let r0_val = !*r0 in
       let r1_val = !*r1 in
      (* Asserts that, if we convert the buffer to its functional seq
          data type counterpart, the value at index 0 is r0_val and
          similarly r1_val *)
      assert (Seq.index (B.as_seq m0 r0) 0 == r0_val);
      assert (Seq.index (B.as_seq m0 r1) 0 == r1_val);
      (* Updates r0 at index 0 as r1_val and r1 at index 0 as r0_val *)
       r0.(0ul) <- r1_val;
       r1.(0ul) <- r0_val;
      (* Get the new memory state after the swap *)
       let m1 = ST.get() in
      (* Asserts that the values are swapped *)
      assert (Seq.index (B.as_seq m1 r1) 0 == r0_val);
      assert (Seq.index (B.as_seq m1 r0) 0 == r1_val);
      (* Return type is unit, equivalent to C void *)
      ()
```

Figure 2.6: A Low* code to swap the contents of two memory locations r_0 and r_1.

```c
void swap(uint8_t *r0, uint8_t *r1)
{
  uint8_t r0_val = *r0;
  uint8_t r1_val = *r1;
  r0[0U] = r1_val;
  r1[0U] = r0_val;
}
```

Figure 2.7: Extracted C code from the Low⋆ code in Figure 2.6

runtime representation of OCaml functions, require special treatment because the GC must use the start-of-environment offset stored in the closure info field to locate the fields that may contain pointers. Mutually recursive functions introduce infix objects embedded within closure blocks, which act as interior pointers. This means when the GC encounters a pointer to an infix object, it must redirect to the parent closure rather than treating the infix object as an independent heap object. These object-level details directly shape the well-formedness conditions, graph construction, and marking behaviour formalised in Chapter 4.

We situated our contribution within the OCaml runtime by describing the OCaml 4.14.1 major collector (Section 2.3.2), which is the algorithm our verified GC implements. We also discussed OCaml 5.0 (Section 2.3.3), whose introduction of shared-memory parallelism across domains fundamentally changes the correctness argument. The color invariants that are sequential properties in 4.14.1 must be re-established under concurrent access in 5.0. The sequential correctness properties established in this thesis remain valid in the single-domain case of OCaml 5.0 and provide a natural foundation for any future concurrent verification effort.

Finally, we introduced the F⋆/Low⋆ verification framework (Section 2.4). F⋆ supports refinement and dependent types, Hoare-logic-style pre and postconditions discharged by the Z3 SMT solver, and both intrinsic and extrinsic proof styles. Low⋆ extends F⋆ with a C-compatible memory model in which reasoning about heap memory is encoded

through liveness, disjointness, and modifies-clause predicates, and from which verified code is extracted to C via the KaRaMeL compiler. The trusted computing base of our verification comprises F*'s type checker, Z3, and KaRaMeL, consistent with projects such as CompCert and seL4. Together, these tools provide the mechanisms needed to connect abstract graph-reachability specifications to concrete, memory-safe C implementations of the GC.

CHAPTER 3

RELATED WORK

When there are multiple solutions to a problem, choose the simplest one.

– John C. Bogle

3.1 INTRODUCTION

Garbage collection sits at the heart of most managed language runtimes, yet getting it right is surprisingly hard. A collector that frees a live object, fails to reclaim a dead one, or silently corrupts a pointer, can bring down an entire program in ways that are notoriously difficult to reproduce and diagnose. Making matters harder, GC implementations do not live in isolation because they are tightly coupled to the memory layout, calling conventions, and runtime assumptions of the language they serve, which makes formal reasoning about them both important and technically demanding.

This chapter surveys the works on formal methods that are most relevant to ours. We start with GC verification (Section 3.2), moving from the most abstract treatments, such as proofs about algorithm descriptions, through verification against compiler intermediate representations, to end-to-end verifications that connect a high-level specification all the way to deployable code, and finally to the additional complications that arise in concurrent settings. From there, we turn to the work on which our own builds more directly. Since our correctness specifications are graph-theoretic in nature, we first survey verified graph libraries and graph algorithm proofs in proof assistants (Section 3.3). We then examine separation logic and the tools used to reason about heap-manipulating programs, including the F*/Low* framework we adopt (Section 3.4). Finally, we look at verified explicit memory management systems, such as allocators and OS kernel memory managers (Section 3.5), whose goals and techniques are closely aligned with our own.

The thread running through all of this is that while the individual pieces, such as abstract GC correctness, graph reachability reasoning, low-level C verification, have each received serious attention, no prior work brings all three together in a single end-to-end verified, portable implementation integrated with a production language runtime.

3.2 GARBAGE COLLECTOR VERIFICATION

Previous work on verifying garbage collectors has either used pen-and-paper proofs or mechanized proofs using theorem provers. Mechanized verification has a significant advantage over pen-and-paper arguments because the proof checker enforces that every step is justified, ruling out the subtle errors and logical gaps that frequently appear in hand-written GC proofs. Our discussion, therefore, focuses primarily on mechanically verified collectors.

While numerous previous works (Russinoff, 1994; Gonthier, 1996; Havelund, 1999; Jackson, 1998; Goguen *et al.*, 1998; Burdy, 2001; Coupet-Grimal and Jakubiec, 2003) have addressed the problem of GC verification, most have tended to focus exclusively on verifying abstract models of GC, instead of actual implementations. The following subsections discuss the four broad categories of mechanically verified collectors, such as verification at the model level, verification against intermediate representations, end-to-end verification, and the unique challenges of concurrent collectors.

3.2.1 Model-Based and Abstract-Level Verification

The earliest mechanized GC proofs aim to establish the correctness of garbage collection algorithms expressed as abstract state machines or pseudocode, without committing to a concrete implementation language or memory model. Coupet-Grimal (Coupet-Grimal and Jakubiec, 2003) verified Ben-Ari's incremental mark-and-sweep algorithm in Coq using a shallow embedding of linear temporal logic. They establish safety and liveness properties of the algorithm, and their temporal reasoning framework has become a reusable component for subsequent algorithm-level proofs.

Gonthier (Gonthier, 1996) verified a tricolor invariant-based GC in a foundational setting by relating it directly to the standard formal description of on-the-fly garbage collection. Although their work is not implemented as executable code, the proof provided early evidence that the subtle mutual invariants between the mutator and the collector could be

rigorously captured in a proof assistant.

Gammie et al. (Gammie *et al.*, 2015) verified a detailed model of a state-of-the-art on-the-fly, concurrent mark-and-sweep collector in Isabelle/HOL with respect to an x86-TSO memory model (Sewell *et al.*, 2010). Their work is notable for its fidelity to a production-grade algorithm (the collector used in the Azul Systems Zing JVM). Still, it operates on an abstract model rather than the actual collector code. Our work verifies at both abstract and concrete implementation levels, with the C code extracted after verification integrated into the OCaml runtime.

Pavlovic et al. (Pavlovic *et al.*, 2010) focus on an earlier stage in the design process, proposing a methodology to *derive* concurrent collection algorithms from abstract specifications by stepwise refinement. The algorithms obtained are at a similar level of abstraction to that from which Gammie et al. begin, suggesting a productive complementary approach to the verify-first style.

Xu et al. (Xu *et al.*, 2022) propose a model checking framework for gaining confidence in collector correctness by systematic state-space exploration, but do not present a concrete verified implementation. Model checking of this kind is complementary to deductive verification because it can find bugs rapidly on finite abstractions, but cannot provide the unbounded guarantees that a mechanized proof delivers.

3.2.2 Verification of Intermediate Representations

A second category of work verifies collector implementations written in a compiler intermediate representation (IR) rather than actual executable code. These verifications are typically machine-checked and provide stronger guarantees than model-level proofs, yet they stop short of producing deployable code.

Zakowski et al. (Zakowski *et al.*, 2017) used Coq to verify a concurrent mark-and-sweep collector expressed in a purpose-built compiler IR. Their work used a rely-guarantee

methodology to decompose the concurrent correctness argument, where the rely condition captures the mutator's actions on the shared state between collector steps. The guarantee condition, on the other hand, shows what the collector has promised to the mutator. This breakdown lets the proof handle fine-grained thread interleavings without listing all possible schedules. Compared to Gammie et al., who check the same type of algorithm at the pseudocode level, Zakowski et al. make the proof more like a real implementation. Still, the IR in question is not compiled to native code and cannot be linked against an existing language runtime.

Lin et al. (Lin *et al.*, 2009) present the verification of a Yuasa-style incremental garbage collector in a Hoare-style proof-carrying code (PCC) framework, which is a Stack-based Certified Assembly Programming (SCAP) system (Feng *et al.*, 2006) augmented with embedded separation-logic primitives. Their Coq development ensures that the collector always preserves the heap objects reachable by the mutator, building on their earlier work verifying a stop-the-world mark-sweep collector (Lin *et al.*, 2007). A simplifying assumption throughout their work is that every object has exactly two fields. Therefore, supporting variable-length objects with heterogeneous field types, as OCaml requires, poses additional verification challenges, which we address in our development.

3.2.3 End-to-End and Code-Level Verification

End-to-end verification closes the gap between an abstract specification and deployable code, typically by extracting executable C or assembly code from the verified artefact. Hawblitzel and Petrank (Hawblitzel and Petrank, 2009) verified a mark-and-sweep collector and a Cheney copying collector implemented in x86 assembly using Boogie (Barnett *et al.*, 2005) and Z3 (de Moura and Bjørner, 2008), where the proof obligations are discharged through extensive code annotations. Their verification does not define GC correctness in terms of object reachability; instead, it relies on object color invariants specific to the Bartok compiler and runtime for .NET bytecode format (Hunt *et al.*, 2005). It cannot address the intricacies arising from OCaml's

memory layout and its integration with the OCaml runtime system. Therefore, it is not straightforward to adapt it to a different collector design. In contrast, our correctness specifications are stated at an abstract graph-theoretic level and are therefore design-agnostic. Furthermore, their assembly-level verification lacks the portability offered by a C implementation. As we discuss in Chapter 5, Section 5.4.2, the same specifications can be used to describe the correctness of a copying collector with minor modifications at the abstract level that make no use of object colors.

McCreight et al. (McCreight *et al.*, 2007) provide a framework for verifying both GC and mutators in Coq, certifying mark-and-sweep and Cheney copying collectors written in a RISC-like assembly language through a common framework based on abstract data types (ADTs), which are later refined by various collectors. The Coq proof development runs to approximately 64,000 lines and uses no axioms beyond those built into Coq's kernel. McCreight's framework introduces a representation-predicate abstraction that hides GC internals from the mutator and provides a clean interface between the mutator and the GC. The limitation common to all assembly-level approaches is portability: a verified x86 or MIPS collector cannot be run on ARM, RISC-V, or IBM s390x without a separate verification effort. Our approach extracts portable C code from the verified Low* source, enabling deployment on all platforms supported by OCaml.

Ericsson et al. (Ericsson *et al.*, 2019) describe the verification of a generational copying collector for the CakeML runtime, carried out entirely in HOL4 and integrated with CakeML's in-logic bootstrapped compiler (Kumar *et al.*, 2014). Similar to our work, they adopt a layered approach, separating an algorithm-level proof from an implementation proof that connects HOL4 function definitions to concrete memory layouts. Their integration of the verified GC with the rest of the CakeML runtime is a notable contribution. However, mark-and-sweep GCs require a completely different form of reasoning compared to copying collectors. One of the main highlights of our work is that it deals with the verification of a mark-and-sweep GC operating on OCaml-style objects,

where alignment with the OCaml object layout is an essential factor for integrating the GC with the rest of the OCaml runtime. The CakeML compiler generates a new collector implementation for each configuration of the data representation. In contrast, our verified GC is a fixed implementation that must handle the full diversity of OCaml object types, including closures, infix objects, and no-scan objects under a single verified specification.

Wang et al. (Wang *et al.*, 2019) develop a mathematical and spatial graph library in Coq, and use it to verify a generational copying collector for the CertiCoq project (Anand *et al.*, 2017). Correctness is stated as preservation of an abstract graph isomorphism induced by the copying function. Their object layout is similar to ours, with a header followed by a variable number of fields; however, because CertiCoq targets purely functional programs, their setting is simpler than ours. Their 400-line verified collector is sufficient for CertiCoq's purposes, whereas supporting OCaml's richer object taxonomy requires additional infrastructure that our development provides.

Verma et al. (Verma *et al.*, 2000) present an early example of a verified garbage collector derived through proof assistant extraction. As part of a broader effort to formalise binary decision diagrams (BDDs) in Coq, they implement and verify a garbage collector for their BDD representation and extract the resulting certified algorithms into Caml via Coq's extraction mechanism. The high-level ambition is similar to ours. That is, getting verified, executable code out of a proof assistant. But the two works differ in what that collector has to deal with. Theirs operates on a single, well-structured data type within the comparatively clean memory model of a Coq formalisation. Ours must handle OCaml's full object model as it exists in a production runtime: header words, tag-based dispatch, closures, infix objects, and fields that may or may not hold pointers. The extraction target also reflects this difference: their output is functional Caml, while ours is portable C via Low*, compiled and linked directly against the OCaml runtime.

Our work is a mechanically verified major garbage collector for OCaml 4.14.1,

implemented in F*/Low* and integrated with the OCaml runtime (Shamsu *et al.*, 2025). The correctness specifications are stated at an abstract graph-theoretic level, independent of GC implementation details, so that the same specification framework can, in principle, be used to describe the correctness of an incremental variant or a copying collector without a drastic revision. While our work builds on many of the ideas proposed in previous works, this is the first end-to-end verified and portable GC implementation integrated with the OCaml runtime environment, which is a significant milestone in the journey towards establishing a highly performant, verified, and robust GC for OCaml.

3.2.4 Concurrent Collector Verification

Concurrent collectors, in which the collector and mutator run in parallel threads, introduce verification challenges distinct from those in sequential settings. Different proof techniques are needed to deal with race conditions, relaxed memory models, and the lack of atomic per-step invariants.

To the best of our knowledge, and of Gammie et al. (Gammie *et al.*, 2015), their work is the first fully machine-checked safety proof for a concurrent collector under relaxed memory semantics.

Zakowski et al. (Zakowski *et al.*, 2017) verify a concurrent collector in a purpose-built IR using rely-guarantee reasoning, where functional correctness is established under fine-grained thread interleavings.

Pavlovic et al.'s synthesis approach (Pavlovic *et al.*, 2010) produces concurrent collection algorithms that are correct by construction with respect to their abstract specification, thereby eliminating the need for a separate post-hoc correctness proof.

As discussed in Section 2.3.3, the OCaml 5.0 runtime requires concurrent separation logic or rely-guarantee reasoning to handle its multi-domain VCGC scheme. Extending our verification to the concurrent setting would require applying frameworks such as

Pulse (Ebner *et al.*, 2025), a concurrent separation logic embedded in F* that provides built-in support for reasoning about shared mutable state and fine-grained concurrency, which we identify as important future work.

Table 3.1 summarises the works discussed in this section along the axes most relevant to our contribution. We use the following abbreviations: STW (stop-the-world), M&S (mark-and-sweep), Copy (copying), Alg (algorithm-specific), Reach (reachability-based), Asm (assembly), Gen (generational), Inc (incremental), Con (concurrent), and Lang (language runtime).

Table 3.1: Comparison with related work on verified garbage collectors

Work	**Mode**	**Algo.**	**Spec**	**Code**	**Lang.**
Hawblitzel (Hawblitzel and Petrank, 2009)	STW	M&S/Copy	Alg	Asm	C#
Ericsson (Ericsson *et al.*, 2019)	Gen	Copy	Reach	Asm	CakeML
McCreight (Mccreight, 2008)	STW/Inc	M&S/Copy	Reach	Asm	–
Gammie (Gammie *et al.*, 2015)	Con	M&S	Reach	Model	–
Zakowski (Zakowski *et al.*, 2017)	Con	M&S	Reach	Model	–
Verma (Verma *et al.*, 2000)	–	Custom	Alg	Caml	BDD
Ours	**STW**	**M&S**	**Reach**	**C**	**OCaml**

From Table 3.1, one can see that ours is the only work that combines a reachability-based correctness specification, a portable C implementation, and direct integration with a production language runtime.

3.3 GRAPH-BASED VERIFICATION

A central novelty of our approach is the separation of abstract GC correctness from its implementation: the former is stated entirely in terms of the object-reachability graph, and the latter connects Low* code to that graph. This design decision makes the correctness specifications design-agnostic, but it also creates a dependency on a verified graph infrastructure.

3.3.1 Graph Libraries in Proof Assistants

Several proof-assistant libraries provide reusable graph theory. Noschinski's graph library for Isabelle (Noschinski, 2015) covers paths, reachability, and connectivity for both directed and undirected graphs and has been used in several downstream verifications. Doczkal and Pous (Doczkal and Pous, 2020) develop a Coq/Ssreflect graph library covering graph minors, treewidth, and isomorphisms, building on the Mathematical Components (MathComp) ecosystem. Wang et al. (Wang *et al.*, 2019) develop a mathematical and spatial graph library in Coq specifically oriented toward heap graphs, making it directly applicable to GC verification; their library handles pointer-structured heaps as directed graphs and provides the lemmas required to state and prove copying-GC correctness as a graph isomorphism.

3.3.2 Verified Graph Algorithms

Verification of graph algorithms is a well-studied area. Russinoff (Russinoff, 1994) gave an early mechanized proof of graph connectivity. Gonthier (Gonthier, 1996) verified properties of graph traversal in the context of GC. Jackson (Jackson, 1998) and Havelund (Havelund, 1999) verify graph properties in model-checking and specification settings.

Lammich and Neumann (Lammich and Neumann, 2015) provide a framework in Isabelle for verifying DFS-based algorithms by stepwise refinement. In their work, a high-level abstract algorithm is successively refined into imperative code, where each refinement step is mechanically checked. Their framework has been applied to reachability, topological sort, and strongly connected components.

Guéneau et al. (Guéneau *et al.*, 2019) use a program logic to verify both the correctness *and* the complexity of an incremental cycle detection algorithm, demonstrating that formal methods can capture performance guarantees as well as functional ones, which is a useful template for future GC verification.

Chen et al. (Chen *et al.*, 2019) verify Tarjan's strongly connected components algorithm in Why3, Coq, and Isabelle. The SMT-based backend of Why3 struggles with induction over paths of arbitrary length, which arises naturally in graph reachability arguments. They used Coq to finish these proofs. We encountered a similar challenge in our F* development. Like Chen et al., our proofs involve reachability over paths of arbitrary length, and Z3 alone cannot perform induction over such structures. We addressed this by decomposing the inductive argument into smaller lemmas that Z3 can handle individually, guiding the solver through the inductive structure step by step. Their comparison of tools makes one thing clear: proof obligations that require induction over unbounded structures remain hard for SMT-based tools, regardless of the framework.

Pottier (Pottier, 2015) verifies depth-first search and strong connectivity in Coq using functional (rather than imperative) algorithms, providing reusable reachability lemmas.

A distinctive feature of our work is a clean architectural separation between graph-theoretic reasoning and GC implementation reasoning. The graph module we developed, covering reachability, connectivity, and subgraph inclusion, has been proved correct independently of any GC-specific concerns, and the GC correctness proofs are then built on top of it by instantiating its lemmas to the OCaml heap graph. This layering makes the graph module a self-contained, reusable component that can be used for any future verification that considers pointer reachability, such as a different collector design, a runtime data structure, or a program analysis.

3.4 VERIFICATION OF HEAP-MANIPULATING PROGRAMS

Reasoning about programs that dynamically allocate and modify heap structures comes under the domain of *separation logic* and its descendants. Separation logic, introduced by Reynolds and O'Hearn (Reynolds, 2002; O'Hearn *et al.*, 2001), extends Hoare logic with the *separating conjunction* `P * Q`, where `P` and `Q` are heap predicates which are logical assertions describing properties of portions of the heap, such as the well-formedness of a

linked list or a tree rooted at some pointer. Then, `P * Q` asserts that both `P` and `Q` hold simultaneously over *disjoint* regions of the heap. The separating conjunction enables the *frame rule*, which allows local reasoning: if a piece of code only touches the region described by `P`, one can conclude that `Q` is entirely unaffected, without reasoning about what `Q` says at all.

3.4.1 Foundations and Tooling

The Verified Software Toolchain (VST) (Appel *et al.*, 2014) provides a machine-checked separation Hoare logic for C programs, sound with respect to the CompCert C (Leroy, 2009) operational semantics. VST-Floyd (Cao *et al.*, 2018) adds a set of semi-automatic tactics that help users construct functional-correctness proofs for C programs in Coq. VST has been used to verify a range of low-level programs, including network stacks, cryptographic implementations, and memory allocators, which are directly relevant to our work (see Section 3.5).

Iris (Jung *et al.*, 2018, 2015) is a higher-order concurrent separation logic framework implemented in Coq, parametrized over the programming language one wishes to reason about. Iris supports impredicative invariants, higher-order ghost state, and weakest-precondition style proofs, making it suitable for fine-grained concurrent programs. It has been deployed on a wide variety of verification projects, including concurrent data structures, Rust unsafe code, and language semantic models. The rely-guarantee reasoning needed to handle OCaml 5.0's concurrent collector (Section 2.3.3) could in principle be expressed within Iris.

Birkedal et al. (Birkedal *et al.*, 2004) gave an early local-reasoning account of a copying GC in separation logic, showing how the collector and the mutator can be given disjoint heap ownership throughout the collection, with a clean "handshake" at the interface. Their work foreshadows our own invariant structure but does not produce a mechanized proof.

Mehta and Nipkow (Mehta and Nipkow, 2003) develop a framework in Isabelle/HOL for mechanically verifying pointer programs in a separation-logic style. The development includes proofs of the frame rule and shows that proofs of small heap operations can be composed.

3.4.2 Automated Reasoning Tools

To formally verify garbage collectors, we need tools that can handle complex invariants over graph structures and low-level memory operations simultaneously. Different verification frameworks have different trade-offs in terms of automation, expressiveness, and code-generation capability.

SMT-based verifiers. Boogie (Barnett *et al.*, 2005) provides an intermediate verification language for SMT-based reasoning. Dafny (Leino, 2010), built on Boogie, offers strong SMT automation but targets high-level languages (.NET, Java, JavaScript) rather than C. VCC (Cohen *et al.*, 2009*b*) extends C with verification annotations whereas Why3 (Filliâtre and Paskevich, 2013) supports multiple backend provers but extracts to OCaml/WhyML rather than C.

Interactive theorem provers. Coq (Bertot and Castéran, 2013) has been extensively applied to GC verification. Coq offers maximum expressiveness but requires significant manual proof effort and does not support C code generation. Isabelle/HOL (Nipkow *et al.*, 2002) provides powerful automation (Sledgehammer, auto). Lean (de Moura *et al.*, 2015; de Moura and Ullrich, 2021) combines dependent types with SMT-like automation. Recent research shows Lean 4's growing relevance to systems programming. Lean 4 can generate C code via its compiler backend; however, this code is closely tied to the Lean runtime and is not intended for direct integration with external C-based systems such as the OCaml runtime. In contrast, Low*'s KaRaMeL produces portable, readable C that can be compiled with standard toolchains and linked with existing runtime components, which better fits our integration goals.

Separation logic tools. Separation logic (Reynolds, 2002) provides an intuitive framework for reasoning about programs that manipulate heaps using separating conjunction and the frame rule.

Smallfoot (Berdine *et al.*, 2005) pioneered automated separation logic verification with symbolic execution and shape analysis. VeriFast (Jacobs *et al.*, 2011) extends this to full functional correctness for C and Java programs, with manual separation-logic annotations, but does not generate code. Infer (Calcagno *et al.*, 2015), deployed at Meta, performs lightweight separation logic analysis for bug-finding at scale, where they analyze hundreds of millions of lines of production code.

GRASShopper (Piskac *et al.*, 2014) offers comprehensive heap verification through the integration of separation logic, first-order logic, and SMT solving. It is especially useful for GC verification because it can combine spatial predicates with arithmetic reasoning, which is needed when both pointer manipulation and address arithmetic must be performed simultaneously. SuSLik (Polikarpova and Sergey, 2019), a deductive synthesis tool, generates programs that manipulate heaps directly from separation logic specifications. Neis (Neis *et al.*, 2021) show that SuSLik's derivations can be verified in VST, HTT, and Iris, which connects synthesis and verification. Iris (Jung *et al.*, 2018), a higher-order concurrent separation logic framework in Coq, has been successfully applied to concurrent GC verification (Section 3.2.2), though it inherits Coq's manual proof burden.

The F* ecosystem. F* (Swamy *et al.*, 2016; Martínez *et al.*, 2019) combines dependent types, refinement types, and SMT automation. Low* (Protzenko *et al.*, 2017) extends F* with a C-compatible memory model and a C code extraction via a compiler called KaRaMeL. Steel (Fromherz *et al.*, 2021) and Pulse (Ebner *et al.*, 2025) add concurrent separation logic for future concurrent GC verification. Production deployments include HACL* (Zinzindohoué *et al.*, 2017), EverCrypt (Protzenko *et al.*, 2020), and

miTLS (Bhargavan *et al.*, 2017*b*).

3.4.3 Choice of Verification Framework

Among the frameworks surveyed above, we chose F⋆/Low⋆ for this work. It combines several features that matter for GC verification: mostly automatic VC discharge through Z3, a region-based memory model with stack and heap discipline enforced at the type level, first-class mutable buffers with precise length and liveness tracking, low-level control over memory layout and pointer arithmetic, and extraction to portable C code via KaRaMeL. This last point is particularly important since KaRaMeL generates C that can be compiled with any standard tool-chain and linked directly with the OCaml runtime. VST, by contrast, verifies programs at the level of CompCert's Clight representation, so its guarantees are preserved only when compiled with CompCert. Coq, while expressive and widely used for GC verification, extracts primarily to OCaml or Haskell and requires significant additional tooling to produce low-level C. For our goal of a verified, portable C implementation that integrates smoothly with the OCaml runtime, F⋆/Low⋆ is the most direct path.

3.5 VERIFICATION OF OTHER MEMORY MANAGEMENT SYSTEMS

Garbage collectors are not the only components of a runtime's memory subsystem. Rather, explicit allocators, free-list managers, and OS-level page managers have also been subject to formal verification. This area of work complements GC verification, and some of the techniques transfer directly.

3.5.1 Verified Allocators

Appel and Naumann (Appel and Naumann, 2020) verify the functional correctness of a segregated free-list `malloc`/`free` implementation in standard C using Verifiable C (VST) in Coq. Their *resource-aware* specification guarantees that `malloc` returns a block whenever sufficient resources are available, a stronger guarantee than the standard POSIX specification requires. They also prove a subsumption theorem relating the resource-

aware and resource-oblivious specifications. Together with our work, this demonstrates that the two central components of a runtime memory subsystem, the collector and the allocator, are each independently amenable to machine-checked verification, raising the prospect of a fully verified runtime memory subsystem as a longer-term goal.

StarMalloc (Reitz *et al.*, 2024*b*) verifies a security-hardened memory allocator in F* using the Steel framework, demonstrating that F* scales to the verification of production systems. As the allocator and collector are complementary runtime components, this suggests a future in which we can have a fully verified runtime memory subsystem.

Marti et al. (Marti *et al.*, 2006) give an early demonstration of separation logic applied to an OS-level heap manager in Coq, working within an almost-C language fragment which is an approach analogous in spirit to our own use of Low* as a restricted, verifiable subset of C. Jiang et al. (Jiang *et al.*, 2019) provide a formally verified buddy allocator model in Isabelle/HOL, establishing that key block invariants are preserved under allocation and deallocation.

3.5.2 OS Kernel Memory Management

Klein et al. (Klein *et al.*, 2009) and the comprehensive follow-up by Klein et al. (Klein *et al.*, 2014) present the formal verification of the seL4 microkernel, including its memory management subsystem. seL4 takes an unusual approach in which kernel memory allocation is *explicit* and *capability-controlled*, with all kernel objects statically allocated at boot time or created by user-level code via typed capabilities. This design simplifies verification of the allocation mechanism (though not of the policy), and the resulting proof, carried out in Isabelle/HOL, spans the full chain from abstract specification to binary code, constituting one of the largest machine-checked correctness proofs for systems software to date.

The CertiKOS project (Gu *et al.*, 2015) verifies an entire certified OS kernel by decomposing the system into abstraction layers, each verified independently and composed

via a certified linking framework. Memory management emerges as a layer, and the layered verification methodology aligns with our own approach of separating abstract GC correctness from OCaml-specific implementation correctness.

Tuch (Tuch, 2009) develops a formal C memory model in Isabelle/HOL that supports both low-level byte-accurate reasoning and high-level separation logic, and uses it to verify the memory allocator of the L4 microkernel. This model is a precursor to the CompCert memory model used in VST and illustrates the challenges of bridging typed abstract values and raw byte representations in C.

3.6 CHAPTER SUMMARY

Among GC verifications, the end-to-end verifications of Hawblitzel and Petrank, the CakeML generational collector, and the CertiCoq spatial graph library are the most similar to ours because all four share the ambition of connecting a high-level correctness statement to deployable code. Where our work differs is in its target, scope, and specification style. We verify a GC for a production-language runtime, handling the rich diversity of OCaml object representations, such as closures, infix objects, no-scan objects, and variable-length fields, that arise from OCaml's rich type system and compilation strategy. We also adopt reachability-based correctness specifications that are independent of implementation details such as color schemes, and connect them to portable C through a functional intermediate layer.

Graph-based reachability specifications are not new. For example, Wang et al. (Wang *et al.*, 2019) use graph isomorphism in CertiCoq, and Birkedal et al. (Birkedal *et al.*, 2004) use separation logic with reachability predicates. Our contribution is in how we combine these ideas and apply them to a production-language runtime with OCaml's complex object representations. To the best of our knowledge, no prior work simultaneously combines reachability-based correctness, support for a production-language runtime with complex object representations, and a connection to portable C through a functional

intermediate layer.

CHAPTER 4

A MECHANICALLY VERIFIED GARBAGE COLLECTOR FOR OCAML

The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise

– Edsger W. Dijkstra

4.1 INTRODUCTION

This chapter is adapted from the article entitled "A Mechanically Verified Garbage Collector for OCaml," published in the *Journal of Automated Reasoning* (Shamsu *et al.*, 2025).

In this chapter, we present our approach to developing a verified GC from scratch using a proof-oriented development methodology. This verification work was entirely performed in F* and its low-level subset Low*. Instead of trying to verify a feature, optimised GC, we start by designing and verifying a stop-the-world, single-heap, mark-and-sweep collector. We make this design choice because we believe formal verification of a GC should focus on identifying and understanding the key rules needed to ensure reachability preservation before adding optimisations and engineering complexity. We also demonstrate how we can evolve the bare minimum GC with additional language features and optimisations, along with the proof engineering changes required to adapt to these changes and verify the correctness of the GC.

Overview of Chapter Structure : The purpose of this chapter is to develop, improve, and validate our verified GC design. It progresses from general correctness principles to explicit assurances regarding actual memory. The fundamental concept of garbage-collector correctness is presented in Section 4.2. Without depending on any particular implementation details, it formalizes safety and liveness in terms of reachability over heap graphs.

Section 4.3 presents our approach to verifying a practical garbage collector for OCaml. It motivates the choice of a stop-the-world mark-and-sweep collector as the starting point and describes how we evolve the collector in a proof-oriented manner, beginning with a base GC model and progressively adding essential features until it is sufficient to integrate with the rest of the OCaml compiler.

Section 4.4 instantiates the abstract GC correctness definition from Section 4.2 for OCaml.

It introduces the F* definitions of the object graph, graph reachability, and the algebraic properties of the constituent GC functions `darken_roots`, `mark`, and `sweep`, and describes how these specifications connect to the abstract correctness definition.

The verified collector's overall proof-oriented architecture is shown in Section 4.5. It describes about our layered approach for verification, the limits of abstraction, and how the design enables step-by-step verification and modular reasoning.

The underlying graph library and a formally verified depth-first search are discussed in Section 4.5.1. In subsequent sections, this is the primary method of reasoning about reachability.

The construction of the GC itself is discussed in Section 4.5.2. It demonstrates how reusing and refining previous proofs can lead to the creation of more intricate GC variants. This section begins with basic definitions in F*, presents a basic mark-and-sweep collector, looks at heap manipulations and how they affect object sets, outlines the color-based header rules, establishes equivalence with graph-based marking, demonstrates sweep correctness using sub-graph reachability, and then expands the collector to support additional object types and coalescing during sweeping while preserving important invariants.

In Section 4.5.3, we show the implementation of our verified collector in Low* modelling low-level memory representation to link abstract heap models to real memory. It covers representation invariants, pointer interpretation, and layout.

The end-to-end correctness theorems are stated and discussed in Section 4.6, which shows that, in accordance with the abstract reachability specification, the concrete implementation of GC preserves safety and liveness.

Section 4.7 presents our proof-oriented approach to extending the minimal verified collector toward an integration-ready GC. The central idea is to enrich the collector by

supporting more complex object layouts and additional object types, while reusing the same high-level specifications established for the minimal collector.

Finally, Section 4.8 evaluates the verified GC in practice. It describes the integration of the extracted C code into the OCaml 4.14.1 bytecode runtime and presents a performance evaluation against the standard OCaml GC and the Boehm-Demers-Weiser conservative collector across a range of benchmarks.

4.2 ABSTRACT GC CORRECTNESS

The mark-and-sweep GC is primarily a graph algorithm. In particular, `mark` is a depth first traversal on the heap. Therefore, the correctness specification of garbage collection is most naturally expressed using graph theoretic terminology, rather than relying on the GC implementation details. The prime consideration for any GC is *soundness* – that is it only collects unreachable objects. A GC is said to be *complete* if it collects all the unreachable objects. We first formally define GC correctness in graph theoretic terms without any reference to the underlying implementation. Later, we will instantiate these definitions for our verified OCaml GC.

Let `h` denote the heap and `|h|` represents the length of the heap in bytes. Let `objs(h)` to be the set of all objects in the heap identified by their unique ids. The id of an object depends on the implementation. For example, in OCaml, the id of an object is the address of the first field. Let `allocs(h)` denote the set of allocated (not free) objects represented by their ids in `h`. Let `ptrs(x,h)` be the set of ids of the objects pointed to by `x`. Let `data(x,h)` be the set of non-pointer, opaque data fields of `x`.

Definition 4.1 (**Well-formed heap**)**.** A heap `h` is said to be well-formed, denoted by ω(`h`) iff ($\forall$ `x, y.` `x` $\in$ `allocs(h)` $\wedge$ `y` $\in$ `ptrs(x,h)` $\implies$ `y` $\in$ `allocs(h)`)

Definition 4.2 (**Object Graph**)**.** An object graph `G(h) = (V,E)` is constructed from a well-formed heap `h` as follows: the vertex set `V = allocs(h)`, and edge set

$E = \{(x,y) \mid x \in V \land y \in ptrs(x,h)\}$. The object graph is represented as $G(h)$.

Definition 4.3 (**Accessibility Relation**). Given $x, y \in allocs(h)$, x and y are related through the accessibility relation (denoted as $x \rightsquigarrow y$) if and only if either (1) $x = y$ or (2) $\exists\ z.\ z \in allocs(h) \land x \rightsquigarrow z \land y \in ptrs(z,h)$.

Definition 4.4 (**Reachable Sub-graph**). The reachable sub-graph $RG(h,r) = (V_{RG},\ E_{RG})$ is formed from a well-formed heap h and a root-set r. Let $G(h) = (V,E)$ be the graph constructed out of the heap h. Then,

- $(\forall\ x.\ x \in V_{RG} \iff x \in V \land (\exists\ y.\ y \in r \land y \rightsquigarrow x))$
- $(\forall\ x\ y.\ (x,y) \in E_{RG} \iff x \in V_{RG} \land (x,y) \in E)$

That is, $RG(h,r)$ only contains the accessible objects from r in h as vertices and the edges between accessible objects in h are preserved in $RG(h,r)$.

Definition 4.5 (**GC Correctness**). Let h_0 be the initial state of the heap on which the GC operates, such that $\omega(h_0)$ holds, and let r be the set of *roots*, which are pointers to objects into h_0. Let h_1 be the heap after the GC terminates and let V be the vertex set of $G(h_1)$. Then, the GC is said to be correct if:

1. $\omega(h_1)$ holds.
2. $G(h_1) = RG(h_0,r)$
3. $(\forall\ x.\ x \in V \implies data(x,h_0) = data(x,h_1))$

This definition addresses both *soundness* and *completeness* of the garbage collector (GC). Property 2 requires that the final object graph must exactly match the reachable subgraph of the initial heap. The forward direction indicates that every vertex in $G(h_1)$ is reachable from r in h_0, which indicates *soundness* or *safety* property. Conversely, the backward direction states that every object that is reachable from r in h_0 only appear in $G(h_1)$, thereby ensuring *completeness* or *liveness*. This ensures that no unreachable objects remain after collection. Property 1 guarantees that the resulting heap is well-formed, while Property 3 ensures that the non-pointer fields of all surviving objects are preserved.

The GC correctness definition says that, after the GC, the heap remains *well-formed*. The object graph after the GC is equal to the sub-graph of accessible objects from $\mathtt{r}$ in $\mathtt{h}_0$. This ensures that only the accessible objects are part of the object graph after the GC terminates, thereby ensuring completeness. Soundness is ensured as $\mathtt{RG}(\mathtt{h}_0,\mathtt{r})$ retains all the reachable objects and their interconnections. Additionally, the third correctness property ensures that the non-pointer fields of accessible objects remain the same.

We note that this definition of GC correctness is generic and applicable across different types of GCs. For example, in a copying collector, while the data fields and the object graph remains the same, the object themselves are moved. The generic GC correctness definition is able to accommodate this since it does not claim to preserve value of the pointer fields across the GC. We note that the main correctness theorem of (Ericsson *et al.*, 2019), which is a verified copying collector for CakeML also captures GC correctness similar to Definition 4.5. In Chapter 5, Section 5.4, we present the abstract correctness specifications for a copying collector as well as an incremental mark-and-sweep GC that uses a snapshot-at-the-beginning deletion barrier (Yuasa, 1990).

With the generic GC correctness specification in place, let us now move on to the implementation of an actual mark-and-sweep GC GC for OCaml in the next section. With the help of the implementation, we show how the abstract specifications are adapted specifically for OCaml and the mark-and-sweep GC (Section 4.4).

4.3 TOWARDS A VERIFIED OCAML GC

In this section, we present our approach to verify a practical garbage collector for OCaml. As mentioned in background chapter, OCaml uses a generational and incremental garbage collector, aimed at supporting high allocation rates and low latency. Given that verifying such a GC implementation is a challenging task, we develop a verified stop-the-world mark-and-sweep GC in a proof-oriented manner. We show in Chapter 5 (Section 5.4), how this GC may be extended to support copying and incremental mark-and-sweep

```c
void mark_and_sweep_GC (uint8_t *hp, uint64_t *st, uint64_t *tp,
                        uint64_t *r, uint64_t r_len, uint64_t *sw,
                        uint64_t *fp) {
  // GC initialization phase starts with pushing of roots
  // into the mark stack
  darken_roots (hp, st, tp, r, r_len);
  // GC mark phase is dfs that operates on different OCaml objects
  mark (hp, st, tp);
  // GC sweep phase frees unreachable objects and updates the free list
  sweep (hp, sw, fp);
}
```

Figure 4.1: Extract C code for the top-level stop-the-world mark-and-sweep GC function.

collection.

Our task involves connecting the abstract graph reachability specification defined in the previous section with performant C code, that involves low-level operations such as pointer arithmetic and bitwise operations. Since our aim is to integrate the verified GC with the rest of the OCaml compiler, our verified GC must be made aware of the different object layouts used by the compiler. We adopt a layered approach for verification similar to (Myreen, 2010; Ericsson *et al.*, 2019). The layered approach allows us to cleanly separate the abstract graph-based correctness from the low-level operations and language-specific features. We present the evolution of a proof-oriented practical GC for OCaml, by starting with a base GC model and then progressively adding essential features until it is sufficient to integrate the GC with the rest of the OCaml compiler.

As we mentioned in the tools background section, a code verified in Low* can be extracted to C code using the KaRaMeL compiler. We implement our GC in Low* and the verified code is extracted to C code. Figure 4.1 shows the top-level GC function extracted from the verified GC code on a 64-bit platform. No change has been made apart from renaming the functions for readability. Our heap `hp` is a single, contiguous, fixed-size byte buffer of size `heap_size`. The GC takes as inputs an array of roots `r` of length `r_len`, a mark stack array `st` with a stack top pointer `tp` that indexes into the stack,

a sweep pointer `sw` and a free list pointer `fp`. Note that `tp`, `sw` and `fp` are singleton buffers containing the stack top pointer, sweep pointer and free list pointer respectively. The free list is a singly linked list of free objects in the heap, which is implicitly stored through the fields of the objects. Recall that free objects are colored blue. Like OCaml, we assume that zero length objects are not on the heap. This implies that each object has at least one field, which we use to store the `next` pointer for the free list. Initially, `st` is empty, `tp` points to the stack base address, `sw` points to the start of the heap (i.e. `hp`), while `fp` points to the first blue object. The GC first calls `darken_roots` to grey all the roots in `r` and pushes them onto the mark stack `st` and suitably updates the stack top pointer `tp`.

Next is the `mark` function. We start with a base implementation first (Figure 4.2), where there is no distinction between different types of objects. The implementation here is to enable us to establish the base invariants necessary to do the verification. Then we extend the base implementation to handle objects with `No_scan_tag` (Figure 4.3) and finally `closure` and `infix` objects (Figure 4.4).

Let us first discuss the base version of the `mark` function as shown in Figure 4.2. The `mark` function repeatedly calls `mark_body` until the stack is empty. `mark_body` pops the object `x` from the top of the stack and finds its header address `h_x` using the function `hd_address`. Then the color bits of the value pointed by `h_x` are made `black` through `colorHeader`. `wosize` returns the object size in words stored at `h_x` and after which `darken` iterates through all the fields of `x`, calling `darken_body` on the fields. `darken_body` darkens the white objects (i.e. turning them grey) and pushes the field pointers onto the mark stack as necessary.

The code snippets in Figure 4.2 hints at the verification challenge in front of us. Given that we are in C, we have to ensure the accesses are memory safe, i.e., all memory accesses are to valid memory. Observe that implementation works by coloring the header words with bitwise operations. Hence, we need to reason about the correctness of bitwise

```
void mark(uint8_t *hp, uint64_t *st, uint64_t *tp) {
  while (*tp > (uint64_t)0U)
    mark_body(hp, st, tp);
}

void mark_body(uint8_t *hp, uint64_t *st, uint64_t *tp) {
  tp[0U] = *tp - (uint64_t)1U; // Decrement tp
  uint64_t x = st[*tp];
  uint64_t h_x = hd_address (x);
  colorHeader (hp, h_x, black);
  uint64_t wz = wosize (h_x, hp);
  darken (hp, st, tp, h_x, (uint64_t)1U);
}

void darken (uint8_t *hp, uint64_t *st,
             uint64_t *tp, uint64_t h_addr, uint64_t j) {
  uint64_t wz = wosize (h_addr, hp);
  for (uint32_t i = j; i < (wz + (uint64_t)1U)); i++) {
    darken_body(hp, st, tp, h_addr, i);
  }
}

void darken_body (uint8_t *hp, uint64_t *st, uint64_t *tp,
                  uint64_t h_addr, uint64_t i) {
  uint64_t succ_indx = h_addr + i * mword;
  uint64_t succ = load64 (hp + succ_indx);
  uint64_t c = color (hd_address (succ), hp);
  if (isPointer (succ_indx, hp)) {
    if (c == white) {
      push_to_stack(hp, st, tp, succ);
    }
  }
}
```

Figure 4.2: Base version of `mark` and `darken` function

arithmetic, also ensuring that the change in color bits does not affect `wosize` and `tag` of the object, which are also stored in the same header word.

The version of `mark` function as shown in Figure 4.3, incorporates the usage of the `tag` bits which are part of the OCaml object layout. Here, tag is used to determine whether the newly popped out object from mark stack needs to be scanned by the GC. Recall from Section 2.3.1 that any object with a tag greater than or equal to `no_scan` (value 251)

```
void mark_body(..omitted...) {
  // Code omitted, same as before...
  uint64_t tg = tag (h_x, hp);
  if (tg < (uint64_t)251U) {
    darken (hp, st, tp, h_x, (uint64_t)1U);
  }
}
```

Figure 4.3: Version of `mark` function that deals with `no_scan` objects

is not scanned by the GC. In this case, the `mark` skips scanning this object and moves on to next object from the mark stack.

The third version of the `mark` function, shown in Figure 4.4, further extends the marking process for objects with tag less than `no_scan`. In particular, it checks whether the object under consideration is a `closure` object or an `infix` object. `mark_body` calls `darken_wrapper` instead of `darken`, which decides the starting address of fields of the particular object under consideration. In the case of closure objects, as explained in Section 2.3.1.2, the offset of the environment needs to be extracted first. The details of the extraction are not shown for brevity. Another change is in `darken_body`, where, if a field points to an `infix_object`, then the *parent* `closure` object is determined and darkened instead.

One might ask whether darkening the parent closure, rather than the infix object itself, correctly identifies all reachable objects. The answer is yes, for the following reason. An infix object is always physically embedded within its parent closure and has no independent existence as an allocated object. This is enforced by the `infix_not_as_obj` well-formedness condition (Figure 4.28), which guarantees that infix objects never appear in `allocs(h)`. Consequently, infix objects are not vertices in the object graph `G(h)`. The graph construction in Section 4.7.3 reflects this: whenever a pointer field of object `x` refers to an infix object `y`, the graph edge is recorded as `(x, z)` where `z` is the parent closure of `y`, rather than `(x, y)`. Since `mark` and the object graph construction are defined to treat infix-

```
void mark_body (..omitted...) {
  // Omitted....
  if (tg < (uint64_t)251U) {
    // Wrapper function for darken
    darken_wrapper(hp, st, tp, h_x);
  }
}

void darken_wrapper (..omitted...) {
  // If the object is closure objs
  if (tag(h_x, hp) == (uint64_t)247U) {
    uint64_t x = f_address(h_x);
    // Start of environment has to be extracted for closure objects
    uint64_t start_env = start_env_clos_info (hp, x);
    darken (hp, st, tp, h_x, start_env + (uint64_t)1U);
  } else {
    darken (hp, st, tp, h_x, (uint64_t)1U);
  }
}

// Darken remains the same as that in base version
void darken_body(...omitted...) {
  // Omitted...
  if (isPointer(succ_indx, hp)) {
    uint64_t h_addr_succ = hd_address(succ);
    uint64_t tg = tag (h_addr_succ,hp);
    // If the field points to an infix object
    if (tg == (uint64_t)249U) {
      // Finds the parent closure
      uint64_t parent_hdr = parent (hp, h_addr, i);
      darken_helper (hp, st, tp, parent_hdr);
    } else {
      darken_helper (hp, st, tp, h_addr_succ);
    }
 }
}

void darken_helper(...omitted...) {
  if (color(hdr_id, hp) == white) {
    push_to_stack (hp, st, tp, hdr_id);
  }
}
```

Figure 4.4: Version of `mark` and `darken` function that deals with `closure` and `infix` objects

object pointers identically, both redirect to the parent closure, the correspondence between `mark`'s tracing behaviour and a standard DFS on `G(h)` is preserved. This correspondence is formally established by the `dfs_mark_equivalence_lemma` (Figure 4.19), which proves that `mark` colors black exactly those objects that `dfs` visits on the adapted graph. Therefore, the modified marking computes the same reachable set as a standard DFS on the object graph.

For simplifying the exposition of our verification process, we will use base version of `mark` throughout the rest of the paper. We note that our verified GC deals with `closure` and `infix` objects, and integrates with the rest of the OCaml compiler and the runtime. In Section 4.7, we expand upon the changes required to verify the implementation in Figure 4.4.

After the completion of the mark phase, the sweep phase scans the objects stored in the heap, starting from sw up to the end of the heap. The extracted code corresponding to a basic sweep variant is shown in Figure 4.5. During this scan, sweep examines the color of each object. If the object is colored black, it is recolored white. If the object is either white or blue, its color is changed to blue, and the object is added to the free list by updating the first field of `fp` to point to this object. Subsequently, the object currently pointed to by `sw` becomes the new value of `fp`.

It is important to note that, in this setting, the sweep phase can introduce heap fragmentation, as illustrated in Figure 4.6. If two unmarked (white) objects are adjacent in the heap prior to sweeping, then, in the absence of coalescing, these objects are reclaimed as two separate consecutive free blocks. This situation leads to external fragmentation. For example, consider two adjacent free blocks of sizes 2 words (a word corresponds to the machine word size) and 3 words, respectively, with no other free blocks available in the heap. In such a case, a memory allocation request of 4 words cannot be satisfied, despite the presence of a total of 5 free words, since no single

contiguous block is sufficiently large.

To mitigate this fragmentation issue, as shown in Figure 4.7, whenever a newly created blue block is found to be adjacent to the block currently pointed to by the free-list pointer, the two blocks are coalesced into a single larger blue block. Since the free list itself is constructed during the sweep phase, coalescing consecutive blue objects at sweep time effectively prevents heap fragmentation. The formal correctness specification for sweep with coalescing is given in Figure 4.29, and the invariants required to establish it are developed in Section 4.7.5.

4.4 OCAML GC SPECIFICATION

We now instantiate the abstract GC correctness definition from Section 4.2 for our GC compatible with OCaml. We express this specification in F* as is done in our artifact.

4.4.1 Basic Definitions

We first define the object graph in F* in Figure 4.8. The graph is defined as a record type in F* with two fields and is parametric over type `a`. Note that the prefix # before type `a` indicates that `a` is an *implicit* argument. The first field `vertices` has type `vertex_set a` and is a type alias of `seq a` with a type refinement that does not allow duplicates. `seq` is an unbounded array like data structure available in the F* standard library. The second field `edges` is defined to have type `edge_set a`, where `edge_set a` is a type alias for `seq (a,a)` with no duplicates. The type refinement on the `edges` which enforces that both the members of an edge should belong to the `vertices` of the graph.

In Figure 4.9, we define the accessibility relation (Definition 4.3) as an inductive predicate `reach`. Note that `vertex` is any type `a`, and `edge` is a type alias for `(vertex & vertex)` (`&` is product type operator in F*). The `reach g x y` predicate encodes a proof of reachability from vertex `x` to vertex `y` in graph `g`. There are two ways to construct this proof: either through `ReachRefl` which encodes that every vertex is reachable from itself, or through

```c
void sweep (uint8_t *g, uint64_t *sw, uint64_t *fp,
            uint64_t limit, uint64_t mword) {
  while (*sw < limit) {
    uint64_t curr_obj_ptr = *sw;
    uint64_t curr_header = hd_address(curr_obj_ptr);
    uint64_t wz = wosize_of_block(curr_header, g);
    uint64_t next_header =  curr_header + (wz + 1ULL) * mword;
    uint64_t next_obj_ptr = next_header + mword;
    sweep_body (g, sw, fp);
    sw[0U] = next_obj_ptr;
  }
}

void sweep_body (uint8_t *g, uint64_t *sw, uint64_t *fp) {
  uint64_t curr_obj_ptr = *sw;
  uint64_t curr_header = hd_address(curr_obj_ptr);
  uint64_t c = color_of_block(curr_header, g);
  uint64_t wz = wosize_of_block(curr_header, g);

  if (c == white || c == blue) {
    /* Unmarked object located */
    colorHeader(g, curr_header, blue);
    uint64_t fp_val = *fp;
    uint32_t x1 = fp_val;
    /* Adding freed blocks to the free-list */
    store64_le(g + x1, curr_obj_ptr);
    fp[0U] = curr_obj_ptr;
  } else {
    /* Marked object located */
    colorHeader(g, curr_header, white);
  }
}
```

Figure 4.5: Extracted C code of `sweep` function without coalescing

`ReachTrans`, which requires a proof of reachability from `x` to `z`, and an edge in `g` from `z` to `y`, captured by the type refinement `edge g z y`.

Our basic definitions in F*, which are related to the OCaml heap, are shown in Figure 4.10. We assume a 64-bit architecture. However, note that our framework is parametric over the machine word size. `mword` indicates the word size in bytes. We define `heap_size` to be an integer `n` such that `n` is a multiple of `mword`. The heap has enough space to store at least 1 object. Since the smallest object on the heap has one word header and one

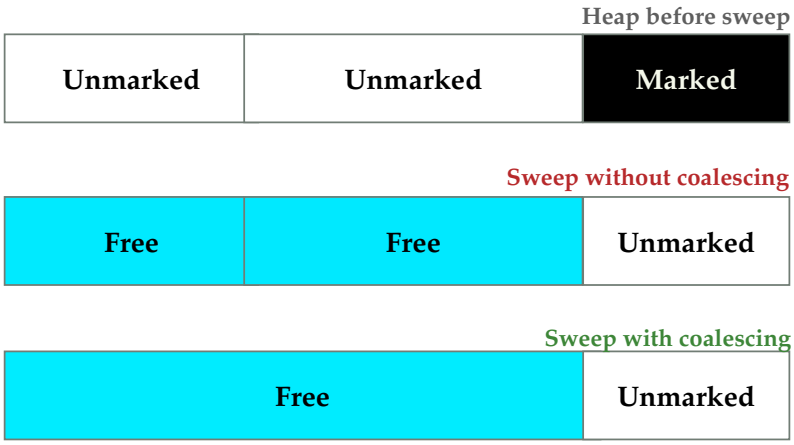


Figure 4.6: Heap state after the sweep phase, illustrating the effect of coalescing versus non-coalescing of free memory blocks

field, the smallest heap size is 16 bytes. We also need an upper bound on the heap size to prevent overflow when we perform arithmetic operations on the heap addresses. We choose the upper bound to be 1 TiB (2^{40} = 1099511627776 bytes), which is a pragmatic upper bound for OCaml programs.

We assume that the heap is densely packed with objects of any color. Recall from Section 2.3.1.2 that the OCaml object header includes two bits in the header for color. Blue color represents a free object, whereas white, black or grey object represents an allocated object. Objects can have arbitrary sizes, encoded in the `wosize` bits of its header block. For example, a completely empty heap (devoid of any allocated objects) may have one blue object that spans the entire heap or may have successive blue objects that span the entire heap.

The heap type is defined as a sequence of 8-bit unsigned machine integers of length `heap_size`. A valid heap address `hp_addr` is defined as a 64-bit unsigned machine integer. The heap address is word aligned and points to a location within the heap. In OCaml, every object is represented by the address of its first field. Therefore, `obj_addr` represents an object address which has an additional restriction that the valid heap address should start from `mword` or greater indicated by the type refinements inside the curly brackets. Similar refinement is applied to the type `hdr_addr` which denotes a header address of the

```c
void sweep (uint8_t *g, uint64_t *sw, uint64_t *fp,
            uint64_t limit, uint64_t mword) {
  while (*sw < limit) {
    uint64_t curr_obj_ptr = *sw;
    uint64_t curr_header = hd_address(curr_obj_ptr);
    uint64_t wz = wosize_of_block(curr_header, g);
    uint64_t next_header =  curr_header + (wz + 1ULL) * mword;
    uint64_t next_obj_ptr = next_header + mword;
    sweep_body (g, sw, fp);
    sw[0U] = next_obj_ptr;
  }
}

void sweep_body (uint8_t *g, uint64_t *sw, uint64_t *fp) {
  uint64_t curr_obj_ptr = *sw;
  uint64_t curr_header = hd_address(curr_obj_ptr);
  uint64_t c = color_of_block(curr_header, g);
  uint64_t wz = wosize_of_block(curr_header, g);

  if (c == white || c == blue) {
    /* Unmarked object located */
    colorHeader(g, curr_header, blue);
    uint64_t fp_val = *fp;
    uint64_t fp_header = hd_address(fp_val);
    uint64_t fp_wz = wosize_of_block(fp_header, g);
    uint64_t next_of_fp = fp_val + fp_wz * 8 + 8;

    if (next_of_fp == curr_obj_ptr) {
      /* Coalescing logic*/
      ((uint64_t *)fp_val)[-1] = makeHeader(fp_wz + wz + 1, blue, 0);
    } else {
       /* Adding freed blocks to the free-list */
      /* Omitted */
     /* Same logic as that of sweep without coalescing */
  } else {
    /* Marked object located */
    /* Omitted */
    /* Same logic as that of sweep without coalescing */
  }
}
```

Figure 4.7: Extracted C code of `sweep` function with coalescing

object. Since the objects on the heap have at least one field, the header address should be at least `mword` less than the heap size. The function `hd_address` takes the address of an object `o` and returns the header address of `o`.

```
noeq type graph (#a:eqtype) = {
  vertices : v: vertex_set #a;
  (* [vertices] are a sequence of type a with no duplicates *)
  edges : e: edge_set #a {edge_ends_are_vertices vertices e}
  (* [edges] are a sequence of type (a,a) with no duplicates *)
}
```

Figure 4.8: The graph type

```
type reach: (g:graph) → (x:vertex) → (y:vertex) → Type =
   (* reachability is reflexive *)
   | ReachRefl  : (g:graph) → (x:vertex) → (reach g x x)
   (* reachability is transitive *)
   | ReachTrans : (g:graph) → (x:vertex) → (z:vertex) →
               (reach g x z) →
               (* [edge g z y] is a type refinement which mandates
                  that [(z,y)] is an edge in [g] *)
               (y:vertex {edge g z y}) → (reach g x y)
```

Figure 4.9: Graph reachability as an inductive predicate `reach`

We now describe a few definitions, which are not shown in the code. `wosize_t`, `color_t` and `tag_t` define the type of `wosize`, `color` and `tag` of an object, respectively. The functions `wosize h_x h`, `color h_x h` and `tag h_x h` returns the wosize, color and tag respectively of the object `x` with header address `h_x` in heap `h`. The value stored at a heap address `x` in a heap `h` is read using a function `r_word h x`. Similarly, `w_word h x` writes to `h` in location specified by `x`.

Unlike the OCaml runtime, in the formalisation, for convenience, we address the fields from the header address of an object. Hence, the first field will have the offset 1. The function `valid_field_number` (shown in Figure 4.10), checks whether a field number is valid. A field number `i` is valid only if it lies within the range of `1` and the wosize of the object. The function `field` reads the ith field of object `x` in `h`, if `i` is a valid field number for the object `x`. We define a boolean predicate `isPointer i h = U64.logand (r_word h i) 1UL = 0`, which holds when the value at address i holds a pointer (the least-significant bit is 0). The predicate `well_formed_heap` is the instantiation of the abstract

```
//Machine integers
module U64 = FStar.UInt64
module U8 = FStar.UInt8
let mword = 8UL
val heap_size : n:int{n `mod` U64.v mword == 0 ∧ n >=16 ∧
                      n < 1099511627776}
(* heap is a sequence of 8 bit unsigned machine integers *)
type heap = h:seq U8.t{length h == heap_size}
(* A valid heap address *)
type hp_addr = addr:U64.t {U64.v addr < heap_size  ∧
                           is_multiple_of_mword addr}
(* object address *)
type obj_addr = x:hp_addr {U64.v x >= mword})
(* header address *)
type hdr_addr = x:hp_addr {U64.v x + mword < heap_size})

(* header address from object address *)
let hd_address (o:obj_addr) = U64.sub o mword

(* object address from header address *)
let f_address (h:hdr_addr) = U64.add h mword

let valid_field_number (i:U64.t) (h:heap) (x:obj_addr) =
    i >= 1 ∧ i <= wosize(hd_address x, h)

let field_addr (x:obj_addr) (h:heap)
               (i:U64.t{valid_field_number i h x}) =
    U64.add (hd_address x) (U64.mul i mword)

(* Field reads of ith field of object x *)
let field (x:obj_addr) (h:heap)
          (i:U64.t{valid_field_number i h x}) =
    r_word h (field_addr x h i)

(* allocs(h) is the set of allocated objects in the heap *)
let well_formed_heap h =
    (∀ x. seq.mem x (allocs (h)) ⟹
       (∀ (i:U64.t{valid_field_number i h x}).
         isPointer (field_addr x h i) h ⟹
           (field x h i) ∈ allocs(h)))
```

Figure 4.10: Basic Definitions in F⋆

well-formed heap (Definition 4.1) defined in Section 4.2. Note that the `well_formed_heap` predicate shown here does not incorporate the well-formedness requirements of OCaml features such as closure and infix objects, and this choice is deliberate for ensuring the

simplicity of presentation. The details on how to incorporate the additional features and their associated specifications are described in Section 4.7.

Using the above basic definitions, we now define some auxiliary functions. Given a heap `h`, `objs h` returns the sequence of object addresses in the heap `h`. It essentially scans the heap from the beginning, using the `wosize` of each object to move to the next object. `allocs h` returns the allocated object addresses in `h` (i.e. objects with a non-blue color). `h_objs h` is a sequence of the header addresses of all objects in `h`. Similarly, `h_allocs h` is the sequence of header addresses of allocated objects of `h`. Additionally we define `blacks h`, `whites h`, `greys h` and `blues h` to represent the sequence of header addresses of black, white, grey and blue objects respectively. The function `valid_hdr` takes a header address of an object and the heap and checks whether the header address is a part of `h_allocs h`.

4.4.2 Specification for GC functions

With these definitions in place, let us see how we can specify the correctness of the GC (Definition 4.5) based on the correctness of the constituent functions `darken_roots`, `mark` and `sweep` introduced in Figure 4.1. Certain useful algebraic properties of these functions as defined in F* are shown in Figure 4.11. The type `st_hp` is a pair of `seq obj_addr` (representing the mark stack) and `heap`. The F* library functions `fst` and `snd` returns the first and second member of a pair respectively. Note that the heap before and after the GC contains only white and blue objects.

The function `darken_roots` recursively fills the stack with all object addresses specified in the root list `r_list`. It maintains the invariant that all objects in the stack are colored grey (pre-condition `3` and post-condition `4`). `darken_roots` returns the modified stack and heap pair. In addition, the heap remains well-formed, and all fields of every object remains the same (post-conditions `1` and `3`).

The `mark` function also ensures the above properties, and additionally ensures that there

```
(* Product type *)
type st_hp = seq obj_addr & heap
type hp_fp = heap & obj_addr

val darken_roots (h:heap) (st:seq obj_addr) (r_list:seq obj_addr)
  : Pure (st_hp)
  (requires (* Only core conditions shown *)
  (*1*) well_formed_heap (h) ∧
  (*2*) (∀ x. x ∈ h_objs(h) ⟹ (x ∈ whites(h) ∨ (x ∈ blues(h)) ∧
  (*3*) (∀ x. x ∈ st ⟺ hd_address x ∈ greys(h)))
  (ensures  (* Only core conditions shown *)
  (*1*) (λ res → well_formed_heap (snd res) ∧
  (*2*) (∀ x. x ∈ r_list ⟹ x ∈ (fst res)) ∧
  (*3*) (∀ x i. (hd_address x) ∈ h_objs(h) ⟹
                      field x h i = field x (snd res) i)) ∧
  (*4*) (∀ x. x ∈ (fst res) ⟺ hd_address x ∈ greys(snd res)))

val mark (h:heap) (st:seq obj_addr)
  : Pure (heap)
  (requires (* Only core conditions shown *)
  (*1*) well_formed_heap (h) ∧
  (*2*) (∀ x.x ∈ st ⟺ hd_address x ∈ greys(h)))
  (ensures  (* Only core conditions shown *)
  (*1*) (λ h₁ → well_formed_heap (h₁) ∧
  (*2*) (∀ x i. (hd_address x) ∈ h_objs(h) ⟹
                      field x h i = field x h₁ i) ∧
  (*3*) (∀ x.x ∈ h_objs(h₁) ⟹ (color (hd_address x h)₁ ≠ grey)))

val sweep (h:heap) (curr_ptr:obj_addr) (fp:obj_addr)
  : Pure (hp_fp)
  (requires (* Only core conditions shown *)
  (*1*) well_formed_heap (h) ∧
  (*2*) (∀ x.x ∈ objs(h) ⟹ (color (hd_address x h) ≠ grey)) ∧
  (ensures  (* Only core conditions shown *)
  (*1*) (λ h₁, fp₁ → well_formed_heap (h₁) ∧
  (*2*) (∀ x. x ∈ blacks(h) ⟺ x ∈ whites(h₁)) ∧
  (*3*) (∀ x. x ∈ whites(h) ∨ blues(h) ⟺ x ∈ blues(h₁)) ∧
  (*4*) (∀ x i. (hd_address x) ∈ h_allocs(h) ⟹
                      field x h i = field x h₁ i) ∧
  (*5*) (∀ x. x ∈ h_objs(h₁) ⟹ x ∈ whites(h₁) ∨ x ∈ blues(h₁)))
```

Figure 4.11: Algebraic properties of the constituent functions of our verified GC

are no grey objects after it finishes, essentially coloring all reachable objects black. The absence of grey objects implies that all reachable objects have been fully traversed and processed, and that no pointers from reachable objects refer to unmarked (white) objects.

Consequently, this property rules out the presence of dangling pointers from live objects to reclaimed memory, ensuring pointer safety at the end of the marking phase.

Again for simplicity of presentation, we have used the `sweep` version in Fig 4.5 (that is, sweep without coalescing), for the correctness specifications. The details of incorporating the specification deltas that arise due to coalescing are described in Section 4.7. The key idea is that the revised specification introduces localized deltas corresponding to the effects of coalescing, while preserving the original specifications for the unaffected portions of the code. This approach enables effective reuse of existing specifications without requiring a complete re-specification. The type of the return value of `sweep` is `hp_fp`, which is a pair of `heap` type and free list pointer type (`obj_addr` type). The `sweep` function ensures that there are no more black objects after it completes. Note that all these functions ensure that the heap remains well-formed, and there is no change to the object fields, effectively ensuring properties (1) and (3) in the definition of GC correctness (Definition 4.5). For proving property (2), we need to consider the reachability of objects in the underlying object graph.

4.4.3 Object graph construction

The construction of the object graph from the heap crucially depends on the well-formedness of the heap (`well_formed_heap` defined in Figure 4.10). Well-formed heap requires that pointers from allocated objects should only refer to other allocated objects. The allocator and the mutator (the OCaml program) must maintain the property of `well_formed_heap` at all times, ensuring that every object created is well-formed and that the mutator does not create dangling pointers or disrupt heap structure during normal operation or stop-the-world pauses. Additionally, it is crucial that the root set `r` used by the garbage collector (GC) correctly identifies all live roots accessible by the mutator during collection; omitting any live roots could lead to mistakenly collecting reachable objects. Lastly, we rely on the correctness of the KaRaMeL compiler to accurately convert the verified Low* implementation to C while preserving the established semantics, as

its correctness has been verified in earlier work. All other aspects, such as reachability preservation and memory safety of the GC, are verified using F*/Low*.

```
module G = Spec.Graph
val edges_of_graph (s:seq obj_addr) (h:heap{well_formed_heap (h)})
     : Tot (e:G.edge_set{∀ x y. mem x s  ⟹   mem (x,y) e  ⟺
                              (∃ i. valid_field_number i h x ∧
                                    isPointer (field_addr x h i) h ∧
                                    y = field x h i)})
val graph_from_heap (h:heap)
     : Pure (G.graph)
       (requires  well_formed_heap (h))
       (ensures  λ g→ g.vertices = allocs h ∧
                      g.edges = edges_of_graph allocs h)
```

Figure 4.12: Constructing graph from the heap

OCaml features such as closure and infix objects also affect graph construction, as these objects have a different layouts to regular objects and influence how the GC scans the objects. As mentioned earlier, for simplifying the presentation, the graph construction of Figure 4.12 ignores closure and infix objects and objects which only have opaque bytes. The details on how to incorporate the additional features is described in Section 4.7. As shown in Figure 4.12, the vertices of the graph are simply the set of allocated objects `allocs h`, while the edges are constructed using a function `edges_of_graph`, that takes as inputs `allocs h` and `h` and creates pairs `(x,y)` for all `x` in `allocs h` such that `y` is a field pointer of `x` in `h`.

Using the graph definitions, we can now specify the additional properties required for proving property (2) in the abstract GC correctness definition. As shown in Figure 4.13, the `mark_reachability_lemma` ensures that the object graph constructed from the heap remains the same before and after `mark`. In addition, all the objects and only the objects that are reachable from the root list `r_list` in the object graph will be colored black in the output heap after `mark`. Note the use of the inductive `reach` predicate from Figure 4.9 in this specification. Notice also that the precondition (2) requires that all objects in the stack

```
val mark_reachability_lemma (h:heap) (st:seq obj_addr)
                            (r_list:seq obj_addr)
  : Lemma
    (requires
    (*1*) well_formed_heap (h)
    (*2*) (∀ x. x ∈ st ⟺ hd_address x ∈ greys(h)) ∧
    (*3*) well_formed_heap (mark h st))

    (ensures
    (*1*) (graph_from_heap (mark h st) = graph_from_heap h) ∧
    (*2*) (∀ x y.y ∈ r_list ∧
              reach (graph_from_heap h) y x ⟺ x ∈ blacks(mark h st)))


val sweep_subgraph_lemma (h:heap) (r_list:seq obj_addr)
                         (curr_ptr:obj_addr) (fp:obj_addr)
  : Lemma
    (requires
    (*1*) well_formed_heap (h) ∧
    (*2*) (∀ x.x ∈ h_objs(h) ⟹ (color (hd_address x h) ≠ grey)) ∧
    (*3*) well_formed_heap (sweep h curr_ptr fp))

    (ensures
    (*1*) (∀ x. x ∈ graph_from_heap (sweep h curr_ptr fp).vertices ⟺
                x ∈ graph_from_heap (h).vertices ∧
                (∃ y. y ∈ r_list ∧ reach (graph_from_heap h) y x)) ∧

    (*2*) (∀ x y.
             (x,y) ∈ graph_from_heap (sweep h curr_ptr fp).edges ⟺
                x ∈ graph_from_heap (sweep h curr_ptr fp).vertices ∧
                y ∈ graph_from_heap (sweep h curr_ptr fp).vertices ∧
                (x,y) ∈ graph_from_heap (h).edges))
```

Figure 4.13: Mark reachability and reachable subgraph preserving lemma

are colored grey in the input heap, which is essentially a post-condition of `darken_roots` in Figure 4.11. The `sweep` correctness conditions are listed in `sweep_subgraph_lemma`. The post-conditions of the lemma ensure that the graph formed after `sweep` has only reachable objects and their interconnections, that is the reachable sub-graph of the original graph before the GC. The fact that `sweep` is the last operation of the GC ensures that the final graph after the GC is the reachable subgraph of the graph before the GC, thus proving property (2) of Definition 4.5.

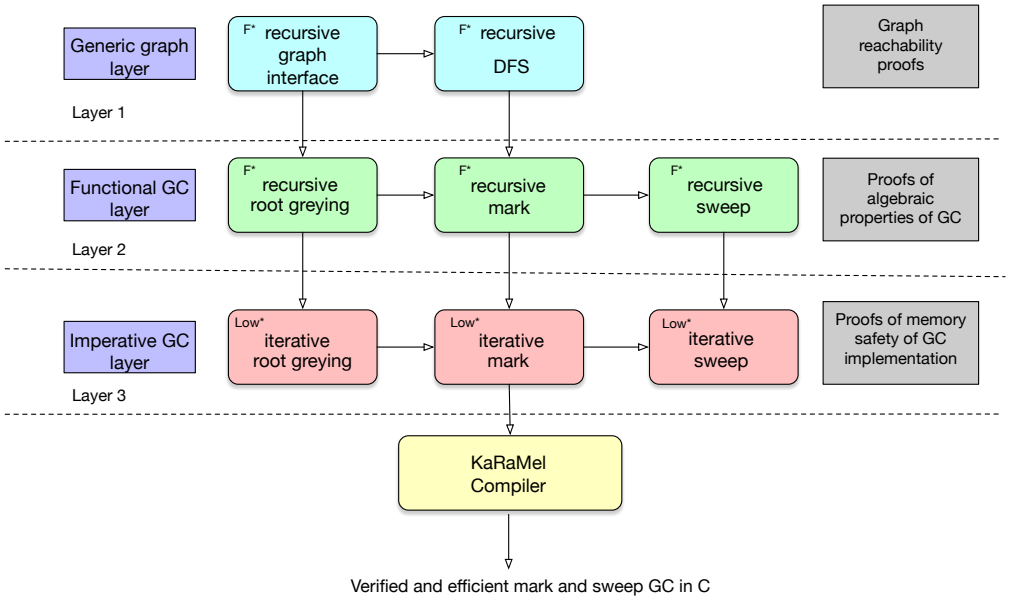


Figure 4.14: The layering approach for GC verification

As evident from the specifications, there are three different dimensions of reasoning required for verifying correctness: (i) first, we must relate the coloring logic with reachability in the object graph, (ii) next, we need to ensure the algebraic properties related to well-formedness and preservation of the object graph for the bit-wise manipulations performed during the GC, (iii) and finally, while the above specifications talk about a functional heap, the C implementation performs in-place mutations, and hence we need to reason about aliasing and memory safety. This naturally points to the need for a layered strategy for verification, which is the focus of the next section.

4.5 A LAYERED VERIFICATION FRAMEWORK AND CORRECTNESS PROOFS FOR A BARE MINIMAL OCAML GC

With the implementation and the specifications in place, we now deep dive into the details of our verification framework and how we have carried out the main correctness proofs. As mentioned earlier, the challenge here is that we need to reason about complex graph theoretic specifications for an optimised and efficient implementation, while ensuring memory-safe behaviour of the GC implementation itself. We have designed our layered verification methodology to cleanly separate various proof obligations involving the

graph reachability based specification, the correctness of bitwise arithmetic and the correctness of concrete memory changes carried out by the GC. As shown in Figure 4.14, the first layer deals with the verification of reachability properties of DFS, the second layer is for proving algebraic properties related to the bitwise arithmetic operations as well as to prove the abstract graph related properties of the GC, and the third layer proves that the GC does not violate memory safety. The third layer also acts as the layer from which the actual C code of the GC is extracted.

4.5.1 Layer 1 – A verified depth-first search implementation in F⋆

We know that `mark` performs a depth-first traversal of the OCaml heap, but it takes advantage of the OCaml object layout to efficiently perform operations such as finding successors, maintaining the visited vertices, etc. Directly proving the `mark_reachability_lemma` would be hard, especially for F⋆, as inductively defined properties such as `reach` do not work well with SMT-based verifiers. Mixing this reasoning with the OCaml object layout and the bit-wise arithmetic operations occurring in `mark` would make the problem even harder. To simplify this proof, we instead focus on proving the reachability properties for a bare bones `dfs` implementation. In the second layer, we establish the functional equivalence between `mark` and `dfs` by proving that `mark` colors all and only those objects reached by `dfs`.

The `dfs` implementation, shown in Figure 4.15, directly takes as input the object graph (whose type is defined in Section 4.4). It explicitly maintains a *visited list*, corresponding to the set of vertices which have been fully explored. Several helper functions appear in this figure and throughout the chapter, including `stack_top`, `stack_rest`, `push_unvisited`, `successors`, `set_insert`, `field_slice`, `successor_slice`, and `field_start_for_object`, whose definitions are elided here for brevity. These are helper functions defined as part of this development whose names are self-explanatory and their full definitions are available in the accompanying artifact. We design the `dfs` implementation to closely resemble the `mark` implementation shown in Figure 4.16. Note that this `mark`

```
(* dfs calls dfs_body until stack empty.
   Inputs are graph, stack and visited set. *)
let rec dfs (g:graph) (st:seq U64.t) (vl:seq U64.t)
 : Pure (seq U64.t)
  (requires ...)
  (ensures (λ res → ...)
  (decreases (length g.vertices - length vl; length st)) =
  if length st = 0 then vl
  else
    let st₁ ,vl₁ = dfs_body g st vl in
    dfs g st₁ vl₁

let dfs_body g st vl
 : Pure ...
  (requires ...)
  (ensures (λ res → ...) =
     let x   = stack_top st in
     let xs  = stack_rest st in
     let s   = successors g x in
     let vl₁ = set_insert x vl in
     let st₁ = push_unvisited s xs vl₁ in
     (st₁, vl₁)
```

Figure 4.15: Functional `dfs`

implementation does not account for no-scan, closure, or infix objects. However, as shown in Section 4.7, the `dfs` implementation and its associated proofs need not be modified when incorporating the specification deltas required to support `mark` operating over a heap containing heterogeneous object types. The `dfs` implementation is functional and recursive: in each recursive call, it removes the vertex at the top of the stack, pushes it into the visited list, and then pushes all unvisited successors onto the stack. At the end, `dfs` returns the set of all vertices reachable from the root set.

The correctness specification of `dfs` is defined using the `reach` predicate in Figure 4.17. Let us first focus on the post-condition, which is the required guarantee that we need in terms of reachability. The forward direction says that every vertex present in the return value of `dfs` must be reachable from some vertex in the root set. The backward direction says that if a vertex is reachable from the root set, then it must be present in the return value of `dfs`.

```
(* mark calls mark body until stack empty.
   Inputs are heap and stack *)
let rec mark (h:heap) (st:seq obj_addr)
 : Pure (heap)
  (requires ...)
  (ensures (λ res → ...)
  (decreases (length allocs h - length blacks h;
              length st)) =
  if length st = 0 then h
  else
    let st₁, h₁ = mark_body h st in
    mark h₁ st₁

let mark_body (h:heap) (st:seq obj_addr)
  : Pure ...
    (requires ...)
    (ensures (λ res → ...) =
  let x   = stack_top st in
  let xs  = stack_rest st in
  let h₁  = colorHeader h x black in
  let st₁ = darken h₁ xs x 1UL in
  (st₁, h₁)
```

Figure 4.16: mark implementation

```
(* r_list is the root set, stack is filled with r_list initially *)
val dfs_reachability_lemma (g:graph) (st:seq obj_addr)
                           (vl:seq obj_addr) (r_list:seq obj_addr)
  : Lemma
   (requires
     (* Pre-conditions required to prove forward direction *)
     (*F1*) mutually_exclusive_sets st vl ∧
     (*F2*) (∀ y.y ∈ st ⟹ (∃ x.x ∈ r_list ∧ reach g x y) ∧
     (*F3*) (∀ y.y ∈ vl  ⟹ (∃ x.x ∈ r_list ∧ reach g x y) ∧

     (* Pre-conditions required to show the backward direction *)
     (*B1*) (∀ x y.x ∈ r_list ∧ reach g x y  ⟹
            (∃ z.z ∈ st ∧ reach g z y) ∨ y ∈ vl)
     (*B2*) (∀ x y.x ∈ vl ∧ reach g x y  ⟹
            (∃ z.z ∈ st ∧ reach g z y) ∨ y ∈ vl)

   (ensures (∀ y.y ∈ (dfs g st vl)  ⟺ (∃ x.x ∈ r_list ∧ reach g x y))
```

Figure 4.17: Correctness specification of dfs

To prove the forward direction, we assert the pre-condition (F2,F3) that all vertices in both the stack and the visited list are always reachable from some vertex in the root set. This holds for the initial call to dfs, as the stack contains the vertices of r_list (which are reachable from themselves through ReachRefl), and the visited list is empty. The dfs_body function shifts the vertex x at the top of the stack to the visited list and adds its unvisited successors to the stack. From the inductive hypothesis, x must be reachable from some vertex in r_list, and through ReachTrans, we can construct a path from that vertex to every successor of x. Thus, the contents of the updated stack and visited list continue to satisfy (F2,F3).

For the backward direction, our pre-condition B1 asserts that every vertex reachable from the root set should either already be part of the visited list, or it should be reachable from some vertex in the stack. However, this property by itself is not inductive, and hence we also require that every vertex reachable from the visited list should either already be part of the visited list, or be reachable from some vertex in the stack (B2). We show that these pre-conditions are ensured by the initial call to dfs, and they are also maintained for every recursive call. The interesting case is when a vertex y is reachable from r_list through some vertex x that has just been popped from the stack, and y is not yet in the visited list. By ReachTrans, y must be reachable from some successor of x. If that successor is added to the stack, (B1,B2) continue to hold. If the successor is already in the visited list and not re-added, then (B2) guarantees the existence of another vertex on the stack from which y is reachable. One further complication arises when this vertex is x itself, meaning there is a path from x to y that contains x as an intermediate vertex. In this case, we use the fact that a shorter path not containing x as an intermediate vertex must always exist. This is established by considering the last occurrence of x on the original path and extracting the sub-path from that point. This type of path-shortening reasoning is where our inductive definition of reach is essential, as it makes such arguments tractable for F*'s SMT backend.

Since `dfs` is a total function, F* also requires a termination proof. The graph remains the same across recursive calls and the visited list never shrinks, while the stack may grow or shrink. Termination is witnessed by a lexicographic `decreases` clause: either the difference between the number of graph vertices and the size of the visited list decreases, or that difference stays the same and the stack size decreases.

4.5.2 Layer 2 – Functional mark-and-sweep in F*

We now consider proving the correctness of the `mark` and `sweep` implementations. Towards this end, we first consider their functional implementations in F*, which operate on the OCaml heap representation. Here, the input `h` will be a sequence of memory words (using the `seq` type in F*), that contains OCaml objects following the format of Figure 2.2. Since the `mark` and `sweep` implementations use various *colors* to indicate different phases of an object, these functions depend heavily on a correct implementation of the `colorHeader` function, which sets the color of an object. The next section focuses on the specifications of `colorHeader`, which is crucial to ensure the correctness of the GC.

4.5.2.1 Specification of `colorHeader`

The specifications of `colorHeader` are shown in Figure 4.18. `colorHeader` takes as input the heap `h`, an address `hdr` into `h` (which will be the address of the object header) and a color `c` that has to be updated in the color bits of the value stored at `hdr`. The function returns the modified heap. `makeHeader` is a bit manipulation function that is used to create a header value. The operations `U64.shift_right` and `U64.logand` are 64-bit right shift and logical AND operations available as part of the F* standard library. The specifications ensure that, only the color bits changes when `colorHeader` is applied. Especially the predicate `heap_same_except` ensures that except `hdr` in `h`, everything else remains the same in the resultant heap `res`. Since `hdr` is a `valid_hdr`, this ensures that, all the fields of all objects remains the same. Using the specifications of `colorHeader`, we show the proof outlines for proving the algebraic properties of the GC functions as shown in Figure 4.11.

```
val makeHeader (wz:wosize_t) (c:color_t) (t:tag_t)
     : Pure (U64.t)
       (requires True)
       (ensures (λ res →
                (U64.shift_right res 10UL = wz) ∧
                (U64.logand (U64.shift_right res 8) 3UL = c) ∧
                (U64.logand res 255UL = t)))

 val colorHeader (h:heap) (hdr:hdr_addr) (c:color_t)
     : Pure (heap)
       (requires (* Only core conditions shown *)
                  well_formed_heap (h) ∧
                  valid_hdr hdr h)
       (ensures λ res →
                well_formed_heap (res) ∧
                valid_hdr hdr res ∧
                objs h = objs res ∧
                heap_same_except hdr h res ∧
                color hdr res  = c ∧
                wosize hdr res = wosize hdr h ∧
                tag hdr res = tag hdr h ∧
                r_word res hdr =
                 makeHeader (wosize hdr h) (c) (tag hdr h))
```

Figure 4.18: Specifications of `colorHeader`

4.5.2.2 Proof outline for the sub-functions of the GC

The specifications are shown in Figure 4.11. The `darken_roots` function pushes all the root pointers in `h_list` to an empty stack and then colors them as grey. Let h_0 be the initial heap before the GC and let h_1 be the heap resulted after `darken_roots`. Since the GC starts with `well_formed_heap` h_0 that contains only white and blue objects and the fact that `h_list` contents are members of `allocs`(h_0) implies that `well_formed_heap` h_1 is preserved as the only change to heap is coloring of `h_list` members from white to grey. Since both white and grey are considered as allocated objects, changing the color from white to grey still preserves the membership in allocated set. Therefore, `allocs`(h_1) = `allocs`(h_0). Therefore the vertex set of both graphs constructed from h_0 and h_1 remains the same. Since the specification of `colorHeader` ensures that except the color bits of the address `hdr` in the heap, nothing else in the heap changes, the edge set of the graphs from h_0 and h_1 also remains the same. Therefore, both the graphs are the same. Since

`darken_roots` starts with an empty stack, when the objects in `h_list` are pushed into the stack and colored them grey in the heap, the only grey objects in the heap are the ones that are pushed onto the stack.

Let h_2 be the heap formed after `mark`. All the algebraic properties of mark as in Figure 4.11 can be proved by using the same approach as `darken_roots` and the fact that `mark` starts with a stack that contains all the grey and only the grey objects of the heap. When `mark` terminates after the stack becomes empty, there are no grey objects in the resultant heap after `mark`.

Similarly the algebraic properties of `sweep` can also be directly proven using the specifications of `colorHeader`. Recall that, `sweep` changes the color of `white` objects to `blue` and `black` objects to `white`. As well as the first field of the free list pointer is changed to point to the newly created blue block during a sweep invocation. Since the well-formedness property only affects the allocated objects and the fact that free list pointer points to a blue object, with the help of some extra lemmas related to the `sweep` properties, F* can prove that the heap resulted after `sweep` remains as well-formed.

4.5.2.3 Proof outline for `mark_reachability_lemma`

Now comes the crucial question: how can we demonstrate that `mark` indeed identifies all the reachable objects discovered by the depth-first search (DFS) and only those objects? A direct proof of `mark`'s reachability would require us to reason about bit-wise arithmetic in conjunction with graph reachability, which adds significant complexity and is challenging to handle with a satisfiability modulo theories (SMT) solver. To mitigate these complexities, we have opted for a modular reasoning approach. Rather than tackling the direct proof, we leverage the reachability proofs from `dfs` for `mark` by establishing behavioural equivalence between `dfs` and `mark` using the lemma presented in Figure 4.19.

Proof outline for `dfs_mark_equivalence_lemma`: Both `dfs` (in Figure 4.15) and `mark`

(in Figure 4.16) are tail-recursive functions, whose outputs are exactly same as the outputs of the recursive calls at the end. Hence, we use the post-condition of `dfs_mark_equivalence_lemma` as a form of inductive invariant, following the classical modular verification technique for recursive functions. We also require two invariants to be obeyed relating the input arguments to `dfs` and `mark`. The `stack invariant` says that the grey objects of the input heap `h` can only be found in the stack `st` and the `visited-list invariant` ensures that the black objects of the heap are only present in the visited list `vl`. This way membership checks in `st` and `vl` can be avoided by replacing it with two color bits check, which is more efficient than the membership checks in `dfs`. We show that if the `mark` and `dfs` methods are called with the same input stack `st` which satisfies the `stack invariant`, and a visited set that satisfies `visited-set` invariant as well as a heap that satisfies `well_formed_heap (h)`, then their outputs should be equivalent, i.e., all objects in the output of `dfs` should be colored *black* in the output heap of `mark`.

```
val dfs_mark_equivalence_lemma (h:heap) (st:seq obj_addr)
                               (vl:seq obj_addr) (h_list:seq obj_addr)
    : Lemma
      (* Only important properties shown *)
      (requires (*1*) mutually_exclusive_sets st vl ∧
                (*2*) well_formed_heap(h) ∧
                (* stack invariant *)
                (*3*) (∀ x.x ∈ st  ⟺  (hd_address x) ∈ greys(h))
                (* visited-list invariant *)
                (*4*) (∀ x.x ∈ vl  ⟺  (hd_address x) ∈ blacks(h))

      (ensures (∀ x. x ∈ (dfs (graph_from_heap h) st vl) ⟺
                     (hd_address x) ∈ blacks(mark h st)))
```

Figure 4.19: Behavioural equivalence between `mark` and `dfs`

Due to the tail-recursive nature of both `dfs` and `mark`, we can use the equivalence of outputs of the recursive calls to infer the required result. Suppose h_1 and st_1 are the results of one invocation of `mark_body` and st_2 and vl_2 are the outputs obtained after

dfs_body. These outputs are passed as inputs to the recursive calls of mark and dfs. Then, we need to ensure that the input arguments to the recursive calls satisfy the pre-conditions mentioned below:

- well_formed_heap h_1
- (∀ x. x ∈ vl_2 ⟺ (hd_address x) ∈ blacks(h_1))
- (∀ x. x ∈ st_1 ⟺ (hd_address x) ∈ greys(h_1))
- st_2 = st_1

Since well_formed_heap h holds for the input heap h, the first property follows as the color changes to the heap during one invocation of mark_body only involves darkening. That is, white objects becomes grey and grey objects become black. That means, the allocs h and allocs h_1 remains the same. Combining this property with the fact that the fields remain unchanged due to the coloring operation, the well-formedness of h_1 can be proved. This also ensures that the graphs formed from both h and h_1 remains the same. A careful inspection of dfs_body and mark_body reveals that, each of the functions removes the top of the stack and mark_body colors it as black whereas dfs_body adds it to the visited list. Since the input stacks are the same, this operation ensures that the visited-list invariant is maintained with vl_2 and h_1. While pushing the field pointers of the top of the stack, mark_body colors them grey, which ensures the stack invariant of st_1 with respect to h_1.

For proving the last property, which is the stack equivalence between the stacks produced by dfs_body and mark_body, let us understand the behaviour of the two tail recursive functions push_unvisited and darken, used in dfs_body and mark_body to obtain st_2 and st_1 respectively. A comparison between these functions are shown in Figure 4.20.

The push_unvisited function scans through the list of successors of x (which was at the top of the stack) in the object graph, while darken scans the fields of the object x in the heap. Both functions starts with the same stack st and populate the stack with unvisited

```
(* push_unvisited calls push_unvisited_body until successors is empty *)
let push_unvisited_body s st vl j
  : Pure ...
    (requires j < length s...)
    (ensures (λ res → ...) =
  if (not (mem (index s j) (set_union st vl))) then
    let st₂ = push (head s) st in
    st'

(* darken calls darken_body until i = wosize + 1 *)
let darken_body h st h_addr i
   : Pure ...
    (requires ...)
    (ensures (λ res → ...) =
  let succ = r_word h (h_addr + i * mword) in
  if not (isPointer succ) then (st, h)
  else
   let c = color h (hd_address succ) in
   if not (c = white) then (st, h)
   else
     let h₁, st₁ = push_to_stack h st succ in
     (st₁, h₁)
```

Figure 4.20: A comparison of push_unvisited_body and darken_body functions

(white) field pointers of x into the stack. push_unvisited and darken perform the bulk of the work by calling push_unvisited_body and darken_body respectively.

The parameter j in push_unvisited_body indicates the index of the successor in s to be examined. Similarly, the parameter i in darken_body indicates the field number of x to be scanned in h. Recall that st and vl are mutually exclusive. The function push_unvisited_body decides whether an element is unvisited by checking the membership in st and in vl. Due to the invariants on st and vl, such an object will also be colored white in the heap. The same action is being performed by the darken_body as well. But the difficulty here is that s is already a filtered list of field pointers (i.e. successors), while the field scan in fields by darken may encounter non-pointer fields as well. Hence, there may not exist a direct one-to-one correspondence between the invocations of push_unvisited_body and darken_body. To get around this issue, we can

make use of the observation that if the `field_slice` which starts at the ith field of `x` in the heap returns the same set of field pointers as that of `successor_slice` that starts at j^{th} index of the successors list in `s`, then the stacks produced by `darken` that starts at field index `i` of `x` in `h` and `push_unvisited` that starts at index `j` of `s` are the same. Formally in F* we prove the lemma as shown in Figure 4.21.

```
val darken_push_unvisited_produces_same_stack (h:heap) (st:seq obj_addr)
      (vl:seq obj_addr) (curr:hdr_addr) (i:U64.t) (j:U64.t)
  : Lemma
   (requires mutually_exclusive_sets st vl ∧
             well_formed_heap (h) ∧
             successor_slice s j = field_slice h hdr_addr i)
   (ensures  darken h st hdr_addr i = push_unvisited s st vl j)
```

Figure 4.21: A comparison of `push_unvisited_body` and `darken_body` functions

The lemma is invoked with `i = 1` and `j = 0` to prove the stack equivalence (i.e., `st' =` $\mathtt{st}_1$). Thus, through the maintenance of pre-conditions, the induction hypothesis through the recursive invocation of the `dfs_mark_equivalence_lemma`, we are able to complete the proof of `dfs_mark_equivalence_lemma`. This way, `dfs_reachability_lemma` and `dfs_mark_equivalence_lemma` is sufficient to complete the proof of `mark_reachability_lemma` in Figure 4.13.

The overall proof structure is illustrated in Figure 4.22. In summary, we consider the execution of `mark` on a heap $\mathtt{h}_1$ with stack `st`, and the execution of `dfs` on the object graph `G(`$\mathtt{h}_1$`)` constructed from $\mathtt{h}_1$, starting with the same stack `st` and an empty visited set `vl`. The visited set is initially empty because there are no black objects in $\mathtt{h}_1$. By maintaining the stack invariant and the visited-set invariant, we prove both the `dfs_reachability_lemma` and the `dfs_mark_equivalence_lemma` on the heap $\mathtt{h}_2$ produced by `mark`. Finally, these two lemmas together establish the `mark_reachability_lemma`.

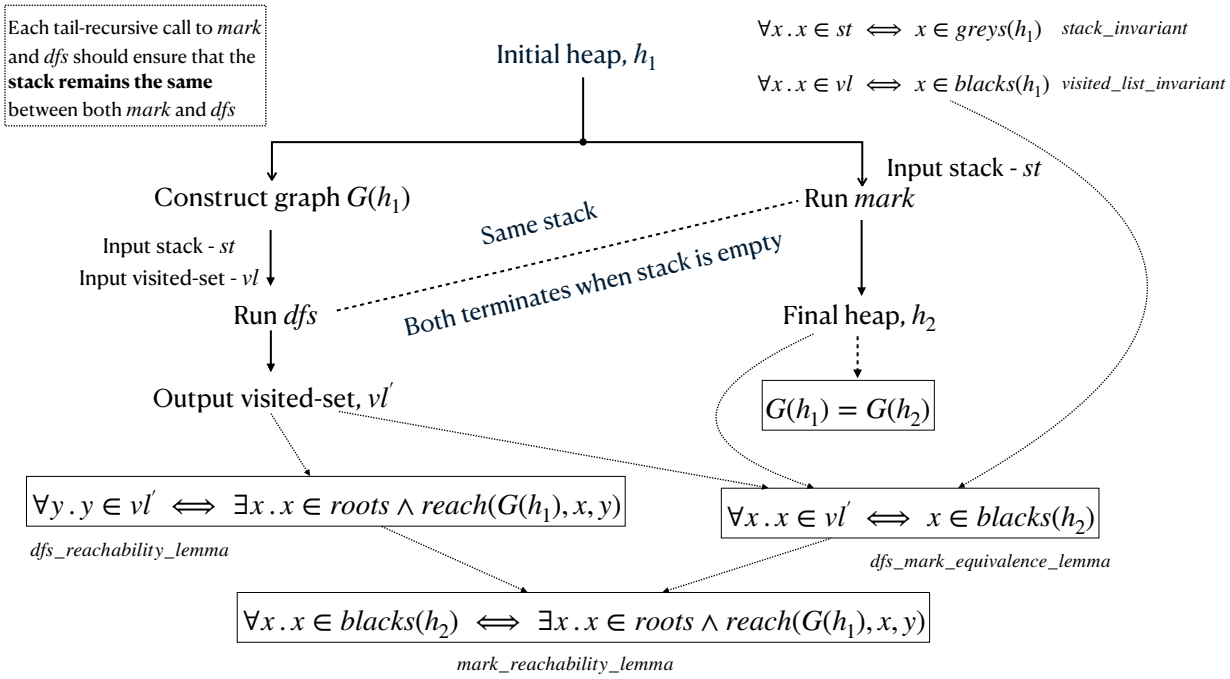


Figure 4.22: The proof flow of `mark_reachability_lemma`

4.5.2.4 Termination of `mark`

At first glance, the termination of `mark` may seem self-evident, as the algorithm repeatedly processes objects until the stack becomes empty. However, from a verification perspective in F*, this intuition is insufficient: one must explicitly justify why the stack will eventually become empty.

A natural termination candidate is the stack length. However, this measure alone is not adequate, since the stack may grow when new grey objects are discovered and pushed. Instead, we rely on the observation that with every recursive invocation of `mark`, at least one object transitions to the black state and is never reverted. Therefore, the difference between the total number of objects in the heap and the number of black objects strictly decreases with each recursive step. We combine this with the stack length to form a lexicographic `decreases` clause: either the number of unprocessed (non-black) objects decreases, or it stays the same and the stack shrinks. Together, these two measures are sufficient to convince F* that `mark` always terminates.

4.5.2.5 Proof outline for `sweep_subgraph_lemma`

Let h_0 be the heap state before the GC, let `h` be the heap after `mark` that satisfies `dfs_mark_equivalence_lemma` and let h_1 be the heap after `sweep`, and thus the heap after the GC. Therefore, from the algebraic properties of `sweep` in Figure 4.11, if `h_list` is the root set, `curr_ptr` is the sweep head and `fp` is the free list pointer then the following property holds:

```
(∀ x y.y ∈ h_list ∧ reach (graph_from_heap h) y x ⟺
                (hd_address x) ∈ whites (fst (sweep h curr_ptr fp)))
 where h₁ = fst (sweep h curr_ptr fp))
```

This means all the white objects that results after the `sweep` (white objects in h_1) are the only reachable objects in `h` (heap before `sweep` and after `mark`). Since the object graph remains unchanged by both `darken_roots` and `mark`, and since `sweep` is the final phase of the garbage collector, we conclude that the objects that remain white after the completion of the GC are exactly those objects that were unreachable before the GC. Again from the algebraic properties of `sweep`, the only allocated objects after the `sweep` are white objects. We have already shown that coloring operation and modification of the first field of free list does not alter the contents of any allocated object fields. These properties ensure that the objects in the graph formed from h_1 contains all the reachable and only the reachable objects in h_0 and the edges between them remains the same in h_1 as that in h_0. This completes the proof of `sweep_subgraph_lemma`. Thus we can see that all the abstract GC correctness properties as mentioned in Definition 4.5 is fulfilled by our functional GC. Now, all it remains is to show that an imperative implementation of such a GC does not violate any of the functional correctness properties of the GC. For that, in the next section, we show how to implement the imperative GC in the third and the final layer, and how to prove its program equivalence with that of the functional GC.

4.5.3 Layer 3 – Imperative mark-and-sweep in Low*

As discussed earlier, the verification focus of this layer is to prove that the imperative GC implementation itself does not cause any memory safety bugs. There are a number of challenges to be dealt with in this layer: the heap and stack are now modeled as fixed-length buffers, thus requiring proofs of absence of buffer overflows, the heap/stack mutations are now in-place instead of functional, thus requiring anti-aliasing proofs. This layer uses Low*, which allows extraction to verified C code.

4.5.3.1 Verifying Program Equivalence of Imperative and Functional `mark`

In Low*, we need to prove the program equivalence between the F* and Low* code intrinsically, that is along with the implementation of the function. To understand how Low* ensures memory safety of the C implementations, let us first see the code listing of `mark` in Low* as an example as shown in Figure 4.23 and its pre- and post-conditions in Figure 4.24. To differentiate from a functional implementation, imperative mark is qualified with a suffix `imp`. The function takes as input a buffer `hp` to store the heap, another buffer `st` to store the stack and `tp` to store the top pointer of `st`. Each of the Low* functions takes a pre-condition (`requires` clause) on the initial memory state `m` and a post-condition (`ensures` clause) about the initial and the final memory states $\texttt{m}_0$ and $\texttt{m}_1$ respectively. For example, the `mark_imp` requires the pre and post conditions about the memory state as shown in Figure 4.24[1].

The term `live m hp` states that `hp` is a `live` buffer in memory state `m`, where the location is specified by `loc_buffer hp`. The condition that the buffers `hp` and `st` should be disjoint in memory is captured in `disjoint (loc_hp) (loc_st)`. In the post-condition, a `modifies` clause is used which ensures that the buffers `hp`, `st` and `tp` got modified between the memory states $\texttt{m}_0$ and $\texttt{m}_1$. The `as_seq` function takes as input a memory state `m` and a buffer and creates the functional `seq` equivalent of the buffer in `m`. We need to reason about the stack contents upto stack top only. Therefore, we take a slice

[1] Some details have been elided. The complete specification can be found in the artefact.

```
let mark_imp hp st tp
 :Stack unit
 (requires λ m → ...)
 (ensures λ m0 _ m1 →  ...) =
  let inv m = ...(*loop invariants*)
  let guard (t: bool) m = inv m ∧
   (t = true  ⟹ B.get m tp 0) > 0) ∧
   (t = false ⟹ B.get m tp 0) = 0) in
  let test ()
   :Stack bool
    (requires λ m → inv m)
    (ensures λ _ ret m1 → guard ret m1)
     = (!*tp) >^ 0UL in
  let body ()
   : Stack unit
    (requires λ m → guard true m))
    (ensures λ _ _ m1 → inv m1)
     = mark_heap_body_imp hp st tp in
  C.Loops.while #(inv) #(guard) test body
```

Figure 4.23: A Low* implementation of mark

or portion of the stack from the start of the stack upto the stack top. This is captured in `(slice seq_st0 0 (index (seq_tp0) 0)`. The final clause ensures that the functional equivalent of buffer `hp` in the final memory state is equivalent to running a functional `mark` with the functional equivalent of `hp` in the initial memory m_0 and the slice of the functional equivalent of `st` upto `tp` in m_0. This clause ensures the output equivalence of functional mark and imperative mark. This way the Low* specification ensures the memory safety of the GC implementation as well as the functional equivalence with the functional implementation. The `inv` and `body` functions are used to specify the loop invariants and the body of the loop respectively.

The proof strategy involves demonstrating the output equivalence of each sub-function that composes the imperative garbage collector (GC) with its functional counterparts. For functions without loops, such as `darken_body` and `colorHeader`, the equivalence proof is straightforward. In these cases, the operations in both the functional and imperative implementations are the same and executed in the same order, with the only difference

```
requires λ m →
  let loc_hp = loc_buffer hp in
  let loc_st = loc_buffer st in
  let loc_tp = loc_buffer tp in
  live m hp ∧ live m st ∧ live m tp ∧
  disjoint (loc_hp) (loc_st) ∧ disjoint (loc_st) (loc_tp) ∧
  disjoint (loc_hp) (loc_tp)

ensures λ m0 _ m1 →
  let union = loc_union (loc_buffer hp)
                (loc_union (loc_buffer st) (loc_buffer tp))
  in
  let seq_st0 = as_seq m0 st in
  let seq_hp0 = as_seq m0 hp in
  let seq_tp0 = as_seq m0 tp in
  let seq_hp1 = as_seq m1 hp in
  let slice_st0 = slice seq_st0 0 (index (seq_tp0) 0) in
  live m1 hp ∧ live m1 st ∧ live m1 tp ∧
  (* Same disjoint clause as above *) ∧
  (modifies union m0 m1) ∧
  seq_hp1  = mark seq_hp0 slice_st0
```

Figure 4.24: Pre- and post-conditions of the Low* `mark` implementation in Figure 4.23

being the underlying data structures (sequences versus buffers).

For functions that include loops, we need an appropriate inductive loop invariant. However, since the functional mark-and-sweep implementation was designed to utilize only tail-recursive functions, which correspond to a well-defined ‘while‘ loop, the loop invariants that establish equivalence are quite simple. They essentially capture the relationship between each iteration of the loop in the imperative implementation and each invocation of the tail-recursive method in the functional implementation.

Loop invariant: In the `mark` function, there are two loops: one that corresponds to `darken`, which darkens the white field pointers of the currently popped object from the stack, and another that repeats the `mark` process itself. Consequently, we need to establish loop invariants for both the $\texttt{mark}_{imp}$ and $\texttt{darken}_{imp}$ functions. Consider an instance of the functional `darken` function. Let `j` represent the starting field of object `x`, which is inspected by `darken` to check for any white field pointers. Suppose $\texttt{seq_hp}_i$ and

$\texttt{slice_st}_i$ are the heap and stack sequences extracted from the memory state at iteration `i`. If $\texttt{seq_hp}_0$, $\texttt{slice_st}_0$ represents the heap, stack sequences extracted from the initial memory before `darken` begins, then we can demonstrate a loop invariant. This loop invariant asserts that calling `darken` with the heap and stack sequences extracted from memory at iteration `i` will yield the same result as calling `darken` on the heap and stack extracted from the initial memory. In mathematical terms, this can be expressed as:

$$\texttt{darken seq_hp}_0\ \texttt{slice_st}_0\ \texttt{x j} = \texttt{darken seq_hp}_i\ \texttt{slice_st}_i\ \texttt{x i},$$

Since the call to `darken` on the RHS of the above equation is supplied with the argument `i`, it only begins processing the field pointers from the index `i` onwards, with the effect of the previous iterations of `darken` being already present in the memory state at iteration `i`. This is due to the tail-recursive nature of `darken`. We now show why this is a valid inductive loop invariant of the `for` loop in `darken`.

Initialization: At the start of the loop $i = j$ and there is no change to memory state from the initial memory state. Thus, the loop invariant trivially holds.

Inductive Case: Consider an intermediate iteration, denoted as `i`. This indicates that the body of the loop, `darken_body`, is executed, resulting in the memory state at iteration `(i + 1)`. As previously stated, the equivalence between `darken_body` in both the imperative and functional programming contexts is clear, and we have already established this equivalence. Consequently, the heap and stack sequences at the beginning of iteration `(i + 1)` contain the results of executing `darken_body` at iteration `i`.

Due to the tail-recursive nature of the `darken` function, this means that `darken` at iteration `(i + 1)` carries forward all the changes made during iteration `i`. Moreover, according to the induction hypothesis, all memory state changes resulting from executing `darken` from the starting field `j` of `x` are preserved up to and including iteration `i`. Therefore the below equation holds,

`darken seq_hp`$_0$ `slice_st`$_0$ `x j = darken seq_hp`$_{(i+1)}$ `slice_st`$_(i+1)$ `x (i + 1),`

Termination: At the end of the loop, we simply return the heap and stack sequences. These sequences reflect all the changes in memory that occurred during the execution of `darken`, based on the initial memory state and the maintenance of the loop invariant. Essentially, this means that when `darken`$_{imp}$ completes its execution, the functional changes resulting from `darken`$_{imp}$ are equivalent to those produced by `darken`, which also starts from the same initial memory state. Given the tail-recursive nature of `mark`, we can establish functional equivalence between the imperative and functional versions of `mark` in a similar manner.

Mark stack overflow: There is a size limitations of concrete buffers and therefore the allocated stack has a fixed-size. This means there is a probability that the stack might overflow during `mark`. Low* rightfully captures this caveat and fails to typecheck if no conditions are provided that prevents stack overflow. Hence, to work around this, we set the stack size equals to the heap size and prove a lemma that states that when there is a non-grey object in the heap, the stack top is less than the heap size. The maximum size required to store all the objects in the heap is heap size in the worst case. Since the stack preserves the `stack invariant`, existence of one non-grey object means the stack top is less than the heap size, and thus less than the stack size. Thus, there is room in the stack to store this non-grey object, which will be converted to grey once it enters the stack.

4.5.3.2 Correctness Transfer from Functional to Imperative `sweep`

We establish program equivalence between the functional `sweep` implemented in F* and its imperative counterpart in Low*. Consequently, every functional property proven for the functional `sweep` is preserved by the imperative `sweep` as well. Figure 4.25 shows the specifications of `sweep` in Low*. `hp` is the concrete memory heap modelled as a `Buffer` data structure in Low*. Unlike `Seq` in F*, as mentioned earlier, the `Buffer` type has a fixed memory allocation. Both `sw` and `fp` are single-length 64-bit buffers that represent

```
val sweep_imp : (hp: heap) -> (sw: B.buffer UInt64.t) ->
    (fp: B.buffer UInt64.t) ->
    Stack unit
    (λ m → .........
        (* omitted buffer live  and disjoint conditions *))
    (λ m0 _ m1 →
      ...... (* omitted buffer live and disjoint conditions *)
      ...... (* omitted buffer modifies conditions *)
       let seq_hp0 = as_seq m0 hp in
       let sw_ptr = index (as_seq m0 sw) 0 in
       let fp_ptr = index (as_seq m0 fp) 0 in
       (fst (sweep seq_hp sw_ptr fp_ptr) == as_seq m1 hp ∧
        snd (sweep seq_hp sw_ptr fp_ptr) == index (as_seq  m1 fp) 0)
```

Figure 4.25: Pre- and post-conditions of the Low* `sweep` implementation

the sweep head and free-list pointer, respectively.

Since functional `sweep` is also a tail-recursive function, establishing the equivalence between functional and imperative `sweep` is performed by first proving the program equivalence between their corresponding sub-functions that do not have a loop and then demonstrating the overall program equivalence by making use of the equivalence proofs of the sub-functions and through a suitable loop invariant. Loop invariant can be discovered and proved in a similar manner as that described in Section 4.5.3.1.

4.5.3.3 Establishing Program Equivalence Between Imperative and Functional GC

The functional GC is defined as the composition of its three constituent functions:

```
let mark_sweep_gc h st r_list sw fp =
  let (st1, h1) = darken_roots h st r_list in
  let h2        = mark h1 st1 in
  sweep h2 sw fp
```

Let us explore how we can establish the functional equivalence between this functional GC and the imperative GC. The function `mark_sweep_gc` takes as input the initial heap `h`, a mark stack `st`, a root list `r_list`, a sweep pointer `sw`, and a free list pointer `fp`. It

```
val mark_sweep_gc_imp (hp: heap) -> (st : B.buffer UInt64.t) ->
        (tp: B.buffer UInt64.t) -> (rlist : B.buffer UInt64.t) ->
        (rlist_len: UInt64.t) ->  (sw: B.buffer UInt64.t) ->
        (fp: B.buffer UInt64.t)
 :Stack unit
 (* Similar conditions for mark...omitted *)
 (λ m → ...)
 (λ m0 _ m1 →
   let seq_st0 = as_seq m0 st in
   let seq_hp0 = as_seq m0 hp in
   let seq_r_list0 = as_seq m0 r_list in
   let seq_tp0 = as_seq m0 tp in
   let seq_rlist_len0 = as_seq m0 rlist_len in
   let seq_hp1 = as_seq m1 hp in
   let slice_rlist0 = slice seq_r_list0 0 (index (seq_rlist_len0) 0) in
   let slice_st0 = slice seq_st0 0 (index (seq_tp0) 0) in
   let result = mark_sweep_gc seq_hp0 slice_st0 slice_rlist0
                         (index (as_seq m0 sw) 0) (index (as_seq m0 fp) 0) in
     seq_hp1 = fst result ∧
     index (as_seq m1 fp) 0 = snd result)
```

Figure 4.26: Pre- and post-conditions of the Low⋆ `mark_sweep_gc` implementation

first calls `darken_roots` to grey the roots and populate the mark stack, then calls `mark` to color all reachable objects black, and finally calls `sweep` to reclaim unreachable objects and update the free list. It returns a pair of type `hp_fp`, consisting of the post-sweep heap and the updated free list pointer. The imperative counterpart `mark_sweep_gc_imp` (Figure 4.26) is shown to produce both the same heap and the same free list pointer as `mark_sweep_gc`.

Since there are no loops in the main body of the garbage collector, it is sufficient to preserve the exact order of operations in the imperative GC to align with that of the functional GC. This alignment, combined with the equivalence proofs of the constituent functions, allows us to verify program equivalence between the imperative GC in Low⋆ with its functional counter part in F⋆.

The specification is shown in Figure 4.26. As explained earlier, `hp`, `st`, `tp` are buffers representing the heap, stack and the stack pointer respectively. Similarly, `rlist`,

`rlist_len`, `sw` and `fp` are all buffers that carries roots, the last location of `rlist` up to which the roots are stored, the sweep pointer and the free-list pointer respectively. This specification establishes the functional correctness of the GC implementation with that of the algebraic properties of the functional GC implementation. Note that, the functional GC implementation acts as the middle layer of specifications, which aids in the final verification of abstract GC correctness defined in Definition 4.5.

4.5.4 Tuning F⋆ and Z3

F⋆ discharges proof obligations automatically for trivial specifications. For complex specifications involving heap reasoning, recursive data structures, or intricate arithmetic, Z3 may time out or fail to terminate. To deal with such cases, the following options are available.

Resource limits: The `--z3rlimit` flag controls how much computational effort Z3 does before giving up. The default is 5 and for difficult obligations, we found values between 500 and 1000 as necessary. For the complex depth-first search proof (Chapter 4, Section 4.5.1), which spans over 300 lines, we had to rely on a value of 2000.

```
#set-options "--z3rlimit 50"
```

Random seed: Z3 uses internal heuristics that depend on a random seed. Changing the seed can allow Z3 to find a proof it previously missed:

```
#set-options "--z3rlimit 50 --z3seed 17"
```

Restarting the solver: As Z3 works through a module, it builds up a significant set of assertions in its current context. Over time, this accumulated state can degrade performance and lead to situations where later proofs might fail even when earlier ones succeeded. We can use `#restart-solver` directive to reset Z3 to a clean state at that point in the file.

Query splitting: By default F* submits all verification conditions for a function to Z3 in bulk. The `--split_queries` option submits each condition separately, helping Z3 focus on one obligation at a time:

```
#set-options "--split_queries always"
```

Fuel and ifuel: For proofs involving recursive functions or inductive types, F* uses fuel to control how many times it unfolds recursive definitions during verification. The `--fuel` and `--ifuel` options set the unfolding depth for functions and inductive types, respectively:

```
#set-options "--fuel 2 --ifuel 2"
```

When working with Z3 for recursive definitions, balancing fuel is key. Increasing the fuel can help when Z3 struggles to unroll the definition fully, but too much can slow down the verification process.

Filtering the proof context: F* sends a larger context to Z3, including all the lemmas from standard libraries such as `FStar.Seq`, `FStar.Pervasives` and `Prims`, by default. However, this extensive context can sometimes overload Z3 and cause it to use irrelevant lemmas, leading to timeouts even when a straightforward proof is available. In such scenarios, we can use `--using_facts_from` option to restrict the facts that Z3 can use. For example, to exclude the entire `FStar.Seq` library and use only facts from the current module, one can use

```
#set-options "--using_facts_from '* -FStar.Seq'"
```

To restrict Z3 to use only facts from the current module (MyModule) and a specific library (e.g., FStar.Buffer), along with the essential Prims module, we can use,

```
#set-options "--using_facts_from 'Prims MyModule FStar.Buffer'"
```

If we want to make a specific definition invisible to Z3, then `[@ "opaque_to_smt"]` attribute can be used. This prevents Z3 from unfolding a definition that is not needed. When the definition has to be unfolded at a specific proof point, it is revealed locally using `reveal_opaque`. This gives precise control over what Z3 sees at each point in the proof, preventing context explosion in large verification developments.

In practice, verifying a garbage collector involves combining all of these techniques. Invariant preservation across GC phases requires increased resource limits and context filtering to prevent Z3 from being overwhelmed by irrelevant sequence lemmas.

4.6 END-TO-END GC CORRECTNESS

In this section, we state and discuss the end-to-end correctness theorem for our verified mark-and-sweep GC, which brings together all the properties established in the preceding sections.

Theorem 4.1 (GC Correctness). *Given a well-formed heap and a valid root set, executing the mark-and-sweep garbage collector produces a heap that is well-formed, whose object graph equals the reachable subgraph of the original heap with respect to the roots, and in which the data fields of all surviving objects are unchanged.*

This theorem is formalised in F* in Figure 4.27.

The end-to-end correctness theorem is written using Layer 2 primitives. The specification takes as arguments a well-formed heap `h_init`, a root set `roots`, a mark stack `st` that contains all the roots with the refinement that all the roots are grey, and a free-list pointer `fp`. We elide additional pre-conditions in the requires clause that relate the arguments to the heap.

The ensures clause captures the post-conditions after the execution of `mark` and `sweep`, and

```
val end_to_end_correctness_theorem
       (* Initial heap *)
       (h_init:heap{well_formed_heap h_init})
       (* mark stack - contains grey objects *)
       (st: seq Usize.t {pre_conditions_on_stack h_init st })
       (* root set *)
       (roots : seq Usize.t{pre_conditions_on_root_set h_init roots})
       (* free list pointer *)
       (fp:hp_addr{pre_conditions_on_free_list h_init fp})
: Lemma
( requires
  (* Pre-conditions elided for brevity. Important ones are:
     + The mark stack [st] contains all the [roots].
     + All the grey objects in the heap are in the mark stack [st].
   *) )
( ensures
   (* run the full GC *)
   let gc_result  = mark_sweep_gc h_init st roots mword fp in
   let h_post_gc  = fst gc_result in
   (* graph at init *)
   let g_init = graph_from_heap h_init in
   (* graph after GC *)
   let g_post_gc = graph_from_heap h_post_gc in
   (* GC preserves well-formedness of the heap *)
   (* 1 *) well_formed_heap h_post_gc ∧

   (* GC preserves reachable object set *)
   (* 2 *) (∀ x. x ∈ g_post_gc.vertices ⟺
              (∃ o. mem o roots ∧ reach g_init o x)) ∧

   (* GC preserves pointers between objects *)
   (* 3 *) (∀ x. mem x (g_post_gc.vertices) ⟹
              (successors g_init x) ==
              (successors g_post_gc x)) ∧

   (* The resultant heap objects are either white or blue only *)
   (* 4 *) (∀ x. mem x (h_objs h_post_gc) ⟹
              color x h_post_gc == white ∨
              color x h_post_gc == blue) ∧

   (* No object field (either pointer or immediate) is modified *)
   (* 5 *) field_reads_equal h_init h_post_gc )
```

Figure 4.27: Overall correctness theorem for the mark-and-sweep GC

is a conjunction of five properties. The first property states that the final heap `h_post_gc` is well-formed, corresponding to property (1) of Definition 4.5. The second property states that the graph formed from `h_post_gc` contains only those objects reachable from `roots` in `h_init`, and every reachable object in `h_init` is present in the final graph. The third property states that edges between reachable objects are preserved by the GC. Properties (2) and (3) together correspond to property (2) of Definition 4.5. The fourth property states that the resultant heap contains only white and blue objects. The abstract GC correctness does not refer to object colors as this is an implementation detail; however, it is an important detail that ensures the GC leaves the heap in a consistent state for the next cycle. Finally, the fifth property states that all fields, both immediate and pointer, of non-blue objects are unmodified by the GC. This is stronger than property (3) of Definition 4.5, which only requires data fields to be preserved. Since the mark-and-sweep GC does not move objects, pointer fields are also preserved.

One might wonder why the end-to-end correctness theorem is defined in Layer 2 ($F^\star$) and not in Layer 3 ($Low^\star$). As described in Section 4.5.3, our approach is to have the proofs of GC correctness in Layer 2 and prove the equivalence of the imperative GC with the functional GC in Layer 3, focusing only on memory safety properties.

4.7 FROM A MINIMAL TO AN INTEGRATION-READY GARBAGE COLLECTOR

This section presents a proof-oriented approach to extending the functionality of a garbage collector. The central idea is to enrich the collector by supporting more complex object layouts and additional object types, while reusing the same high-level specifications established for a minimal collector. Rather than redesigning correctness arguments from scratch, we retain the core specifications and most of the invariants and focus solely on adapting the proof engineering to accommodate the new features.

The motivation behind this approach is to demonstrate that fundamental lemmas and

end-to-end correctness theorem can be proved once for a bare-minimal garbage collector and then systematically reuse as the collector evolves. By progressively introducing richer object representations and heap behaviours, without altering the high-level correctness guarantees, we move incrementally toward a production-grade verified garbage collector, while maintaining a clear and modular proof structure.

In all preceding discussions, and as described in Section 4.3, we have worked with a base version of the `mark` function together with a simplified sweeping phase that does not perform block coalescing. This baseline design allows us to isolate and establish the core reachability preservation properties of the collector. As we refine the garbage collector to support additional OCaml object types such as closures and infix objects and to incorporate coalescing during sweeping, the collector begins to perform a richer set of heap transformations. These transformations affect not only the concrete heap layout but also the abstract object graph used to reason about reachability. As a result, the verification must account for new forms of heap manipulation, strengthened well-formedness conditions, modifications in the construction of the graph's edges and additional invariants to ensure that reachability preservation and memory safety are maintained.

The object graph construction continues to serve as the critical bridge between the abstract graph-level reasoning and the functional implementation of the garbage collector. Any extension to the collector therefore requires careful synchronization between heap transformations, graph construction, marking behaviour, and sweeping semantics. The following subsections address these refinements in a structured manner. We begin by characterizing the garbage collector as a heap transformer and classifying the different kinds of heap manipulations it performs (Section 4.7.1). We then strengthen the `well_formedness` predicate with additional conditions required to accommodate richer object representations (Section 4.7.2). Next, we adapt the object graph construction to accurately reflect reachability for these extended object types (Section 4.7.3). To

ensure the *safety* of the collector, we need to verify the `mark reachability lemma` (see Figure 4.13) and a modified `sweep subgraph lemma`. The necessary modifications to the `mark reachability lemma` are discussed in Section 4.7.4. Additionally, the new invariants required to establish the `sweep subgraph lemma` in the presence of coalescing are covered in Section 4.7.5. Furthermore, coalescing requires us to develop a new specification to ensure *liveness*, which will also be addressed in Section 4.7.5.

4.7.1 The Types of Heap Manipulations induced by our GC

A garbage collector can be viewed as a heap transformer operating via a restricted set of heap manipulations. We classify the transformations performed by our collector into four distinct types based on how they affect the allocated object set.

Type 1: Header recoloring. The allocated object set remains unchanged. The transformation is limited to recoloring object headers between allocated colors (white, grey, or black). The `mark` phase performs only this type of manipulation.

Type 2: Reclamation. An object's header color changes from white to blue, indicating reclamation. This reduces the allocated object set and occurs during the `sweep` phase.

Type 3: Free-list update. Newly reclaimed blocks are added to the free list by updating free-list pointers. This captures changes to free-list connectivity rather than allocation status, and also occurs during `sweep`.

Type 4: Coalescing. Adjacent free blocks are merged during `sweep`. The header of the leftmost block is updated to represent the combined size, reducing both fragmentation and the total number of objects in the heap.

While the `mark` phase is restricted to Type 1, the `sweep` phase introduces Types 2–4. A crucial implication of these manipulations is that they can affect heap well-formedness if not handled carefully. Since well-formedness is essential for constructing the object graph, maintaining it throughout the GC is the primary task in ensuring correctness.

```
let valid_field_number i h x field_start =
    i >= field_start ∧ i <= wosize(hd_address x, h)

let all_allocs_points_to_allocs h =
  (∀ x. seq.mem x (allocs h)  ⟹  points_to_allocs h x)

let points_to_allocs h x =
  let field_start = field_start_for_object h x in
  is_fields_points_to_allocs x h field_start

let get_parent_closure x h y =
               sub y (mul (wosize (hd_address y h) mword))

let is_fields_points_to_allocs x h field_start =
       (∀ (i:U64.t{valid_field_number i h x field_start}).
         isPointer (field_addr x h i) h   ⟹
         ((tag (hd_address (field x h i)) h ≠ infix_tag  ⟹
           (field x h i) ∈ allocs(h)) ∨
         (tag (hd_address (field x h i)) h = infix_tag  ⟹
          (let parent_closure = get_parent_closure x h (field x h i) in
           parent_closure ∈ allocs(h))))
         )

let well_formed_heap h =
   (* 1 *) well_formed_closure_objs h ∧
   (* 2 *) infix_not_as_obj h ∧
   (* 3 *) obj_does_not_overlap h ∧
   (* 4 *) all_allocs_points_to_allocs h
```

Figure 4.28: New well_formedness conditions

4.7.2 Extending the Heap `well-formedness`

We have shown how to verify the core tracing actions of a mark-and-sweep GC over OCaml objects. To scale to a production-grade GC, we must introduce support for additional OCaml object types, since the GC treats these differently, as shown in Figure 4.4. The `sweep` function also requires changes to support coalescing, as shown in Figure 4.7.

We now describe how the verification adapts as these features are added. The first step is extending the heap well-formedness property in Figure 4.10, which captures the guarantees the allocator must uphold when allocating different object types. The updated definition is given in Figure 4.28. Closure objects store the starting address of their

environment as a bit value within the object itself (see Section 2.3.1.2). To accommodate scanning fields of different object types, we extend `valid_field_number` with a new parameter `field_start`, alongside the heap `h`, field number `i`, and object address `x`.

The condition `all_allocs_points_to_allocs` checks that every allocated object in `h` satisfies `points_to_allocs`, meaning all its fields point to allocated objects. For fields pointing to infix objects, we locate the parent closure by subtracting `wosize` machine words from the infix object's address using `get_parent_closure`, and check that this parent is allocated. For non-infix fields, we simply verify that the field pointer is part of the allocated set.

The condition `well_formed_closure_objs` checks that all closure objects in `h` are structurally valid according to closure semantics (see Section 2.3.1.2), for example, that each closure has at least two fields. The condition `infix_not_as_obj` ensures no infix objects appear directly in the object set, and `obj_does_not_overlap` ensures no two objects overlap. With these conditions in place, the `mark` and `sweep` functions can be updated as shown in Figures 4.4 and 4.7, and together they form the foundation needed to establish the end-to-end correctness theorem (Figure 4.27) for the updated mark-and-sweep GC.

4.7.3 Adapting the Graph Construction

As discussed in Section 4.4.3, heap well-formedness is a prerequisite for graph construction. When adapting to different object types, the general graph definition in Figure 4.12 still holds. The only change is in `edges_of_graph`: the vertex set remains `allocs h`, and infix objects are never vertices in `G(h)`. This follows from `infix_not_as_obj`, which excludes infix objects from the object set. This is consistent with object semantics, as infix objects are embedded within their parent closure and are not independent heap objects. The `mark` function reflects this by marking the parent closure whenever it encounters an infix object.

We now describe how edge construction is affected by closure objects, infix objects, and objects with `tag` values at or above `no_scan_tag`. Edge construction mirrors the scanning behaviour of `mark`. Objects with `tag` $\geq$ `no_scan_tag` have no edges, since `mark` does not scan their fields, as shown in Figure 4.3. For closure objects, edges are constructed by scanning fields from the start of the environment (see Section 2.3.1.2), matching how `mark` handles them in Figure 4.4. This is where `well_formed_closure_objs` becomes essential: without it, neither `mark` nor graph construction can safely access the second field of the closure, which stores `startenv`, since there would be no guarantee the field exists.

For an edge `(x, y)`, `y` must belong to `allocs h`, which is maintained by `all_allocs_points_to_allocs`. Edges are then constructed as follows.

1. If field `i` of `x` points to an infix object `y`, the edge `(x, z)` is created instead, where `z` is the parent closure of `y`.
2. Otherwise, the edge `(x, y)` is created directly from the field pointer.

Without `all_allocs_points_to_allocs`, both `mark` and graph construction would fail to satisfy the necessary verification conditions. These adaptations keep graph construction in sync with `mark`, and high-level lemmas such as `mark_reachability_lemma` in Figure 4.13 remain unchanged.

4.7.4 Extending `mark` and Its Correctness Proof

With the addition of closure, infix, and no-scan objects, the behaviour of `mark` changes as shown in Figures 4.3 and 4.4. However, since `mark` performs only Type 1 heap manipulations (Section 4.7.1), all objects and allocated regions are preserved. In particular, `mark` does not modify object sizes, tags, or field values, which is sufficient to show that all conditions in the new well-formedness property (Figure 4.28) continue to hold after marking.

As a result, the high-level specification `mark_reachability_lemma` remains unchanged.

The object graph is also preserved across `mark`, since the updated graph construction ensures it continues to accurately reflect the heap in the presence of the new object types. The only work required is in the proof engineering to accommodate them.

The main proof engineering effort concerns stack equivalence (Section 4.5.2.3). We adapt the graph construction so that simulating `mark`'s tracing on the heap corresponds to running `dfs` on the object graph. With this correspondence established, stack equivalence follows with the help of a few additional lemmas, after which the `stack_invariant` and `visited-set_invariant` are sufficient to prove the `mark_reachability_lemma` (see Figure 4.22).

4.7.5 Correctness of `sweep` with Coalescing

Recall that the primary function of a GC is to partition the heap into allocated and free objects. As discussed in previous chapters, the *safety* property of a GC ensures that all allocated objects are reachable prior to the garbage collection process. In contrast, the *liveness* property guarantees that all free objects correspond to objects that were unreachable before the GC runs. Therefore, the correctness of a GC depends on both the accurate identification of reachable objects and the proper reclamation of unused objects.

Our `mark_reachability_lemma` establishes the correctness of reachability, while the `sweep_subgraph_lemma` ensures that only the unreachable objects are freed by the GC. However, in the presence of sweep with coalescing, as shown in Figure 4.7, the specifications must be revised to preserve the liveness property of `sweep`. In addition, new invariants must be identified to ensure that `sweep` continues to preserve the reachable subgraph. We will discuss these issues in detail below.

Invariant discovery. As noted in Section 4.7.1, the `sweep` function performs type 2–4 heap manipulations. Proving that `sweep` maintains heap well-formedness, as defined in Figure 4.28, is therefore non-trivial. Heap well-formedness is a fundamental requirement for constructing the object graph, as discussed in Section 4.7.3. Consequently, we cannot

```
(* All blue or white objects before sweep *)
(∀ x. x ∈ whites(h) ∨ x ∈ blues(h) ⟺
    (* Either x part of blue objects after sweep*)
     x ∈ blues(h_1) ∨
    (* or *)
    (* x is not part of blue objects and x is a field address of
       some blue object y after sweep *)
     ¬(x ∈ blues(h_1)) ∧
       (∃ y i. y ∈ blues(h_1) ∧
          valid_field_number i h_1 y (field_start y h_1) ∧
          x = field_addr y h_1 i))
```

Figure 4.29: New liveness Specification for sweep

establish the sweep_subgraph_lemma without first reasoning about well-formedness in the presence of these heap updates. In the following discussion, we show how the well-formedness conditions can be established even when sweep performs type 2–4 manipulations.

When the body of sweep changes the color of a white object to blue, the set of allocated objects shrinks. As a result, if an allocated object later in the heap, one that the sweep head has not yet reached, contains a field pointer to this new blue object, the well-formedness condition is violated. This is because well-formedness requires that all allocated objects point only to other allocated objects. Therefore, the sweep body does not preserve heap well-formedness, and an inductive proof over the sweep loop cannot be applied directly.

Instead, we rely on the mark_reachability_lemma to show that all black objects are reachable from the root set. By the transitivity of the reach predicate, the field pointers of all black objects are also reachable, which implies that they point only to black objects. From the algebraic properties of sweep shown in Figure 4.11, all black objects before sweep become white after sweep. Moreover, the same algebraic properties establish directly that sweep does not modify the field values of allocated objects.

In the presence of coalescing, however, a more intricate invariant is required. Let h and h_1 be the heap before and after sweep respectively. The following invariant captures the

necessary property that helps us to prove the well-formedness of the heap after sweep:

```
(∀ x. x ∈ blacks(h) ∨ tag x h = infix ⟹
   wosize x h = wosize x h₁ ∧
   tag x h = tag x h₁)
```

Here, tag and wosize denote the tag and object size of an object x in heap h. This invariant is proved for the sweep body and then lifted to sweep using induction. If a field pointer refers to an infix object, then the preservation of its tag and size ensures that the corresponding parent closure remains unchanged. Similarly, when a field pointer refers to a non-infix object, that object is black before sweep and therefore white afterwards. Additionally, preserving the tag and object size of black objects ensures that, for closure objects, the field-start address is also preserved. Therefore, since black objects before sweep become white after sweep, and since the object size and tag of both black and infix objects are preserved as well as the field values of white objects after sweep remain unchanged, it follows that all white objects after sweep point only to white objects through their unmodified field pointers. This invariant is now sufficient to ensure the new well-formedness property (Figure 4.28).

Specification change to ensure liveness. In the presence of coalescing, adjacent free objects may be merged during sweep, causing the object set to shrink. As a result, there is no longer a one-to-one correspondence between white or blue objects before sweep and blue objects after sweep. Consequently, the third algebraic property of sweep, which relates white or blue objects before sweep to blue objects after sweep (Figure 4.11), no longer holds.

This necessitates a revised specification to capture the liveness property of sweep in the presence of coalescing. The new specification is presented in Figure 4.29. Let h denote the heap before sweep and h_1 the heap after sweep. Then, every object that is white or blue before sweep either remains as a blue object after sweep or is coalesced into another

blue object that occurs earlier in the heap. It is important to note that coalescing an object x with object y means that the size of y is adjusted to include x. This specification essentially specifies that x must be a valid field address of y after the coalescing process.

4.8 BENCHMARKING THE GARBAGE COLLECTOR

A verified system is only practically useful if it can be integrated into a real runtime and run real programs at acceptable performance. These two goals, successful integration and competitive runtime performance, are both necessary for verification to have practical impact. The results presented here demonstrate that both are achievable: the verified GC successfully runs a range of OCaml programs, and its performance is competitive with conservative collection approaches while providing the strong formal correctness guarantees that those approaches lack.

We begin by quantifying the verification effort in Section 4.8.1, analysing the development time and proof complexity required across the different verification layers. Section 4.8.2 describes the integration of our verified GC with the OCaml 4.14.1 runtime system, detailing the necessary modifications for compatibility. Finally, Section 4.8.3 presents a performance evaluation against the standard OCaml GC and the Boehm-Demers-Weiser conservative GC. The results show that our verified GC is competitive with the standard OCaml GC on many benchmarks, while preserving the formal correctness guarantees.

The integration work described in Section 4.8.2 was performed by Dipesh Kafle and is presented here to provide a complete picture of the end-to-end system. The benchmark suite design, performance measurements, and analysis presented in Section 4.8.3 were conducted as part of this thesis work.

4.8.1 Verification effort

The verification effort is summarized in Table 4.1. We calculate the effort in terms of different metrics such as the lines of code, the number of definitions and lemmas, the

time required by F*/Low* to discharge the VCs, and the human development time. The development time is in terms of number of person-months. We divide the verification effort across the different layers of our approach. Since we co-develop programs and proofs, the total number of lines of code include both the programs and proofs together.

Table 4.1: Approximate Verification effort. Development effort (Dev effort) is in person-months.

Modules	**#Lines**	**#Defns**	**#Lemmas**	**Verif time**	**Dev effort**
Graph	4653	72	81	2m3s	3
DFS	657	1	9	2m5s	9
Functional GC	30000	70	350	120m	12
Imperative GC	4000	20	20	27m43s	3

The Graph module contains the mathematical graph and reachability definitions, and several functions and lemmas on paths in the graph. These are needed to implement and prove the correctness of the DFS module. Proving the reachability property of DFS (Figure 4.17) was particularly tricky as we needed to discover complex inductive invariants involving the `reach` predicate. The high development cost of the DFS module (9 person-months for 657 lines) reflects the difficulty of discovering the correct inductive invariants involving the `reach` predicate, rather than the size of the code itself. Z3 cannot perform induction autonomously, so each inductive step required manually decomposing the reachability argument into lemmas that the solver could handle individually. This invariant discovery phase dominated the effort.

For context, Hawblitzel and Petrank (Hawblitzel and Petrank, 2009) report 2,398 lines for the copying collector, 3,038 lines for the mark-sweep collector, and 779 lines of shared code (all non-comment, non-blank BoogiePL lines), with verification times of 115 seconds and 70 seconds respectively, but do not report person-months. McCreight et al. (McCreight *et al.*, 2007) report approximately 64,000 lines of Coq across their framework. The CakeML generational collector (Ericsson *et al.*, 2019) does not report person-months either. Direct comparison is therefore difficult, as each project uses a different framework, targets a different language runtime, and verifies different properties.

What is clear is that the layered approach adopted here allowed graph-theoretic reasoning to be separated from implementation concerns, which we believe reduced the overall proof burden compared to a monolithic approach.

The Functional GC module incorporates multiple proofs that assert the correctness of the functional mark-and-sweep implementation, alongside graph construction and demonstrations of equivalence between the `mark` and `dfs` functions. This requires development of inductive invariants to show equivalence between corresponding methods of DFS and functional mark, as well as proving algebraic properties of the various bit manipulation operations performed by the GC. Once the functional GC was developed, implementing and verifying the imperative GC in Low* is more straightforward. The various verification tasks associated with the imperative GC module include establishing suitable loop invariants to prove equivalence between the functional and imperative GC, proving various memory safety properties such as lack of aliasing, ensuring allocation of memory before access, no use-after-free bugs, etc.

Throughout the layers, we also adopted an incremental approach in adding complex GC features, such as closures and infix objects. Initially, we proved the GC correctness over a basic version that does not distinguish between different types of OCaml objects. Subsequently, we introduced modifications in both the implementations and proofs to accommodate more complex GC variants.

Note that our trusted code base now includes F*/Low* and the KaRaMeL compiler which translates Low* to C. It has to be noted that these tools also serve as the trusted code bases for previous projects such as Everest (Bhargavan *et al.*, 2017*a*) and Everparse (Ramananandro *et al.*, 2019), which focus on verifying cryptographic implementations and parsers.

The verification through F*/Low* was challenging due to several reasons. F* uses Z3 SMT solver. Our verification conditions (VCs) are not restricted to decidable logics.

While Z3 does best-effort reasoning, it may take a long time to prove some VCs or the proof does not terminate. We had to tune the timeouts for individual lemmas to get them to discharge. In addition, Z3 is also non-deterministic. The same VC may be discharged in one run and not in another depending upon the solver state. F* provides some support for dealing with proof instability, such as running in `quake` mode or using `proof-recovery` mode to recover from proof failures. However, the fundamental issue of non-determinism remains.

4.8.2 Integration with OCaml

Our goal in this work is to develop a practical verified GC for OCaml that can serve as a replacement for the unverified GC. We have successfully extracted the verified C code for the GC functionality from Low* using the KaRaMeL compiler. We have integrated the extracted code into the OCaml 4.14.1 runtime system, replacing the existing GC in the bytecode runtime.

Unlike the GC in OCaml 4.14.1, our verified GC is stop-the-world and non-generational. We use an unverified next-fit allocator written in Rust by Dipesh Kafle, which allocates objects in the verified heap. As mentioned before, our heap is a single, contiguous block of memory (encoded as an F* buffer), into which the objects are allocated. The verification of the allocator is orthogonal to the focus of the work. There is a recent work on StarMalloc (Reitz *et al.*, 2024*a*) which provides a verified, hardened memory allocator written in F*/Low*. We plan to investigate integrating StarMalloc with our verified GC in the future.

When the heap is full, the verified GC is triggered. We use the existing root marking procedure in the OCaml runtime to darken the roots and push them to the verified mark stack. This is followed by the call to the verified mark-and-sweep function.

We made the following small modifications to the extracted code to facilitate integration with the OCaml runtime. The first modification is in the sweep code, where we have

implemented coalescing of consecutive free blocks. This is done to reduce fragmentation. The correctness of the coalescing operation, and in particular its preservation of the reachable subgraph, is verified as part of the sweep correctness proof in Section 4.7.5. The second modification is necessitated by the fact that infix objects do not appear in the mark stack in the verified GC. In contrast, they do (during root marking) in the OCaml runtime. Since the root marking is done by the OCaml runtime, we have added a wrapper function that inserts the parent closure of an infix object into the mark stack if an infix object appears as a GC root.

It is important to acknowledge that the unverified components surrounding the verified GC core form a substantial trusted computing base. Specifically, the Rust allocator, the OCaml root marking procedure, and the infix object wrapper are all unverified. Bugs in any of these components could corrupt the heap or produce an incorrect root set, undermining the guarantees established by the GC proofs. The verified GC proofs assume that the heap is well-formed and the root set is complete at the point of collection; if either assumption is violated by an unverified component, the formal guarantees do not apply to the resulting behaviour.

4.8.3 GC evaluation

Having integrated the verified GC with the OCaml runtime, we now evaluate its performance characteristics to assess whether the strong correctness guarantees come at an acceptable performance cost.

4.8.3.1 Experimental Setup

We evaluate the performance of the verified GC on a variety of benchmark programs from the Computer Language Benchmarks Game (Gouy) as well as larger programs – cpdf (an industrial-strength PDF processing tool) and yojson (JSON processing library) – from the OCaml ecosystem. The larger programs have a deep dependency graph of other packages from the OCaml ecosystem. The performance evaluation was performed on a

2-socket, Intel® Xeon® Gold 5120 CPU x86-64 server, with 28 physical cores (14 cores on each socket), and 2 hardware threads per core. Each core runs at 2.20GHz and has 32 KB of L1 data cache, 32 KB of L1 instruction cache and 1MB of L2 cache. The cores on a socket share a 19 MB L3 cache. The server has 64GB of main memory and runs Ubuntu 20.04 LTS.

4.8.3.2 Benchmark Characteristics

The benchmark characteristics are given in Table 4.2. Alloc, Promote and RSS indicate the allocated memory, memory promoted from minor to major heap and the maximum resident set size in MB. Since the running times for OCaml programs are a function of the heap size, for a fair comparison, we have chosen the heap size of the verified GC such that the maximum resident set sizes in both the cases are similar, except in cases where the verified GC runs out of memory with small heap sizes. The exceptional case occurs since the verified GC can waste space due to fragmentation. Coalescing of adjacent free blocks during sweep reduces fragmentation but does not eliminate it entirely, as the non-moving nature of the collector means that long-lived objects can strand free space between them. From the table, we can see that the verified GC is able to run fairly large programs using similar RSS. Some of the programs also allocate a lot of memory, triggering many GCs.

4.8.3.3 Performance Results

Figure 4.30 shows the running time of the benchmarks run using different GCs, normalised against the default OCaml 4.14.1 GC. Each benchmark was run five times and the reported figure shows the mean normalised running time. Variance across runs was low (within 2–3% for most benchmarks), confirming that the results are stable. The comparison also includes OCaml equipped with Boehm-Demers-Weiser (BDW) GC (Boehm and Weiser, 1988). BDW GC is a widely known, pragmatic GC for uncooperative environments. This means that, unlike the other GCs used in the comparison, BDW GC does not have access to precise root set information. It operates conservatively and may over-approximate

Table 4.2: Benchmark characteristics. Alloc, Promote and RSS are in MB.

	OCaml 4.14.1					Verified GC	
Benchmark	**Alloc**	**Promote**	**# Minor**	**# Major**	**RSS**	**GCs**	**RSS**
BinaryTrees	15206	7900	7647	87	516	42	515
CountChange	905	140	458	11	145	5	260
FannkuchRedux	0.03	0	0	0	2.68	0	3.5
Fasta	3171	0.03	1569	4	44	72	67
Quicksort	19	0.02	1	0	22	0	22
Nbodies	808	0.04	405	2	4.66	3819	3.63
Mandelbrot	3009	0.13	1508	9	4.3	34201	3.65
Spectralnorm	3052	0.06	1529	8	4.7	47	67
Knucleotide	140	17	52	6	57	2	67
Cpdf	512	200	254	11	140	1	517
Yojson	129	14	45	16	17	48	14

the actual set of accessible objects. On many programs, the verified GC performs on par with the baseline GC and never worse than the BDW GC. On benchmarks where the verified GC and BDW GC are slower; we can attribute the slowdown to the lack of a generational collector. For example, on the Nbodies benchmark, the verified GC is almost 6× slower than the baseline. From Table 4.2, we observe that almost none of the memory is promoted to the major heap. Without a generational collector, the verified GC spends a lot of time sweeping garbage. In contrast, a copying minor collector in the baseline only needs to copy live objects to the major heap. The 6× slowdown on Nbodies is primarily driven by the number of GC invocations: the verified GC triggers 3,819 collections compared to only 2 major collections in the baseline (Table 4.2). This occurs because Nbodies allocates heavily into the minor heap, and the baseline's copying minor collector promotes survivors efficiently with minimal pause. Without a generational design, the verified GC must perform a full mark-and-sweep cycle for every allocation pressure event, spending the majority of execution time in the sweep phase reclaiming short-lived objects that a minor collector would handle in microseconds.

Despite these performance differences on allocation-intensive benchmarks, the results demonstrate that the verified GC is competitive on the benchmarks tested, performing

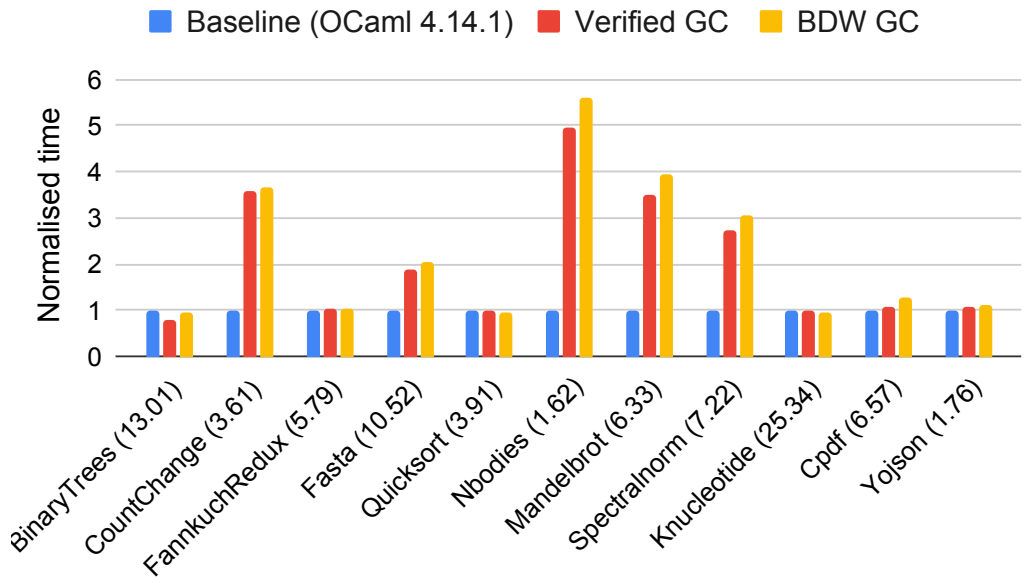


Figure 4.30: Normalized running time of different OCaml GCs. The numbers in the parenthesis next to the benchmark names are the running time in seconds for the baseline OCaml 4.14.1 GC.

comparably to the baseline on many workloads and consistently outperforming the conservative BDW collector. The key result is that we achieve strong correctness guarantees: no live objects are mistakenly collected, and unreachable ones are eventually reclaimed while maintaining practical performance. In many cases, our performance is comparable to or even better than conservative collectors like BDW GC.

4.9 CHAPTER SUMMARY

This chapter presented the design, implementation, and formal verification of a stop-the-world mark-and-sweep GC for OCaml, developed in F* and Low*. We began with a generic, implementation-independent definition of GC correctness (Section 4.2) expressed in graph-theoretic terms, requiring that the object graph after collection equals the reachable subgraph of the initial heap. This captures both safety (only unreachable objects are collected) and liveness (all unreachable objects are reclaimed).

We then instantiated this specification for OCaml (Section 4.4), formalising the heap representation, object layouts, and the algebraic properties of `darken_roots`, `mark`, and

`sweep`. The verification followed a three-layer methodology (Section 4.5). Layer 1 provides a verified depth-first search with a proof that it correctly computes the reachable vertex set. Layer 2 establishes functional correctness of `mark` and `sweep` by proving that `mark` behaves equivalently to `dfs`, then connects coloring behaviour to graph reachability via the `mark_reachability_lemma` and `sweep_subgraph_lemma`. Layer 3 provides the imperative Low* implementation and proves its equivalence with the Layer 2 model, ensuring memory safety and the absence of buffer overflows. This equivalence carries the abstract correctness guarantees through to the extracted C code.

The end-to-end correctness theorem (Section 4.6) brings these layers together, establishing that the verified GC preserves heap well-formedness, produces an object graph equal to the reachable subgraph, and leaves all surviving object fields unchanged.

Section 4.7 then showed how the baseline collector can be extended toward a production-grade GC. Supporting closure objects requires handling infix objects as interior pointers, while no-scan objects must be left untraced. These richer object types required strengthening the well-formedness predicate and adapting the graph construction to match `mark`'s tracing behaviour. We also added sweep coalescing to reduce fragmentation, which required revising the sweep specification. Throughout all these extensions, the high-level correctness specifications remained unchanged, with the additional effort confined to proof engineering.

Finally, Section 4.8 evaluated the verified GC in practice. The collector is integrated into the OCaml 4.14.1 bytecode runtime, replacing the existing GC with the extracted verified C code. Performance results across benchmarks from the Computer Language Benchmarks Game and larger programs such as `cpdf` and `yojson` show that the verified GC is competitive with the baseline OCaml GC on many workloads and consistently outperforms the conservative Boehm-Demers-Weiser collector, demonstrating that strong formal correctness guarantees and practical performance are not mutually exclusive.

CHAPTER 5

FUTURE WORK AND CONCLUSIONS

Let me try to explain to you what, to my taste, is characteristic of all intelligent thinking. It is that one is willing to study in depth an aspect of one's subject matter in isolation for the sake of its own consistency, all the time knowing that one is occupying oneself only with one of the aspects... It is what I sometimes have called the separation of concerns, which, even if not perfectly possible, is yet the only available technique for effective ordering of one's thoughts, that I know of.

– Edsger Dijkstra; On the role of scientific thought; EWD447; 30th August 1974

5.1 SUMMARY OF CONTRIBUTIONS

This thesis demonstrates that a production-grade mark-and-sweep GC for OCaml can be formally verified end-to-end using a layered approach, with reachability-based specifications independent of the GC design, and extracted into portable C that integrates with the OCaml runtime. The key novelty is the combination of three things, (1) a graph-reachability top layer that is design-agnostic, (2) an intermediate functional GC layer that handles OCaml's complex object model, and (3) a Low* bottom layer that extracts to portable C. Together, these layers connect high-level formal reasoning with low-level systems implementation, showing that verification can be applied to non-trivial runtime components.

It is important to be clear about what this verification covers and what it does not. The verified component is a stop-the-world major heap collector for OCaml 4.14.1. The minor heap collector, the memory allocator, incremental scheduling mechanisms, and concurrent support introduced in OCaml 5.0 are outside the scope of this work. These represent a substantial gap between the verified collector and a fully verified production runtime, and they constitute the natural agenda for future work.

5.2 CHALLENGES AND LIMITATIONS

One major difficulty encountered during development is **proof maintenance**. When new object semantics were incrementally added, substantial updates to existing proofs were necessary. This was especially evident when modifying traversal logic or introducing new object representations, as these changes affected multiple invariants simultaneously.

SMT-based automation also introduced its own set of challenges. While SMT significantly reduced the burden of manual proof construction, it also introduced fragility. Proofs can break due to limitations in solver heuristics, and when working with non-linear arithmetic, recursive definitions, or quantified properties, SMT-based systems can be unpredictable.

Like any formal verification effort, this work has boundaries that should be stated clearly. The verified component is the stop-the-world major heap collector. The root enumeration code, the memory allocator, and the minor collector are not verified, and bugs in any of these could affect the runtime behaviour despite the GC proofs. The verification also depends on a trusted computing base comprising F*'s type checker, the Z3 SMT solver, and the KaRaMeL compiler, each of which is assumed correct. F*'s core metatheory has been formally studied (Swamy *et al.*, 2016) and KaRaMeL's correctness for the Low* fragment has been established through pen-and-paper argument (Protzenko *et al.*, 2017), which provides reasonable confidence, though not an absolute guarantee. These are standard and accepted trade-offs in verified systems work, consistent with projects such as CompCert (Leroy, 2009) and seL4 (Klein *et al.*, 2009).

5.3 KEY INSIGHT: THE IMPORTANCE OF A VERIFICATION-FRIENDLY DESIGN

A central lesson that emerged during this work is that, *the structure of the system has a direct impact on the complexity of its verification*. As the garbage collector evolved, the addition of each new feature, such as distinguishing pointer and non-pointer fields, handling closures and infix objects, extending the sweep phase to construct free-lists and coalescing, required modifications across multiple parts of the implementation and hence proofs, particularly in traversal logic, graph construction, and invariants.

A fundamental issue revealed through this experience is that the implementation tightly couples object semantics, traversal behaviour, and memory management policies. This coupling is not incidental because it reflects the GC design of OCaml 4.14.1, which prioritises performance over modularity. It was only through incremental verification of this system that we came to appreciate how much this coupling intensifies the effort required to prove it. Each time a new object type or GC feature was added, previously established proofs had to be revisited because the invariants governing traversal, graph

construction, and memory management were not clearly separated.

This points to a deeper lesson for proof-oriented GC development. When the goal is to incrementally build and verify a system by discovering specifications and invariants along the way, modularity is essential. Already-proved components should remain stable as the system grows, and adding new functionality should require only localised proof effort rather than cascading changes across the development. The less coupling there is between object semantics, traversal, and memory management, the more naturally verification scales.

Note that this thesis follows a post-hoc verification approach applied to an existing GC design. The lesson is therefore not that this approach was wrong, but that *future* verified GC designs should be verification-first and modular from the outset. For practitioners attempting to verify their own runtime components, this experience offers several concrete takeaways. First, grounding specifications in abstract reachability rather than implementation-specific invariants makes proofs more robust and easier to maintain. Second, separating concerns through layers ensures that each layer remains individually tractable. Third, investing early in reusable infrastructure such as graph and traversal libraries pays dividends throughout the project, even if the upfront cost feels significant. Finally, verifiability should be treated as a first-class design criterion alongside performance, ideally from the very beginning of development.

5.4 ABSTRACT GC CORRECTNESS: A REUSABLE FOUNDATION

The abstract GC correctness specification developed in Chapter 4 was designed with generality in mind. Rather than being tied to the particulars of stop-the-world mark-and-sweep, it is formulated at the level of graph reachability and heap well-formedness. This chapter examines whether that specification can serve as a reusable foundation for reasoning about collectors that differ substantially in their runtime behaviour.

We consider two collectors present in the OCaml runtime: an incremental variant of mark-and-sweep (used for the major heap) and a copying collector (used for the minor heap). For each, we show how the abstract correctness specification from Chapter 4 can be extended to accommodate the collector's distinguishing features, such as mutator interleaving and write barriers in the incremental case, and object relocation via a mapping function in the copying case, while preserving the same core structure of reachability preservation and heap well-formedness.

The goal of this chapter is not to deliver fully mechanised proofs for these collectors. Rather, it demonstrates that our abstract specification is extensible in the sense that the definitions generalise naturally, and the proof obligations they generate are structurally related to those already discharged for the stop-the-world collector. Completing these proofs and implementations is left as future work and we discuss the anticipated proof-reuse opportunities and the points where fresh effort would be required at the end of each section.

5.4.1 Incremental mark-and-sweep GC

An incremental mark-and-sweep GC, as the name suggests performs the GC work in *slices*. During each slice, the GC performs a part of the mark or sweep work and these slices are interleaved with the mutator actions, i.e., the execution of the OCaml program. The main advantage of an incremental GC is that it can reduce the pause times of the application. In a stop-the-world GC, the GC actions are performed in a single shot, which means that the application is paused for the entire duration of the GC. With an incremental GC, the program is paused only for the duration of a slice.

OCaml uses an incremental mark-and-sweep collector for the major heap, employing a snapshot-at-the-beginning variant (Yuasa, 1990) of the incremental GC. This ensures that the objects that are reachable at the beginning of the cycle remain reachable at the end of the cycle as well. This is achieved by using a deletion write barrier, which, on a

field update, marks the old value at that field. As a result, the old value, if unmarked, gets marked before it is overwritten by the new value. This ensures that the old value and the objects transitively reachable from it are reachable at the end of the cycle. As a result, despite the updates to the object graph, all the objects that were reachable at the start of the cycle remain reachable at the end of the cycle (which is the snapshot-at-the-beginning property). Note that the write barrier is the means by which the mutator coordinates with the collector.

How do we reason about the correctness of an incremental mark-and-sweep GC? We can still reason about the overall correctness of the GC cycle similar to the abstract GC correctness (Definition 4.5) from Section 4.2. However, we need the mutator to provide us a summary of the changes that it has made to the heap during the GC cycle. Let `new_allocs` be the set of objects that are allocated by the mutator during a GC cycle. Whenever the mutator allocates a new object, the object id is added to the `new_allocs` set. Let `added_edges` and `deleted_edges` be the set of edges that are added and deleted by the mutator during the GC cycle. Assume that `new_allocs`, `added_edges` and `deleted_edges` are empty at the start of the cycle. The `added_edges` and `deleted_edges` are computed in the write barrier. The write barrier is a function that gets called before a write `x := y` is performed. The sets `added_edges` and `deleted_edges` are computed in the write barrier as follows:

```
(* called before [x := y] *)
let write_barrier (x,y) = (* assuming that [y] is a heap object *)
  let old = !x in (* assuming that [old] is a heap object *)
  added_edges := (added_edges \ {(x,old)}) ∪ {(x,y)};
  deleted_edges := (deleted_edges ∪ {(x,old)}) \ {(x,y)};
  (* ...other write barrier actions... *)
```

Note that the above write barrier maintains the invariant that `added_edges` ∩

deleted_edges = $\emptyset$ at all times. To see why, consider the three cases that can arise for any edge e:

- **Edge added then deleted.** When e is first added, it enters added_edges and is removed from deleted_edges. When e is subsequently deleted, it enters deleted_edges and is removed from added_edges. The net effect is that e appears only in deleted_edges.
- **Edge deleted then added.** When e is first deleted, it enters deleted_edges and is removed from added_edges. When e is subsequently added, it enters added_edges and is removed from deleted_edges. The net effect is that e appears only in added_edges.
- **Edge added, deleted, then added again.** After the first add, $e \in$ added_edges. After the delete, $e \in$ deleted_edges and $e \notin$ added_edges. After the second add, e is removed from deleted_edges and re-enters added_edges. The net effect is again that e appears only in added_edges.

In each case, the set operations ensure that e can appear in at most one of the two sets at any point, preserving the invariant added_edges $\cap$ deleted_edges = $\emptyset$.

With this information, we can define the correctness of an incremental mark-and-sweep GC. Let $\mathtt{h}_0$ be the initial state of the heap on which the GC operates, such that $\omega(\mathtt{h}_0)$ holds, and let r be the set of *roots*, which are pointers to objects into $\mathtt{h}_0$. Let $\mathtt{G}(\mathtt{h}_0).\mathtt{V}$ be the vertex-set and $\mathtt{G}(\mathtt{h}_0).\mathtt{E}$ be the edge-set of $\mathtt{G}(\mathtt{h}_0)$. Let $\mathtt{h}_1$ be the heap after a full cycle of the incremental mark-and-sweep GC. Let $\mathtt{G}(\mathtt{h}_1).\mathtt{V}$ be the vertex-set and $\mathtt{G}(\mathtt{h}_1).\mathtt{E}$ be the edge-set of $\mathtt{G}(\mathtt{h}_1)$. Let $\mathtt{RG}(\mathtt{h}_0,\mathtt{r})$ be the reachable sub-graph residing in $\mathtt{G}(\mathtt{h}_0)$. Also let $data(x, h)$ denotes the non-pointer field values of object x in heap h, that is, the sequence of immediate values stored in the fields of x that are not heap pointers. This captures the requirement that the GC does not modify the program-visible data stored in live objects.

Definition 5.1 (**GC Correctness**)**.** An incremental mark-and-sweep GC is said to be

correct if the following conditions hold:

1. $\omega(\mathtt{h}_1)$
2. a) $\mathtt{G}(\mathtt{h}_1).\mathtt{V} = \mathtt{RG}(\mathtt{h}_0,\mathtt{r}).\mathtt{V} \cup \mathtt{new_allocs}$

   b) $\mathtt{G}(\mathtt{h}_1).\mathtt{E} = (\mathtt{RG}(\mathtt{h}_0,\mathtt{r}).\mathtt{E} \cup \mathtt{added_edges}) \setminus \mathtt{deleted_edges}$
3. $(\forall\, \mathtt{x}.\ \mathtt{x} \in \mathtt{G}(\mathtt{h}_1).\mathtt{V} \implies \mathtt{data}(\mathtt{x},\mathtt{h}_0) = \mathtt{data}(\mathtt{x},\mathtt{h}_1))$

Together, conditions 2 and 3 capture the preservation of the full object layout for all surviving objects: condition 2 ensures that the pointer structure of the reachable subgraph is maintained, accounting for mutator-induced additions and deletions, while condition 3 ensures that the non-pointer data fields of those objects are left untouched. Combined with condition 1, which guarantees heap well-formedness, the three conditions jointly assert that the GC correctly reclaims unreachable memory without corrupting the structure or content of any live object. Observe that the correctness specification of the incremental mark-and-sweep GC is the same as Definition 4.5, except for the change summary from the mutator.

The incremental mark-and-sweep GC does not require building its correctness proofs from scratch. Since each GC slice is simply a sequence of atomic `mark_body` and `sweep_body` steps, the correctness proofs already established for the stop-the-world collector apply directly here. The key observation is that `new_allocs`, `added_edges`, and `deleted_edges` remain fixed throughout a single GC slice, so the behaviour of `mark_body` and `sweep_body` within a slice is indistinguishable from their behaviour in the stop-the-world setting.

Concretely, proof reuse would work as follows. Each GC slice is modelled as a loop over `mark_body` or `sweep_body`, treating `new_allocs`, `added_edges`, and `deleted_edges` as ghost variables that do not change during the loop. The existing lemmas for `mark_body` and `sweep_body` can then be applied at each loop iteration to show that the tricolor invariants are maintained. Finally, the slice-level results are composed with the mutator

change summary to obtain a full-cycle correctness theorem that connects back to the incremental correctness definition (Definition 5.1). We leave the concrete realisation of this proof reuse as a direction for future work, building directly on the proof infrastructure developed in Chapter 4.

5.4.2 Copying collector

In the context of a copying collector, the heap is divided into two disjoint spaces, namely `from_space` and `to_space`. The goal of the copying collector is to copy all the reachable and only the reachable objects from the `from_space` to the `to_space`. Earlier works (Myreen, 2010) have proved the correctness of a standalone copying collector (albeit with a different object layout than OCaml), and we simply adapt their correctness specifications in our framework.

Let $\mathtt{h}_0$ be the state of `from_space` of the heap on which the collector operates, such that $\omega(\mathtt{h}_0)$ holds, and let `r` be the set of *roots*, which are pointers to objects into $\mathtt{h}_0$. Let $\mathtt{G}(\mathtt{h}_0).\mathtt{V}$ be the vertex-set and $\mathtt{G}(\mathtt{h}_0).\mathtt{E}$ be the edge-set of $\mathtt{G}(\mathtt{h}_0)$. Let $\mathtt{h}_1$ be the state of the `to_space` of the heap after the GC terminates. And let $\mathtt{G}(\mathtt{h}_1).\mathtt{V}$ be the vertex-set and $\mathtt{G}(\mathtt{h}_1).\mathtt{E}$ be the edge-set of $\mathtt{G}(\mathtt{h}_1)$. Note that in a copying collector, the `to_space` is empty at the beginning of the GC.

Let $\mathtt{RG}(\mathtt{h}_0,\mathtt{r})$ be the reachable sub-graph residing in $\mathtt{G}(\mathtt{h}_0)$. Let `f` be a one-to-one mapping function which maps objects in the `from_space` to objects in the `to_space`. For a set s, we define $f_S(s)$ as the set obtained by applying f to every element in s. For a graph g, we define $f_G(g)$ as the graph obtained by applying f to every object in vertex set $g.V$ and to the components of the pair in the edge set $g.E$.

Definition 5.2 (**Correctness of Copying Collector**)**.** The copying collector is said to be correct if the following conditions hold:

1. $\omega(h_1)$

2. $G(h_1) = f_{\mathcal{G}}(RG(h_0, r))$, where $G(h_1).V = f_{\mathcal{S}}(RG(h_0, r).V)$ and
   $(\forall x_1, y_1.\ x_1 \in G(h_1).V \wedge y_1 \in G(h_1).V \wedge (x_1, y_1) \in G(h_1).E) \iff$
   $(\exists x_0, y_0.\ x_0 \in RG(h_0, r).V \wedge y_0 \in RG(h_0, r).V \wedge x_1 = f(x_0) \wedge y_1 = f(y_0) \wedge$
   $(x_0, y_0) \in G(h_0).E)$
3. $\forall x_1.\ x_1 \in G(h_1).V \iff$
   $(\exists x_0.\ x_0 \in RG(h_0, r).V \wedge x_1 = f(x_0) \wedge data(x_0, h_0) = data(x_1, h_1))$

The copying collector definition shares the same abstract framework as the mark-and-sweep correctness specification in Section 4.2 because both use the reachability predicate $RG(h_0, r)$ to identify the set of live objects, both require well-formedness of the post-GC heap, and both require data fields of surviving objects to be preserved. These shared elements confirm that the abstract graph-theoretic framework is reusable across collector designs.

However, the copying collector introduces a fundamental structural difference through the mapping function f. In the mark-and-sweep setting, live objects remain at their original addresses and the correctness condition is simply that the reachable subgraph is preserved in place. In the copying collector, objects are relocated, so correctness must be stated as a graph isomorphism: the heap after collection is not the reachable subgraph itself, but rather its image under f. This changes the proof structure significantly. Mark-and-sweep proofs reason about invariants on a fixed set of memory locations, whereas copying collector proofs must additionally establish that f is well-defined and one-to-one, that all pointer fields are correctly updated to reflect the new addresses, and that the isomorphism is preserved throughout the copying process. As noted earlier, this is why the copying collector will require substantial re-engineering at the implementation layer, even though the top-level specification shares the same abstract foundation.

In the context of OCaml 4 runtime system, the copying collector is used for collecting the minor heap. The `from_space` will be the minor heap, while the `to_space` would be

the major heap. Since we already have a verified major collector, a verified copying collector for the minor heap can be incorporated fairly independently. The only subtlety is that for the minor collection, the pointers from the major heap to the minor heap must be included in the root set of the minor collection. This is ensured by the mutator, which maintains a remembered set of pointers from the major heap to the minor heap. Crucially, this is an expectation on the mutator and not the collector. However, unlike the incremental mark-and-sweep GC, where significant parts of the layer 2 proofs may be reused, we anticipate that the copying collector will require significant re-engineering in layer 2 as the copying collector algorithm is quite different from a mark-and-sweep GC.

5.5 FUTURE DIRECTIONS

The verification developed in this thesis establishes a solid foundation, but it also makes clear how much remains to be done before a fully verified OCaml runtime becomes a reality. The work has been deliberately scoped to the stop-the-world major collector, and each boundary of that scope points naturally to the next challenge. The directions below follow a logical progression, starting from making the existing verification more maintainable, then extending it to the remaining components of OCaml 4.14.1, and finally scaling it to the concurrent setting of OCaml 5.0 and towards practical adoption.

Modular architecture for verified GC. The most immediate priority is redesigning the verification architecture to be parametric with respect to object semantics. Concretely, this means abstracting the traversal predicate so that new object types such as closures, infix objects, and custom tags can be added by providing a new instance of the abstraction rather than modifying core proofs. This is achievable within the F*/Low* framework and would directly address the proof maintenance problem observed in this thesis.

Incremental and concurrent collectors. The natural next step is extending the verified collector to the incremental mark-and-sweep design used in OCaml 4.14.1 in production. Abstract correctness specifications for both the incremental mark-and-sweep and copying

collectors are developed in Section 5.4 as a foundation for this effort. The remaining work is to complete their full verification, including the slice budget mechanism and the snapshot-at-the-beginning write barrier invariant. Beyond that, verifying a concurrent collector for OCaml 5.0 requires reasoning about idempotent concurrent marking, per-domain mark stacks, and the VCGC color scheme, for which the Pulse framework (Ebner *et al.*, 2025) provides a promising pathway. Encouragingly, recent work by the F* team has shown that our verified GC can be translated into Pulse (Ioannidis and Swamy, 2026) with the help of large language models, opening a promising direction through *agentic proof-oriented programming* (Swamy, 2026).

Verified minor collector, allocator, and generational integration. A production-ready verified runtime requires verifying the remaining components of OCaml 4.14.1's memory subsystem. The minor heap collector is a stop-the-world copying collector that introduces relocation proofs and forwarding-pointer reasoning. Verifying the allocator would close the most significant gap in the current trusted computing base. Beyond verifying each component in isolation, a complete generational GC verification must also address the interaction between the minor and major collectors, specifically the correctness of the remembered set, which records pointers from the major heap into the minor heap, and the write barrier which maintains it. Together, these components ensure that minor collections can be performed without tracing the entire heap. Verifying this interaction is the final step towards a fully verified OCaml 4.14.1 runtime. Section 5.4 already takes a step in this direction by developing abstract correctness specifications for both the incremental mark-and-sweep and copying collectors, though their full verification remains future work.

Practical adoption. Integrating the verified collector into the mainline OCaml runtime would require performance parity with the existing collector across a wider benchmark suite, compatibility with the full OCaml object model including features not currently handled, and acceptance by the OCaml core team. The benchmarks presented in this

thesis demonstrate competitive performance, which is a promising first step.

5.6 FINAL REFLECTION

Designing and developing a correct-by-construction garbage collector requires not only a deep understanding of how runtime components interact, but also strong expertise in formal verification. The process of formalisation forces one to make assumptions and invariants explicit, reveals hidden dependencies, and deepens understanding of the system's core structure.

Our experience from this work suggests a broader lesson:

> *To scale verification to complex systems, we must design not only for efficient execution, but also for modular reasoning.*

In conclusion, this thesis demonstrates that verified runtime components are both feasible and practical. The natural next step is to make such systems easier to build, extend, and maintain, enabling verification to become a standard part of systems development rather than an exceptional effort.

REFERENCES

1. **Anand, A.**, **A. Appel**, **G. Morrisett**, **Z. Paraskevopoulou**, **R. Pollack**, **M. Sozeau**, **B. Spitters**, and **M. Weaver**, CertiCoq: A verified compiler for Coq. *In Workshop on Coq for Programming Languages (CoqPL)*. 2017.

2. **Appel, A. W.**, **R. Dockins**, **A. Hobor**, **L. Beringer**, **J. Dodds**, **G. Stewart**, **S. Blazy**, and **X. Leroy**, *Program Logics for Certified Compilers*. Cambridge University Press, 2014.

3. **Appel, A. W.** and **D. A. Naumann**, Verified sequential Malloc/Free. *In International Symposium on Memory Management (ISMM)*. ACM, 2020.

4. **Barnes, N.** and **OCaml Development Team** (2024). Issue 14123: Fix integer-overflow problems in heap compaction. https://github.com/ocaml/ocaml/pull/14123. Fixes memory corruption in parallel GC on 32-bit and large 64-bit heaps. Fixed in OCaml 5.3.0.

5. **Barnett, M.**, **B.-Y. E. Chang**, **R. DeLine**, **B. Jacobs**, and **K. R. M. Leino**, Boogie: A modular reusable verifier for object-oriented programs. *In Formal Methods for Components and Objects (FMCO)*, volume 4111 of *Lecture Notes in Computer Science*. Springer, 2005.

6. **Berdine, J.**, **C. Calcagno**, and **P. W. O'Hearn**, Smallfoot: Modular automatic assertion checking with separation logic. *In Formal Methods for Components and Objects (FMCO)*, volume 4111 of *Lecture Notes in Computer Science*. Springer, 2005.

7. **Bertot, Y.** and **P. Castéran**, *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*. Springer Science & Business Media, 2013.

8. **Bhargavan, K.**, **B. Bond**, **A. Delignat-Lavaud**, **C. Fournet**, **C. Hawblitzel**, **C. Hritcu**, **S. Ishtiaq**, **M. Kohlweiss**, **R. Leino**, **J. Lorch**, *et al.*, Everest: Towards a verified, drop-in replacement of https. *In 2nd Summit on Advances in Programming Languages (SNAPL 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2017*a*.

9. **Bhargavan, K.**, **C. Fournet**, **M. Kohlweiss**, **A. Pironti**, **P.-Y. Strub**, and **S. Zanella-Béguelin**, Implementing and proving the tls 1.3 record layer. *In 2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017*b*.

10. **Birkedal, L.**, **N. Torp-Smith**, and **J. C. Reynolds**, Local reasoning about a copying garbage collector. *In Principles of Programming Languages (POPL)*. ACM, 2004.

11. **Boehm, H.-J.** and **M. Weiser** (1988). Garbage collection in an uncooperative environment. *Software: Practice and Experience*, **18**(9), 807–820.

12. **Burdy, L.**, B vs. coq to prove a garbage collector. *In the 14th International Conference on Theorem Proving in Higher Order Logics: Supplemental Proceedings*. 2001.

13. **Calcagno, C.**, **D. Distefano**, **J. Dubreil**, **D. Gabi**, **P. Hooimeijer**, **M. Luca**, **P. W. O'Hearn**, **I. Papakonstantinou**, **J. Purbrick**, and **D. Rodriguez**, Moving fast with software verification. *In NASA Formal Methods Symposium*, volume 9058 of *Lecture Notes in Computer Science*. Springer, 2015.

14. **Cao, Q.**, **L. Beringer**, **S. Gruber**, **J. Dodds**, and **A. W. Appel** (2018). VST-Floyd: A separation logic tool to verify correctness of C programs. *Journal of Automated Reasoning*, **61**(1–4), 367–422.

15. **Chen, R.**, **C. Cohen**, **J.-J. Lévy**, **S. Merz**, and **L. Théry**, Formal proofs of tarjan's strongly connected components algorithm in why3, coq and isabelle. *In ITP 2019-10th International Conference on Interactive Theorem Proving*, volume 141. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019.

16. **Cisco Systems** (2015). Cisco IOS and IOS XE Software TCP Packet Memory Leak Vulnerability. Cisco Security Advisory `cisco-sa-20150325-tcpleak`. URL https://sec.cloudapps.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-20150325-tcpleak. CVE-2015-0646.

17. **Cohen, E.**, **M. Dahlweid**, **M. Hillebrand**, **D. Leinenbach**, **M. Moskal**, **T. Santen**, **W. Schulte**, and **S. Stephan**, VCC: A verifier for concurrent C. *In Theorem Proving in Higher Order Logics (TPHOLs 2009)*. Springer, 2009*a*.

18. **Cohen, E.**, **M. Dahlweid**, **M. Hillebrand**, **D. Leinenbach**, **M. Moskal**, **T. Santen**, **W. Schulte**, and **S. Tobies**, Vcc: A practical system for verifying concurrent c. *In Theorem Proving in Higher Order Logics: 22nd International Conference, TPHOLs 2009, Munich, Germany, August 17-20, 2009. Proceedings 22*. Springer, 2009*b*.

19. **Coupet-Grimal, S.** and **L. Jakubiec**, Coq and hardware verification: A case study. *In Theorem Proving in Higher Order Logics (TPHOLs)*. 2003. Also related work: Coupet-Grimal, Formalisation of Ben-Ari's GC algorithm using linear temporal logic in Coq.

20. **Cuoq, P.**, **F. Kirchner**, **N. Kosmatov**, **V. Prevosto**, **A. Puccetti**, and **B. Yakobowski**, Frama-C: A software analysis perspective. *In Software Engineering and Formal Methods (SEFM 2012)*. Springer, 2012.

21. **de Moura, L.** and **N. Bjørner**, Z3: An efficient SMT solver. *In Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *Lecture Notes in Computer Science*. Springer, 2008.

22. **de Moura, L.**, **S. Kong**, **J. Avigad**, **F. Van Doorn**, and **J. von Raumer**, The lean theorem prover (system description). *In International Conference on Automated Deduction*.

Springer, 2015.

23. **de Moura, L.** and **S. Ullrich** (2021). The lean 4 theorem prover and programming language. *International Conference on Automated Deduction*, 625–635.

24. **Dijkstra, E. W.** (1970). On the reliability of programs. EWD303, E.W. Dijkstra Archive. URL https://www.cs.utexas.edu/~EWD/transcriptions/EWD03xx/EWD303.html.

25. **Doczkal, C.** and **D. Pous** (2020). Graph theory in Coq: Minors, treewidth, and isomorphisms. *Journal of Automated Reasoning*, **64**(5), 795–825.

26. **Doligez, D.** and **OCaml Development Team** (2014). Issue 6481: Major GC sweep phase corrupts free list in rare cases. https://github.com/ocaml/ocaml/issues/6481. Found in OCaml 4.01.0; fixed in 4.02.0.

27. **Durumeric, Z.**, **F. Li**, **J. Kasten**, **J. Amann**, **J. Beekman**, **M. Payer**, **N. Weaver**, **D. Adrian**, **V. Paxson**, **M. Bailey**, and **J. A. Halderman**, The matter of heartbleed. Association for Computing Machinery, New York, NY, USA, 2014. ISBN 9781450332132. URL https://doi.org/10.1145/2663716.2663755.

28. **Ebner, G.**, **G. Martínez**, **A. Rastogi**, **T. Dardinier**, **M. Frisella**, **T. Ramananandro**, and **N. Swamy** (2025). Pulsecore: An impredicative concurrent separation logic for dependently typed programs. *Proceedings of the ACM on Programming Languages*, **9**(PLDI), 1516–1539.

29. **Eichin, M. W.** and **J. A. Rochlis** (1989). With microscope and tweezers: the internet worm program: an analysis. *ACM SIGCOMM Computer Communication Review*, **19**(2), 326–345. URL https://dl.acm.org/doi/10.1145/66093.66095.

30. **Ericsson, A. S.**, **M. O. Myreen**, and **J. Å. Pohjola** (2019). A Verified Generational Garbage Collector for CakeML. *Journal of Automated Reasoning*, **63**(2), 463–488.

31. **Feng, X.**, **Z. Shao**, **A. Vaynberg**, **S. Xiang**, and **Z. Ni** (2006). Modular verification of assembly code with stack-based control abstractions. *ACM SIGPLAN Notices*, **41**(6), 401–414.

32. **Filliâtre, J.-C.** and **A. Paskevich**, Why3—where programs meet provers. *In Programming Languages and Systems: 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings 22*. Springer, 2013.

33. **Fromherz, A.**, **D. Ahman**, **G. Martínez**, **C. Fournet**, **A. Rastogi**, and **N. Swamy**, Steel: Proof-oriented programming in a dependently typed concurrent separation logic. *In Proceedings of the ACM on Programming Languages*, volume 5. ACM, 2021.

34. **Gammie, P.**, **A. L. Hosking**, and **K. Engelhardt** (2015). Relaxing safely: verified

on-the-fly garbage collection for x86-tso. *ACM SIGPLAN Notices*, **50**(6), 99–109.

35. **Goguen, H.**, **R. Brooksby**, and **R. Burstall** (1998). An abstract formulation of memory management.

36. **GOLANG-22350** (2017). Issue 22350: cmd/compile: liveness map is incorrect for some code sequences. https://github.com/golang/go/issues/22350. Accessed: 2026-04-05.

37. **GOLANG-3636** (2019). Issue 36365: runtime: memory corruption in concurrent sweep and scavenge. https://github.com/golang/go/issues/36365. Accessed: 2026-04-05.

38. **Gonthier, G.**, Verifying the safety of a practical concurrent garbage collector. *In Computer Aided Verification: 8th International Conference, CAV'96 New Brunswick, NJ, USA, July 31–August 3, 1996 Proceedings 8*. Springer Berlin Heidelberg, 1996.

39. **Google Chrome Team** (2021). Stable channel update for desktop – CVE-2021-21206 and CVE-2021-37975. Google Chrome Releases Blog. URL https://chromereleases.googleblog.com/2021/10/stable-channel-update-for-desktop_7.html.

40. **Gouy, I.** (). The computer language benchmarks game. https://benchmarksgame-team.pages.debian.net/benchmarksgame/.

41. **Gu, R.**, **J. Koenig**, **T. Ramananandro**, **Z. Shao**, **X. N. Wu**, **S.-C. Weng**, **H. Zhang**, and **Y. Guo**, Deep specifications and certified abstraction layers. *In Principles of Programming Languages (POPL)*. ACM, 2015.

42. **Guéneau, A.**, **J.-H. Jourdan**, **A. Charguéraud**, and **F. Pottier**, Formal proof and analysis of an incremental cycle detection algorithm. *In Interactive Theorem Proving*, 141. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2019.

43. **Havelund, K.**, Mechanical verification of a garbage collector. *In Parallel and Distributed Processing: 11th IPPS/SPDP'99 Workshops Held in Conjunction with the 13th International Parallel Processing Symposium and 10th Symposium on Parallel and Distributed Processing San Juan, Puerto Rico, USA, April 12–16, 1999 Proceedings 13*. Springer, 1999.

44. **Hawblitzel, C.** and **E. Petrank** (2009). Automated verification of practical garbage collectors. *ACM SIGPLAN Notices*, **44**(1), 441–453.

45. **Hoare, C.** (1969). An axiomatic basis for computer programming. *Communications of the ACM*.

46. **Huelsbergen, L.** and **P. Winterbottom** (1998). Very concurrent mark-&-sweep garbage collection without fine-grain synchronization. *ACM SIGPLAN Notices*, **34**(3), 166–175.

47. **Hunt, G.**, **M. Aiken**, **M. Fähndrich**, **C. Hawblitzel**, **O. Hodson**, **J. Larus**, **S. Levi**, **B. Steensgaard**, **D. Tarditi**, **T. Wobber**, and **B. Zill** (2005). An overview of the Singularity project. Technical Report MSR-TR-2005-135, Microsoft Research.

48. **Ioannidis, L.** and **N. Swamy** (2026). pulse-verified-gc: A verified garbage collector in Pulse. https://github.com/FStarLang/pulse-verified-gc. GitHub repository, accessed April 2026.

49. **Jackson, P. B.**, Verifying a garbage collection algorithm. *In Theorem Proving in Higher Order Logics: 11th International Conference, TPHOLs' 98 Canberra, Australia September 27–October 1, 1998 Proceedings 11*. Springer, 1998.

50. **Jacobs, B.**, **J. Smans**, **P. Philippaerts**, **F. Vogels**, **W. Penninckx**, and **F. Piessens**, VeriFast: A powerful, sound, predictable, fast verifier for C and Java. *In NASA Formal Methods Symposium*, volume 6617 of *Lecture Notes in Computer Science*. Springer, 2011.

51. **JDK-8210708** (2019). Jdk-8210708: Use separate PreservedMarksSet for each GC worker to avoid contention and corruption. https://bugs.openjdk.org/browse/JDK-8210708. Accessed: 2026-04-05.

52. **JDK-8284632** (2022). Jdk-8284632: ZGC: C2 allows bypassing load barriers in array cloning. https://bugs.openjdk.org/browse/JDK-8284632. Accessed: 2026-04-05.

53. **Jiang, K.**, **D. Sanan**, **Y. Zhao**, **S. Kan**, and **Y. Liu**, A formally verified buddy memory allocation model. *In International Conference on Engineering of Complex Computer Systems (ICECCS)*. IEEE, 2019.

54. **Jones, R.**, **A. Hosking**, and **E. Moss**, *The Garbage Collection Handbook*. CRC Press, 2011.

55. **Jones, R.**, **A. Hosking**, and **E. Moss**, *The garbage collection handbook: the art of automatic memory management*. CRC Press, Boca Raton, FL, 2016.

56. **Jung, R.**, **R. Krebbers**, **J.-H. Jourdan**, **A. Bizjak**, **L. Birkedal**, and **D. Dreyer** (2018). Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, **28**, e20.

57. **Jung, R.**, **D. Swasey**, **F. Sieczkowski**, **K. Svendsen**, **A. Turon**, **L. Birkedal**, and **D. Dreyer**, Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. *In Principles of Programming Languages (POPL)*. ACM, 2015.

58. **Klein, G.**, **J. Andronick**, **K. Elphinstone**, **T. Murray**, **T. Sewell**, **R. Kolanski**, and **G. Heiser** (2014). Comprehensive formal verification of an OS microkernel. *ACM Transactions on Computer Systems*, **32**(1), 2:1–2:70.

59. **Klein, G.**, **K. Elphinstone**, **G. Heiser**, **J. Andronick**, **D. Cock**, **P. Derrin**, **D. Elkaduwe**,

K. Engelhardt, **R. Kolanski**, **M. Norrish**, **T. Sewell**, **H. Tuch**, and **S. Winwood**, seL4: Formal verification of an OS kernel. *In ACM Symposium on Operating Systems Principles (SOSP)*. ACM, 2009.

60. **Kumar, R.**, **M. O. Myreen**, **M. Norrish**, and **S. Owens**, CakeML: A verified implementation of ML. *In Principles of Programming Languages (POPL)*. ACM, 2014.

61. **Lammich, P.** and **R. Neumann**, A framework for verifying depth-first search algorithms. *In Proceedings of the 2015 Conference on Certified Programs and Proofs*. 2015.

62. **Leino, K. R. M.**, Dafny: An automatic program verifier for functional correctness. *In International Conference on Logic for Programming Artificial Intelligence and Reasoning*. Springer, 2010.

63. **Leroy, X.** (2009). Formal verification of a realistic compiler. *Communications of the ACM*, **52**(7), 107–115.

64. **Leveson, N. G.** and **C. S. Turner** (1993). An investigation of the therac-25 accidents. **26**(7). ISSN 0018-9162. URL https://doi.org/10.1109/MC.1993.274940.

65. **Lin, C.**, **Y. Chen**, and **B. Hua** (2009). Verification of an incremental garbage collector in hoare-style logic. *Int. J. Softw. Informatics*, **3**(1), 67–88.

66. **Lin, C.-X.**, **Y.-Y. Chen**, **L. Li**, and **B. Hua** (2007). Garbage collector verification for proof-carrying code. *Journal of Computer Science and Technology*, **22**(3), 426–437.

67. **Lions, J.-L.**, *Ariane 5 Flight 501 Failure: Report of the Inquiry Board*. European Space Agency, Paris, France, 1996. URL https://www.esa.int/esapub/bulletin/bullet87/inbrie87.htm.

68. **Madhavapeddy, A.** and **Y. Minsky**, *Real World OCaml: Functional Programming for the Masses*. Cambridge University Press, 2022*a*. ISBN 9781009125802. URL https://www.cambridge.org/core/books/real-world-ocaml-functional-programming-for-the-masses/052E4BCCB09D56A0FE875DD81B1ED571. 2nd ed., includes coverage of OCaml tools, testing, runtime, and memory representation.

69. **Madhavapeddy, A.** and **Y. Minsky**, *Real World OCaml: Functional Programming for the Masses*. Cambridge University Press, Cambridge, 2022*b*.

70. **Marti, N.**, **R. Affeldt**, and **A. Yonezawa**, Formal verification of the heap manager of an operating system using separation logic. *In Workshop on Semantics, Program Analysis, and Computing Environments for Memory Management (SPACE)*. 2006.

71. **Martin, C.**, **A. Sejdic**, and **G. Piper**, From disruption to insight: Crowdstrike's impact on client services and security. Association for Computing Machinery, New York, NY, USA,

2025. ISBN 9798400710605. URL https://doi.org/10.1145/3675229.3712568.

72. **Martínez, G.**, **D. Ahman**, **V. Dumitrescu**, **N. Giannarakis**, **C. Hawblitzel**, **C. Hriţcu**, **M. Narasimhamurthy**, **Z. Paraskevopoulou**, **C. Pit-Claudel**, **J. Protzenko**, *et al.*, Meta-f: Proof automation with smt, tactics, and metaprograms. *In Programming Languages and Systems: 28th European Symposium on Programming, ESOP 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019, Prague, Czech Republic, April 6–11, 2019, Proceedings*. Springer International Publishing Cham, 2019.

73. **McCreight, A.**, **Z. Shao**, **C. Lin**, and **L. Li**, A general framework for certifying garbage collectors and their mutators. *In Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 2007.

74. **Mccreight, A. E.** (2008). The mechanized verification of garbage collector implementations.

75. **Mehta, F.** and **T. Nipkow**, Proving pointer programs in higher-order logic. *In Conference on Automated Deduction (CADE)*, volume 2741 of *Lecture Notes in Artificial Intelligence*. Springer, 2003.

76. **Meta Platforms, I.** (2019). Whatsapp voip stack buffer overflow vulnerability. https://www.whatsapp.com/security/advisories/archive. CVE-2019-3568.

77. **Mozilla Foundation** (2014). Security advisory: CVE-2014-1564 – uninitialized memory with truncated images in Firefox. Mozilla Foundation Security Advisory. URL https://bugzilla.mozilla.org/show_bug.cgi?id=1045977. Fixed in Firefox 32.

78. **Myreen, M. O.**, Reusable verification of a copying collector. *In International Conference on Verified Software: Theories, Tools, and Experiments*. Springer, 2010.

79. **National Vulnerability Database (NVD)** (2014). CVE-2014-1776 - Use-after-free vulnerability in Microsoft Internet Explorer 6 through 11. https://nvd.nist.gov/vuln/detail/CVE-2014-1776. Accessed on 2026-04-05.

80. **Neis, G.**, **I. Sergey**, and **N. Polikarpova** (2021). Certifying the synthesis of heap-manipulating programs. *Proceedings of the ACM on Programming Languages (OOPSLA)*, **5**.

81. **Nipkow, T.**, **L. C. Paulson**, and **M. Wenzel**, *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283. Springer Science & Business Media, 2002.

82. **Noschinski, L.** (2015). A graph library for Isabelle. *Mathematics in Computer Science*, **9**(1), 23–39.

83. **O'Hearn, P. W.**, **J. C. Reynolds**, and **H. Yang**, Local reasoning about programs that alter data structures. *In Computer Science Logic (CSL)*, volume 2142 of *Lecture Notes in*

Computer Science. Springer, 2001.

84. **Pavlovic, D.**, **P. Pepper**, and **D. R. Smith**, Formal derivation of concurrent garbage collectors. *In Mathematics of Program Construction (MPC)*, volume 6120 of *Lecture Notes in Computer Science*. Springer, 2010.

85. **Piskac, R.**, **T. Wies**, and **D. Zufferey**, GRASShopper – complete heap verification with mixed specifications. *In Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 8413 of *Lecture Notes in Computer Science*. Springer, 2014.

86. **Polikarpova, N.** and **I. Sergey**, Structuring the synthesis of heap-manipulating programs. *In Principles of Programming Languages (POPL)*. ACM, 2019.

87. **Pottier, F.**, Depth-first search and strong connectivity in Coq. *In Journées Francophones des Langages Applicatifs (JFLA)*. 2015.

88. **Protzenko, J.**, **B. Parno**, **A. Fromherz**, **C. Hawblitzel**, **M. Polubelova**, **K. Bhargavan**, **B. Beurdouche**, **J. Choi**, **A. Delignat-Lavaud**, **C. Fournet**, *et al.*, Evercrypt: A fast, verified, cross-platform cryptographic provider. *In 2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020.

89. **Protzenko, J.**, **J.-K. Zinzindohoué**, **A. Rastogi**, **T. Ramananandro**, **P. Wang**, **S. Zanella-Béguelin**, **A. Delignat-Lavaud**, **C. Hriţcu**, **K. Bhargavan**, **C. Fournet**, and **N. Swamy** (2017). Verified low-level programming embedded in F*. *Proc. ACM Program. Lang.*, **1**(ICFP). URL https://doi.org/10.1145/3110261.

90. **Ramananandro, T.**, **A. Delignat-Lavaud**, **C. Fournet**, **N. Swamy**, **T. Chajed**, **N. Kobeissi**, and **J. Protzenko**, {EverParse}: Verified secure {Zero-Copy} parsers for authenticated message formats. *In 28th USENIX Security Symposium (USENIX Security 19)*. 2019.

91. **Reitz, A.**, **A. Fromherz**, and **J. Protzenko** (2024*a*). Starmalloc: Verifying a modern, hardened memory allocator. *Proc. ACM Program. Lang.*, **8**(OOPSLA2). URL https://doi.org/10.1145/3689773.

92. **Reitz, A.**, **A. Fromherz**, **T. Ramananandro**, **A. Rastogi**, **N. Swamy**, and **C. Fournet** (2024*b*). StarMalloc: Verifying a modern, hardened memory allocator. *Proceedings of the ACM on Programming Languages (OOPSLA)*, **8**.

93. **Reynolds, J. C.**, Separation logic: A logic for shared mutable data structures. *In Logic in Computer Science (LICS)*. IEEE, 2002.

94. **Russinoff, D. M.** (1994). A mechanically verified incremental garbage collector. *Formal Aspects of Computing*, **6**(4), 359–390.

95. **Sewell, P.**, **S. Sarkar**, **S. Owens**, **F. Z. Nardelli**, and **M. O. Myreen** (2010). x86-TSO: A rigorous and usable programmer's model for x86 multiprocessors. *Communications of*

the ACM, **53**(7), 89–97.

96. **Shamsu, S.**, **D. Kafle**, **D. Maroo**, **K. Nagar**, **K. Bhargavan**, and **K. Sivaramakrishnan** (2025). A mechanically verified garbage collector for OCaml. **69**(2). ISSN 0168-7433. URL https://doi.org/10.1007/s10817-025-09721-0.

97. **Sivaramakrishnan, K.**, **S. Dolan**, **L. White**, **S. Jaffer**, **T. Kelly**, **A. Sahoo**, **S. Parimala**, **A. Dhiman**, and **A. Madhavapeddy** (2020). Retrofitting parallelism onto ocaml. *Proceedings of the ACM on Programming Languages*, **4**(ICFP), 1–30.

98. **Swamy, N.** (2026). Agentic proof-oriented programming. https://risemsr.github.io/blog/2026-02-04-nik-agentic-pop/. RiSE MSR Blog. Accessed: 2026-05-03.

99. **Swamy, N.**, **C. Hriţcu**, **C. Keller**, **A. Rastogi**, **A. Delignat-Lavaud**, **S. Forest**, **K. Bhargavan**, **C. Fournet**, **P.-Y. Strub**, **M. Kohlweiss**, *et al.*, Dependent types and multi-monadic effects in f. *In Proceedings of the 43rd annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 2016.

100. **Tuch, H.** (2009). Formal verification of C systems code. *Journal of Automated Reasoning*, **42**(2–4), 125–187.

101. **Verma, K. N.**, **J. Goubault-Larrecq**, **S. Prasad**, and **S. Arun-Kumar**, Reflecting BDDs in Coq. *In Advances in Computing Science — ASIAN 2000*, volume 1961 of *Lecture Notes in Computer Science*. Springer, 2000.

102. **Wang, S.**, **Q. Cao**, **A. Mohan**, and **A. Hobor** (2019). Certifying Graph-Manipulating C Programs via Localizations within Data Structures. *Proc. ACM Program. Lang.*, **3**(OOPSLA). URL https://doi.org/10.1145/3360597.

103. **Xu, B.**, **E. Moss**, and **S. M. Blackburn**, Towards a model checking framework for a new collector framework. *In Proceedings of the 19th International Conference on Managed Programming Languages and Runtimes*. 2022.

104. **Yuasa, T.** (1990). Real-time garbage collection on general-purpose machines. *Journal of Systems and Software*, **11**(3), 181–198.

105. **Zakowski, Y.**, **D. Cachera**, **D. Demange**, **G. Petri**, **D. Pichardie**, **S. Jagannathan**, and **J. Vitek**, Verifying a concurrent garbage collector using a rely-guarantee methodology. *In Interactive Theorem Proving: 8th International Conference, ITP 2017, Brasília, Brazil, September 26–29, 2017, Proceedings 8*. Springer, 2017.

106. **Zinzindohoué, J.-K.**, **K. Bhargavan**, **J. Protzenko**, and **B. Beurdouche**, Hacl*: A verified modern cryptographic library. *In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2017.

CURRICULUM VITAE

NAME Sheera Shamsu

DATE OF BIRTH 27 March 1982

EDUCATION QUALIFICATIONS

2004 **B.Tech**

Institution	LBS College of Engineering
Specialization	CSE

2011 **M.Tech**

Institution	National Institute of Technology Calicut, Kerala
Specialization	CSE

Doctor of Philosophy

Institution	IIT Madras
Specialization	CSE
Registration Date	27/08/2020

DOCTORAL COMMITTEE

Chairperson

Dr. Sukhendu Das
Professor, Department of CSE
IIT Madras

Guide(s)

Dr. KC Sivaramakrishnan
Assistant Professor, Department of CSE
IIT Madras

Dr. Kartik Nagar
Assistant Professor, Department of CSE
IIT Madras

Member(s)

Dr. Rupesh Nasre
Professor, Department of CSE
IIT Madras

Dr. Chester Rebeiro
Professor, Department of CSE
IIT Madras

Dr. Nitin Chandrachoodan
Professor, Department of EE
IIT Madras